

ÍNDICE

Nota previa	5
Unidad 1 Ingeniería de software en contexto	6
Ingeniería del Software. ¿Qué comprende la ingeniería de software?	6
Software	6
Ingeniería de Software	6
Estado actual y antecedentes. La crisis del software.....	6
Disciplinas que conforman la ingeniería de software.....	6
Ejemplos de grandes proyectos de software fallidos y exitosos.....	6
Ciclos de vida y su influencia en la Administración de Proyectos de Software.....	6
Relación entre el ciclo de vida del proyecto y del producto	6
Clasificación de los ciclos de vida.....	7
Modelos de ciclo de vida	7
Administración de proyectos dependiendo en ciclo de vida.....	8
Procesos de Desarrollo Empíricos vs. Definidos	8
Procesos Definidos	9
Procesos Empíricos	9
Ciclos de vida (Modelos de Proceso) y Procesos de Desarrollo de Software.....	9
Criterios para elección de ciclos de vida para proyectos de desarrollo de software.....	9
Proyecto de Software	9
Características de un Proyecto.....	9
Administración de proyecto de software	10
La triple restricción – enfoque tradicional.....	10
Líder de Proyecto.....	10
Equipo de proyecto	11
Stakeholders.....	11
Plan de proyecto	11
Planes de Soporte.....	13
Factores para el Éxito de un Proyecto.....	13
Factores para el Fracaso de un Proyecto	13
Vínculo proceso-proyecto-producto en la gestión de un proyecto.....	14
No Silver Bullet.....	14
Dificultades Esenciales	14
Dificultades Accidentales.....	14
Resolución de Dificultades	14
Unidad 2 Gestión Lean-Ágil de Productos de Software.....	15
Gestión de Proyectos basados en procesos definidos.....	15
Planificación para el desarrollo y el mantenimiento de productos de software.....	15
Métricas y Estimaciones de Software	15
Manifiesto Ágil/Filosofía Lean	15
Objetivo del Enfoque Ágil	15
Manifiesto ágil.....	15
Filosofía Ágil	16
Pilares del empirismo.....	16

¿Cuándo es aplicable ágil?	17
El triángulo ágil	17
Filosofía Lean	17
Relación Lean-Ágil	18
Requerimientos en ambientes Lean-Agile	18
Fundamentación de los requerimientos ágiles	18
La triple restricción – Enfoque ágil	18
BRUF – Big Requirements Up Front	19
Product Backlog	19
Requerimientos Just in Time	19
Fuente de requerimientos	19
Tipos de requerimientos	20
Principios ágiles relacionados con requerimientos ágiles	20
User Stories	20
Investment Themes (Temas de Inversión)	23
Estimación	24
Proceso de estimación	24
Métodos utilizados para estimar	24
Problemas de estimaciones tradicionales	25
Estimaciones en ambientes ágiles	25
¿Qué se estima?	25
Tamaño	25
Velocidad	26
Reestimación	26
Propósito de la Estimación en Desarrollo Ágil	26
Estimación de la Duración del Proyecto	26
Poker Planning o Poker Estimation	26
Planificación	27
Estimaciones en Lean	27
Estimaciones en proyectos pequeños	27
Diferencias con las estimaciones en ambientes tradicionales	27
SCRUM	28
Valores de Scrum	28
El equipo Scrum (Scrum Team)	28
Eventos de Scrum	29
Artefactos de Scrum	30
Timebox	30
Capacidad del equipo en un Sprint	30
Niveles de Planificación	31
Herramientas de SCRUM	31
Frameworks para escalar SCRUM	31
NEXUS	32
Responsabilidades	32
Eventos	32
Artefactos	32

LeSS	33
Roles	33
Eventos.....	33
Artefactos	34
LeSS Huge	34
KANBAN	34
Principios fundacionales.....	34
Principios directores.....	35
Prácticas Generales de Kanban	35
Cómo aplicar Kanban.....	36
MÉTRICAS.....	37
Mediciones en el software	37
Integración de Métricas	38
Métricas en el Enfoque Tradicional	38
Métricas en Ambientes Ágiles.....	39
Métricas en Kanban	39
Métricas orientadas a servicio	40
Gestión de productos de software.....	40
EVOLUCIÓN DE LOS PRODUCTOS DE SOFTWARE.....	40
Planificación del producto ágil	40
Design Thinking	43
Unidad 3 Gestión del Software como producto	44
Conceptos introductorios de la gestión de configuración	44
Gestión de Configuración	44
Conceptos Generales	45
ÍTEM DE CONFIGURACIÓN.....	45
REPOSITORIO	45
VERSIÓN	45
VARIANTE	45
CONFIGURACIÓN DE SOFTWARE	46
LINEA BASE	46
RAMA (BRANCH)	46
Planificación de la SCM.....	46
ACTIVIDADES RELACIONADAS A LA SCM.....	46
Gestión de configuración en ambientes ágiles	48
Continuous integration	48
Continuous Delivery	48
Continuous Deployment	48
Unidad 4 Aseguramiento de calidad de Proceso y de Producto	50
Conceptos generales sobre calidad	50
Calidad	50
Gestión y aseguramiento de calidad.....	50
Importancia de trabajar para y con Calidad.	51
Principios de Aseguramiento de Calidad.....	51
Ventajas y Desventajas	51

Visiones de Calidad - ¿Calidad para quién?	51
Calidad en el Software	52
Calidad de Producto.....	52
Calidad en el Proceso	53
Actividades relacionadas con el Aseguramiento de la Calidad del Software.	54
Principales Modelos de Calidad existentes (CMMI – SPICE – ISO).....	54
Modelos de Mejora de Procesos	54
Modelos de Calidad (para evaluar procesos).....	55
Intereses Fundamentales en la Gestión de Calidad	57
CMMI Cara a Cara con Ágil.....	57
Diferencias entre CMMI y Agile	58
Similitudes entre CMM y Métodos Ágiles.....	59
AUDITORÍAS DE SOFTWARE	59
Tipos de Auditorías.....	59
Roles en una Auditoría	60
Proceso de Auditorías	60
Calidad de Producto	62
Verificación y Validación	62
Principios	63
Revisiones Técnicas – Peer Review	64
Testing en ambientes Ágiles	65
Definición	65
Principios del Testing.....	66
Mitos del Testing	66
Hasta dónde testear – ¿Cuánto Testing es suficiente?.....	67
Conceptos Importantes	67
Artefactos de Testing.....	69
El Testing en el Ciclo de Vida del Software	69
Métodos/estrategias de Prueba	70
Tipos de Prueba.....	71
TDD – Test Driven Development	71
ATDD - Acceptance Test-Driven Development.....	72
Manifiesto de testing	72
Testing tradicional vs. Testing agile.....	73
Prácticas de Testing Ágil	73
Desafíos Culturales en la Transición de Equipos de Prueba Tradicionales a Ágiles.....	74
Cuadrantes del testing	74
Pirámide del testing.....	74
Roles y competencias del tester	75
Ser ágil y hacer ágil.....	75

RESUMEN FINAL ISW

NOTA PREVIA

Esta nota la escribo antes de hacer este resumen, que espero me queme pocas neuronas porque todavía me falta rendir el final. Mi idea es hacer un resumen lo más conciso que se pueda, sin tantas vueltas, tomando data de otros resúmenes que tengo a mano y libros de la catedra (un par, ni loca leo todo eso), para cubrir TODOS los temas del plan 2023. A mí la materia genuinamente me gusta (veremos qué digo después de hacer este resumen) así que le voy a poner amor, no sé hasta donde vaya a llegar este resumen, pero, así rindas un parcial o final, éxitos, espero te sirva y no recuses :).

Aviso de antemano que hay comentarios randoms a lo largo del resumen, ¿Por qué? Me divierte y se me hace más leve cuando lo tengo que leer de nuevo, así que nada, no pregunten y gocen (?).

Como dijo Judith: "No se reciban con esta materia", ya falta poco y hay que sacársela rápido. Sin más preámbulo, let's go.

UNIDAD 1 | INGENIERÍA DE SOFTWARE EN CONTEXTO

INGENIERÍA DEL SOFTWARE. ¿QUÉ COMPRENDE LA INGENIERÍA DE SOFTWARE?

SOFTWARE

El **software** abarca **programas, documentación esencial y herramientas** empleadas en la **creación, mantenimiento y entrega** de programas ejecutables al cliente. Esto incluye archivos de configuración, documentos para usuarios, descripciones de la estructura del sistema y las herramientas utilizadas en su construcción.

Se puede conceptualizar también como **información** que se presenta en diversos **niveles de abstracción**, desde los **requerimientos** del sistema **hasta el código** ejecutable. (*Definición que le gusta a Judith*).

INGENIERÍA DE SOFTWARE

La Ingeniería de Software abarca **todas las fases de producción del software**, desde la **especificación inicial** hasta el **mantenimiento del sistema** en operación.

Su importancia radica en la necesidad de producir sistemas confiables de manera **rápida y económica**, ya que modificar el software después de su implementación resulta costoso a largo plazo.

Esta disciplina se basa en **comprender a fondo el problema** y las **necesidades** del cliente **antes de proponer soluciones**, destacando la **importancia del diseño** para lograr **calidad y facilidad de mantenimiento**.

Surgió en respuesta a las **dificultades y errores en la planificación, costos, productividad y calidad** del software durante la llamada **"crisis del software"**.

ESTADO ACTUAL Y ANTECEDENTES. LA CRISIS DEL SOFTWARE.

La **"crisis del software"** se refiere a desafíos planteados en la conferencia de la OTAN en 1968 por Friedrich Bauer, destacando la **dificultad para desarrollar software libre de defectos, comprensible y verificable**. Sin embargo, este término fue utilizado anteriormente por Dijkstra en El Humilde Programador (JAJAJAJ ke).

Factores como la **evolución del hardware** sin un desarrollo equivalente en software (actividad humana cuyo producto es abstracto e intangible), la **subestimación de la complejidad del desarrollo de software** y la **falta de una disciplina integral** contribuyeron a esta crisis.

En la **actualidad**, muchos **sistemas fallan** debido a **demandas crecientes** (sistemas más grandes y complejos) y **bajas expectativas** (costoso y no fiable) en el desarrollo de software, con la necesidad de **nuevas técnicas de gestión y desarrollo**, así como una **mejor educación en ingeniería** de software para abordar estos problemas.

DISCIPLINAS QUE CONFORMAN LA INGENIERÍA DE SOFTWARE.

1. **Las disciplinas técnicas** en el desarrollo de software abarcan actividades clave desde la **toma de requerimientos** hasta la **implementación, prueba, despliegue, documentación y capacitación** a usuarios.
2. **Las disciplinas de gestión** se centran en la **planificación, monitoreo y control del proyecto**, utilizando metodologías como LEAN Agile.
3. **Las disciplinas de soporte**, como la **Gestión de Configuración del Software**, toma de **métricas y Aseguramiento de Calidad (QA)**, son cruciales para verificar la integridad y calidad del producto de software.

EJEMPLOS DE GRANDES PROYECTOS DE SOFTWARE FALLIDOS Y EXITOSOS.

Me preguntan esto y agarro la guitarra, pero por las dudas dejo unos links de otro resumen para pegarles una leída:

- [Proyectos exitosos](#)
- [Proyectos fallidos](#)

CICLOS DE VIDA Y SU INFLUENCIA EN LA ADMINISTRACIÓN DE PROYECTOS DE SOFTWARE.

Un **ciclo de vida** es una **representación simplificada de un proceso**, destacando elementos como **actividades, productos y tareas**, así como el **flujo de trabajo y la relación entre estos elementos** a lo largo del tiempo. Estos modelos son **genéricos**, estableciendo **criterios para avanzar** de una tarea a la siguiente, **independientes de los procedimientos específicos** de cada actividad.

Los **modelos de proceso** pueden compartir actividades, pero **enfatan diferentes aspectos y definen flujos de procesos** distintos. Además, los **ciclos de vida especifican las fases del proceso** (requerimientos, especificaciones, diseño, etc.) y **el orden** en el que se llevan a cabo.

RELACIÓN ENTRE EL CICLO DE VIDA DEL PROYECTO Y DEL PRODUCTO

La relación entre el ciclo de vida del proyecto y del producto es que **el ciclo de vida del producto siempre es más extenso** que el ciclo de vida del proyecto. Mientras el ciclo de vida del **proyecto se limita al desarrollo** del software, el ciclo de vida del **producto persiste hasta que el software deja de ser utilizado**, dependiendo de su madurez y las necesidades del mercado. **Un producto puede experimentar múltiples proyectos** a lo largo de su ciclo de vida debido a cambios y actualizaciones constantes, permitiendo así el desarrollo de **varios ciclos de vida** de proyectos dentro del ciclo de vida del producto.

CLASIFICACIÓN DE LOS CICLOS DE VIDA

Los ciclos de vida que se presentan a continuación NO son mutuamente excluyentes y con frecuencia se usan en conjunto, sobre todo para sistemas complejos.

SECUENCIAL

Este modelo organiza las **actividades de forma lineal, progresando a través de fases** ordenadas donde cada actividad depende de la finalización de la anterior.

- ✓ Desarrollo dirigido por un **plan**
- ✓ **Cada etapa genera documentación** para realizar un **monitoreo** constante contra el plan.
- ✓ **Simplicidad** en la utilización de un **método de gestión común** a lo largo de todo el proyecto.
- ✓ Efectividad en productos con **definición clara y estable**, y cuando se trabaja con metodologías técnicas claras.
- ⊗ Los **proyectos raramente siguen un flujo secuencial**.
- ⊗ Exigencia de **requerimientos** completamente **explícitos desde el principio**, esto no pasa xd.
- ⊗ Entrega de una **versión funcional** del producto **en etapas muy avanzadas** del proyecto.
- ⊗ **Dificultad en corregir defectos**, ya que implica un gran rediseño en una etapa avanzada.
- ⊗ **Dependencia entre equipos** de trabajo, uno no puede comenzar hasta que el otro haya finalizado.
- ⊗ **Burocracia y exceso de documentación** debido a un desarrollo dirigido por un plan y constante monitoreo.

ITERATIVO/INCREMENTAL

El Modelo Iterativo/Incremental emplea **iteraciones escalonadas** a lo largo del tiempo, **generando incrementos** de software funcional **potencialmente entregables** en cada ciclo. Inicialmente, **se priorizan las funciones básicas/críticas** del cliente. Este enfoque **integra las actividades de especificación, desarrollo y validación**, desarrollando el sistema en **versiones sucesivas**, donde cada una incorpora nuevas funcionalidades. Se utiliza comúnmente en **metodologías ágiles** (inserte brillitos) ✨.

- ✓ **Reduce el costo y retrabajo** al **adaptarse a cambios** en los requerimientos del cliente en cada incremento.
- ✓ **Incluye al cliente** en el proceso de desarrollo, permitiendo **retroalimentación temprana**.
- ✓ **Posibilita entregas más rápidas** y tempranas de software útil al cliente.
- ✓ Adecuado para sistemas con **requerimientos cambiantes**.
- ✓ Entrelaza **especificación, desarrollo y validación**.
- ⊗ **Invisibiliza el proceso**.
- ⊗ **Documentar** cada incremento conlleva un **alto costo**.
- ⊗ **La estructura** del software **puede degradarse frente a** una cantidad considerable de **incrementos**.

RECURSIVO (ESPIRAL)

El **modelo recursivo**, especialmente ejemplificado por el **modelo en espiral**, se emplea para **gestionar riesgos** en el desarrollo de **sistemas complejos a gran escala**, involucrando **activamente al cliente**.

- ⊗ Consta de **4 etapas recursivas**:
 1. **Determinar objetivos**
 2. **Identificar y resolver riesgos**
 3. **Desarrollar y probar**
 4. **Planificar la próxima iteración**
- ✓ **Proyecto dividido en mini proyectos**, abordando los **riesgos más relevantes en cada iteración** (hasta terminar)
- ✓ **Se inicia con algo completo**, presentando un **prototipo que mejora** con cada vuelta.
- ✓ Genera **productos independientes** de la implementación, **potencialmente reutilizables** en sistemas similares.
- ⊗ **Costoso en tiempo y dinero** readaptar productos para diferentes proyectos.
- ⊗ **Riesgo de obsolescencia tecnológica**. (O sea que se devalúa, como el peso)
- ⊗ Posible **falta de mantenimiento o documentación**, complicando las tareas.
- ⊗ **La versión del producto se genera al final**, presentando desafíos en la visibilidad del progreso.

MODELOS DE CICLO DE VIDA

CODE AND FIX

- Desarrollo sin especificaciones o diseño, ajustándolo hasta la satisfacción del cliente.
- Ventajas: No requiere planificación extensa ni documentación inicial. Apto para proyectos pequeños destinados a ser descartados.

MODELO EN CASCADA PURO

- Secuencia de pasos ordenada con revisión al final de cada fase.
- Ventajas: Detecta errores temprano. Exceso de documentación. No flexible, requiere especificar completamente los requerimientos al inicio.

MODELO EN CASCADA CON FASES SOLAPADAS

- Mayor solapamiento, menos documentación.
- Seguimiento de progreso más difícil.

MODELO DE ENTREGA POR ETAPAS

- Signos tangibles de progreso. Requiere una planificación rigurosa.

MODELO EN CASCADA CON RETROALIMENTACIÓN

- Posibilidad de volver a una etapa anterior, pero costoso.

MODELO EN CASCADA CON SUBPROYECTOS

- Avance en cascada hasta diseño arquitectónico, luego avance en partes divididas en subproyectos.

MODELO EN ESPIRAL

- Minimiza riesgos con análisis y evaluación en cada fase.
- Adecuado para proyectos largos con progreso visible al final de cada ciclo.

MODELO EVOLUTIVO

- Fija tiempo o alcance, permite cambios en el otro.
- Útil cuando los requerimientos no son claros, con entregas tempranas.

DISEÑO PARA CRONOGRAMA

- No asegura la versión final, pero entrega aspectos importantes en fecha definida.
- Útil cuando hay incertidumbre sobre capacidades de programación.

DISEÑO PARA HERRAMIENTAS

- Incluye solo funcionalidad compatible con herramientas existentes.
- Se usa en proyectos sensibles al tiempo, puede combinarse con otros ciclos de vida.

PROTOTIPACIÓN EVOLUTIVA

- Cambios rápidos de requerimientos, desarrollo del concepto mientras se avanza.
- Menos documentación, útil cuando no hay certeza sobre algoritmos o arquitectura óptima.

ADMINISTRACIÓN DE PROYECTOS DEPENDIENDO EN CICLO DE VIDA

La **elección del ciclo de vida** del proyecto **impacta directamente en la administración**, ya que **cada modelo de proceso tiene características y enfoques distintos**. Por ejemplo, si hay riesgo de cambios significativos en los requerimientos, se optaría por ciclos de vida iterativos. Esto afecta la administración y planificación del proyecto. Durante el desarrollo, la gestión del proyecto se adapta a esta elección, diferenciando la gestión de un proyecto iterativo de uno secuencial o en cascada.

PROCESOS DE DESARROLLO EMPÍRICOS VS. DEFINIDOS

Un proceso es una secuencia de pasos ejecutados para un propósito dado (IEEE), mientras que un **proceso de software** se define como un **conjunto de actividades, métodos, prácticas y transformaciones** que la gente usa para **desarrollar o mantener software** y sus productos asociados. Este proceso **implica actividades** como la **especificación de requerimientos, análisis, diseño, implementación y prueba**, cuyo orden y relación dependen del ciclo de vida elegido.

Tres factores clave influyen en el proceso:

1. **Procedimientos y métodos:** Tener definidos y escritos los procedimientos.
2. **Personas motivadas y capacitadas:** Las personas deben estar capacitadas y con habilidades para realizar sus tareas asignadas de la manera correcta y motivados, para lograr aún mayor eficiencia.
3. **Herramientas y equipos:** Materiales necesarios para llevar a cabo el proceso

La **adaptación de procesos** establecidos, la **documentación transparente** de actividades, la **habilidad y motivación** del personal, y el **uso eficiente de herramientas** son **esenciales** para lograr la **calidad** del producto.

La **complejidad** de los procesos de software **impide** la existencia de un **proceso ideal**, y muchas organizaciones han desarrollado sus propios enfoques. Estos **procesos evolucionan** para aprovechar las capacidades del personal y las características específicas de los sistemas. Para sistemas críticos, se requiere un proceso estructurado, mientras que, para sistemas empresariales con requisitos cambiantes, un enfoque ágil y menos formal puede ser más efectivo.

1. Consideraciones clave:

- **Variabilidad de actividades** según la organización y tipo de sistema.
- **Necesidad de modelar explícitamente el proceso** para una gestión efectiva.
- **Aplicabilidad a diversos ciclos de vida.**
- **Importancia de herramientas y equipos** en la ejecución de procedimientos.

2. Actividades fundamentales del proceso:

- **Especificación de software:** definición del producto y restricciones por clientes e ingenieros.
- **Desarrollo:** diseño y programación del software.
- **Validación:** verificación de conformidad con los requisitos del cliente.
- **Evolución:** ajuste del software para adaptarse a cambios en los requerimientos del cliente y el mercado.



Herramientas



Procesos



Personas

PROCESOS DEFINIDOS

Los procesos definidos en el desarrollo de software son considerados **deterministas**, buscando obtener la **misma salida con la misma entrada** de manera consistente. Están **inspirados en las líneas de producción** y buscan la repetibilidad para **lograr previsibilidad en esfuerzo, riesgos y costos**. La administración y control se basan en la predictibilidad del proceso, aunque en la realidad, **factores contextuales, temporales y personales afectan los resultados**, haciéndolos menos predecibles. En algunos casos, **la gestión del proceso puede tener más peso que el progreso real** del software, destacando la importancia de la documentación sobre el avance en sí (ke? 🐼).

PROCESOS EMPÍRICOS

Los procesos empíricos se basan en la **experiencia interna y externa** de las personas en un **contexto específico**, y no son considerados deterministas, ya que **no garantizan los mismos resultados** al aplicarse repetidamente. Se destacan por:

1. **Utilizar ciclos cortos de inspección y adaptación**, adecuados para dominios complejos donde la creatividad y la complejidad son primordiales.
2. La administración y control se logran mediante **inspecciones frecuentes y adaptaciones**.
3. Se **recomienda el modelo iterativo-incremental y se desaconseja el modelo secuencial**, especialmente después de dos años (según una teoría, cuál? No sé).

La **experiencia** adquirida en estos procesos **no es extrapolable** (esto lo suuuuper repitieron), **siendo válido solo para el proyecto** en curso. La **adaptación** de procesos a cada contexto **es esencial**. La **transparencia** de la información, la **inspección y adaptación**, y la **compartición** de conocimientos **son pilares clave en los procesos empíricos**.

CICLOS DE VIDA (MODELOS DE PROCESO) Y PROCESOS DE DESARROLLO DE SOFTWARE.

Esto está desarrollado más arriba, los ciclos de vida (clasificación y modelos) y la definición de los procesos de desarrollo de software. Sigamos.

CRITERIOS PARA ELECCIÓN DE CICLOS DE VIDA PARA PROYECTOS DE DESARROLLO DE SOFTWARE.

Para **seleccionar un modelo** de proceso adecuado, es esencial considerar **varios criterios**. La evaluación debe abordar los **niveles de riesgo asociados**, la **claridad y cantidad de requerimientos**, los **conocimientos técnicos** del **equipo de desarrollo**, los **plazos disponibles** para la ejecución del proyecto y la **viabilidad de validar los resultados** con el **cliente**. Además, se debe tener en cuenta el **dinamismo y eficacia del trabajo** en equipo.

1. **Modelo Secuencial:**
 - Requiere requerimientos claros y exhaustivos.
 - Adecuado para contextos de baja incertidumbre.
 - Ideal cuando el cliente no necesita el producto rápidamente.
2. **Modelo Iterativo-Incremental:**
 - Seleccionado cuando los requerimientos no son claros.
 - Preferido si el cliente busca resultados rápidos.
 - Enfocado en la entrega de las funciones principales del sistema de manera iterativa.
3. **Modelo Recursivo:**
 - Óptimo si el objetivo es reducir riesgos en el desarrollo de sistemas grandes y complejos.

PROYECTO DE SOFTWARE

(esto no está explícitamente en los contenidos pero lo re preguntan y se da en clase, seguro está dentro de alguno de los puntos que se desarrollaron antes pero no sabía dónde meterlo)

Un **proyecto de software** es un **esfuerzo temporal** que implica la **colaboración** de diversas **especialidades y recursos** para **crear un producto, servicio o resultado único**. Se configura como una unidad organizativa que **se adapta al marco teórico del proceso** según **las personas y recursos disponibles**. Su definición abarca el **establecimiento de procesos y tareas**, la **asignación de personal** a actividades específicas, y la **implementación de mecanismos** para evaluar riesgos, controlar cambios y garantizar la calidad del resultado final.

CARACTERÍSTICAS DE UN PROYECTO

1. **Únicos:**
 - Cada proyecto es **único e irrepetible**, con características propias como calendarios y presupuestos distintos.
2. **Orientado a Objetivos:**
 - Objetivos **no ambiguos y claros** para guiar el desarrollo del proyecto.
 - **Medibles** para **evaluar el progreso**.
 - **Alcanzables**, factibles de ser realizados por el equipo (no vuelen).
3. **Duración Limitada de Tiempo:**
 - Con un **principio y un fin** definidos.
 - Libera recursos y personas al alcanzar los objetivos.
4. **Conjunto de Tareas Interrelacionadas:**

- Define tareas, dependencias, recursos y personas asignadas.
- Permite manejar la complejidad inherente de los sistemas.

ADMINISTRACIÓN DE PROYECTO DE SOFTWARE

La administración de proyectos implica **organizar y coordinar personas y recursos** para **desarrollar un producto o servicio** de alta calidad, **cumpliendo con el presupuesto, el tiempo y las funcionalidades acordadas** con el cliente. Se divide en dos grandes actividades: **planificación y monitoreo/control**. Incluye la identificación de requerimientos, establecimiento de objetivos claros y adaptación de especificaciones a los intereses de los involucrados.

Metas Importantes:

1. **Entregar software** en el **tiempo acordado**.
2. **Mantener costos** dentro del **presupuesto**.
3. **Cumplir con las expectativas** del cliente.
4. **Mantener un equipo** de desarrollo **óptimo**.

ATRIBUTOS Y COMPLEJIDADES

1. **Intangibilidad del Producto:** La **falta de tangibilidad** dificulta la **medición del progreso** del proyecto, a diferencia de proyectos de construcción civil.
2. **Excepcionalidad de los Proyectos de Software:** Dificultad para **extrapolar conocimientos** de un proyecto a otro debido a la singularidad y factores tec.
3. **Variabilidad y Especificidad de los Procesos:** A pesar de intentos de estandarización, es **difícil predecir el proceso adecuado** para cada proyecto debido a la variabilidad y especificidad de los procesos de desarrollo.

LA TRIPLE RESTRICCIÓN – ENFOQUE TRADICIONAL

La **triple restricción** en proyectos, centrada en **tiempo, costo y alcance**, **impacta directamente en la calidad** del proyecto. **La calidad se logra al satisfacer los objetivos en el tiempo y presupuesto planificados**. El líder de proyecto debe **equilibrar** estas variables, **fijando dos de ellas y ajustando la tercera** según sea necesario.

- **Alcance:**
 - Define los **requisitos** del proyecto y la **funcionalidad** del producto.
 - Enfoque **tradicional:** **Se fija** al inicio y se negocia con el cliente.
- **Tiempo:**
 - **Establece la fecha de finalización** del proyecto y la entrega del producto.
 - Enfoque **tradicional:** **Variable** negociable, pero en **metodologías ágiles es fija**.
- **Costo:**
 - Define el **gasto en recursos**, incluyendo equipos, salarios y más.
 - El **costo y tiempo varían directamente con el alcance del proyecto**.

Enfoques tradicionales con ciclos de vida **en cascada** pueden generar **problemas al definir requerimientos** al inicio del proyecto. Modificar el alcance afecta tiempo y presupuesto, comprometiendo la calidad, ya que se sacrifica funcionalidad o se omiten etapas de prueba para cumplir con el cliente.

LÍDER DE PROYECTO

En un enfoque **tradicional**, el líder de proyecto **toma decisiones sin considerar la opinión del equipo** y maneja todas las relaciones con stakeholders. Debe poseer habilidades blandas como motivación, organización, innovación, empatía, comunicación y liderazgo. Además, requiere habilidades duras relacionadas con conocimientos técnicos y herramientas.

CARACTERÍSTICAS DEL LÍDER

1. **Motivación:** **Estimula al personal técnico** para alcanzar su máximo rendimiento.
2. **Organización:** **Adapta el proceso** para transformar el concepto inicial en el producto final.
3. **Innovación:** Fomenta la **creatividad y la innovación** en el equipo.
4. **Empatía, Comunicación y Liderazgo:** Habilidades esenciales para tratar con personas.
5. **Habilidades Duras:** **Conocimientos del producto, técnicas y herramientas**.

CARACTERÍSTICAS ADICIONALES

1. **Resolución de Problemas:** Identifica y resuelve conflictos técnicos y organizativos.
2. **Identidad Administrativa:** **Asume el control** cuando es necesario.
3. **Logro:** **Reconoce iniciativas y logros**, alentando al equipo a asumir riesgos controlados.
4. **Influencia:** Define la estructura del equipo mediante **retroalimentación verbal y gestual**.

RESPONSABILIDADES EN NUEVAS METODOLOGÍAS

1. **Definir el alcance** del proyecto.
2. **Identificar involucrados, recursos y presupuesto**.
3. **Detallar tareas, estimar tiempos y requerimientos**.
4. **Identificar y evaluar riesgos**.
5. **Preparar planes de contingencia.** (planes alternativos, por si las moscas)
6. **Controlar hitos y participar en revisiones** de fases del proyecto.
7. **Administrar** el proceso de **control de cambios**.
8. **Producir informes** de estado.

EQUIPO DE PROYECTO

Un **equipo cohesivo** es un grupo comprometido con **objetivos compartidos**, **mutuamente responsables** y caracterizado por:

Características

1. **Diversidad:**
 - ☺ Integrantes con **conocimientos, habilidades técnicas, motivación, experiencias y personalidades** diversas.
 - ☺ Fomento de la **mejora continua y el aprendizaje mutuo**.
2. **Tamaño Pequeño:**
 - ☺ **Facilita la comunicación** efectiva entre los miembros.
3. **Trabajo Conjunto:**
 - ☺ **Espíritu de grupo** que crea cohesión y sinergia.
 - ☺ **Estabilidad y madurez del equipo**, evitando cambios frecuentes en los integrantes.
4. **Sentido de Responsabilidad:**
 - ☺ **Unidad en la responsabilidad y enfoque** en el logro de **metas grupales**.

En Equipos Cohesivos

1. **Estándares Propios:**
 - Establecen sus **propios estándares de calidad**.
2. **Aprendizaje Mutuo:**
 - Los individuos **aprenden y se apoyan entre sí**.
3. **Compartir Conocimiento:**
 - Fomentan la **colaboración y el intercambio de conocimientos**.
4. **Mejora Continua:**
 - Prevalece un enfoque constante en la **mejora continua**.

STAKEHOLDERS

Son los interesados del proyecto, incluye el equipo de proyecto, el equipo de dirección, el líder de proyecto y el patrocinador.

PLAN DE PROYECTO

El **plan de proyecto**, también conocido como **plan de desarrollo de software**, actúa como una **guía estratégica** para el **proyecto**, **documentando decisiones clave** a lo largo del desarrollo. Incluye **detalles sobre el proceso**, distribución de **tareas**, **asignación de responsabilidades**, **equipos**, **calendario de desarrollo y actividades de soporte**, como planes de contingencia y evaluación de riesgos. Entonces, ¿Qué implica la planificación de un proyecto?:

1. Definición del alcance del proyecto.

- **Definición del Alcance:**
 - *Alcance del Producto:* Se centra en las **funcionalidades del software para cumplir con los requisitos** del cliente, medido contra la Especificación de Requerimientos (ERS).
 - *Alcance del Proyecto:* Define el **trabajo necesario para el desarrollo y la entrega del producto**, medido contra el **Plan de Proyecto**.
- **Modificaciones a lo largo del Proyecto:**
 - *Alcance y Objetivo del Proyecto:* **Pueden ser modificados** durante el desarrollo **sin necesariamente cambiar el alcance del producto**.
 - *Alcance del Producto:* **Puede condicionar los alcances del proyecto**, aunque esto no es una regla.
- **Relación Jerárquica:**
 - *Alcance del Proyecto:* Es inherentemente menor que el alcance del producto.
 - *Alcance del Producto:* Define para satisfacer los objetivos del cliente y condiciona los alcances del proyect.
- **Ejemplos de Alcances del Proyecto:**
 - Toma de requerimientos, testing de componentes específicos, desarrollo de código para un componente, etc....

Nota Importante: Aunque ambos alcances pueden modificarse, la relación entre ellos destaca la relevancia de la claridad en la planificación y ejecución del proyecto.

2. Definición de proceso y ciclo de vida.

- **Definición del Proceso:**
 - **Responde al "cómo"** se desarrollará el proyecto.
 - **Considera el contexto** del desarrollo, como objetivos, alcances, recursos, y forma de trabajo.
 - **Adaptación según la complejidad** del software y las características del proyecto.
- **Ciclo de Vida del Proyecto:**
 - Determina la **cantidad y el momento de cada tarea** durante el desarrollo.
 - Define **qué trabajo técnico se realiza en cada fase, quién está involucrado, cómo se controla y aprueba cada fase**, y cómo se generan los entregables.
- **Enfoque Metodológico:**
 - **Metodologías tradicionales** permiten elegir diversos ciclos de vida (cascada, iterativo, espiral), mientras que en **metodologías ágiles** se requiere un enfoque iterativo e incremental.

3. Estimación.

- **Definición:**
 - Estimación en desarrollo de software se refiere a **predecir valores relacionados con el proyecto o producto, como tiempo, costo y recursos.**
- **Metodología Tradicional:**
 - **Líder de proyecto** realiza estimaciones.
 - Orden recomendado:
- 1. **Tamaño del Producto:** Se estima **a partir de requerimientos**, crucial para calcular esfuerzo, tiempo y costos. Puede medirse en **líneas de código** (ay, Dios jajajajajajaj), **requerimientos, puntos de función**, etc.
- 2. **Esfuerzo:** **Cantidad de horas persona lineales** necesarias para construir el producto. **Varía** directamente con el **tamaño del proyecto** y puede ser afectado por las habilidades del equipo.
- 3. **Calendario:** **Estimación de cuándo se realizarán las tareas** del proyecto, no es necesario realizar una calendarización muy específica a nivel de día, sino a mes o trimestre. Proporcional al tamaño y esfuerzo.
- 4. **Costos:** **Estimación final, depende directamente del tamaño y esfuerzo** del proyecto. Útil para dar al cliente una idea del costo aproximado.
- **Importancia de Estimación:**
 - La estimación **afecta otros aspectos del proyecto y su flexibilidad** ante cambios.
 - **El tamaño del producto es crucial** para evitar errores en la estimación de costos, tiempos y esfuerzo.
 - **El esfuerzo impacta significativamente en el costo** del proyecto. Es el tópico que más repercute en el costo del proyecto, abarcando casi el 80% de estos.

4. Gestión de riesgos.

- **Definición de Riesgo:**
 - Riesgo es la **probabilidad de pérdida o daño** que afecta al proyecto directamente, **comprometiendo su objetivo.**
- **Clasificación de Riesgos:**
 1. **Riesgos del Proyecto:** Afectan **calendario o recursos**. Ejemplo: renuncia de un diseñador.
 2. **Riesgos del Producto:** Impactan **calidad o rendimiento** del software. Ejemplo: falla en un componente.
 3. **Riesgos Organizacionales o de Negocio:** Afectan a la **organización** que desarrolla o adquiere el software. Ejemplo: introducción de un nuevo producto por un competidor.
- **Medición de Riesgos:**
 - Se utiliza la **exposición al riesgo** (probabilidad x impacto) **para organizar y gestionar riesgos**. Hacemos foco en reducir la exposición.
- **Actitudes ante Riesgos Identificados:**
 1. **Negación:** Ignorar la existencia de riesgos (fingir demencia).
 2. **Gestión Reactiva:** Reaccionar cuando el riesgo ocurre.
 3. **Gestión Proactiva:** Trabajar en planes de mitigación y contingencia antes de que los riesgos se activen
- **Riesgos Comunes en Desarrollo de Software:**
 - Cambio de requerimientos, abandono de equipo, falta de capacidad del equipo, avance tecnológico no comprendido.
- **Gestión del Riesgo:**
 - Es crucial para **prevenir pérdidas** y mantener el proyecto en curso. Esta gestión consta de:
 2. **Identificación de Riesgos:** Reconocer amenazas al proceso de ingeniería y al software.
 3. **Análisis de Riesgos:** Evaluar probabilidad e impacto de cada riesgo identificado.
 4. **Planeación del Riesgo:** Definir estrategias y acciones para minimizar o evitar impactos.
 5. **Monitorización y Control:** Evaluar cambios en suposiciones acerca de los riesgos durante todo el proy.

5. Asignación de recursos.

- **Asignación de Responsabilidades:**
 - Implica **asignar roles a los miembros** del equipo de trabajo en lugar de referirse a ellos como "RRHH".
 - En entornos con procesos definidos (como PUD), los roles se conceptualizan como "trabajadores".
- **Flexibilidad de Roles:**
 - En proyectos más pequeños, una persona puede desempeñar **múltiples roles** debido a **limitaciones presupuestarias o menor complejidad** del proyecto.
- **Documentación de Roles:**
 - La información sobre roles y responsabilidades **se registra en una tabla** para claridad y transparencia.
- **Importancia de la Selección del Personal:**
 - La calidad del software y el desarrollo del proyecto dependen en gran medida de las **habilidades y capacidades del equipo.**
- **Enfoque Humano-Intensivo:**
 - Dado que el desarrollo de software es intensivo en recursos humanos, la **selección y asignación de roles** se vuelve **crítica para el éxito** del proyecto.

6. Programación de proyectos.

- **Objetivo de la Etapa:**
 - Definir detalladamente cada tarea necesaria para el desarrollo del proyecto. Busca una planificación práctica y detallada para guiar la ejecución eficiente del proyecto.
- **Basado en Estimación de Calendario:**
 - Utiliza como base la estimación del calendario previamente realizada, pero busca refinar cada actividad al máximo detalle.
- **Descomposición de Tareas:**
 - Implica desglosar el proceso de desarrollo en actividades detalladas a nivel de días, especificando quién las realiza, cuándo y cuánto esfuerzo teórico se requiere.
- **Herramienta Común:**
 - La calendarización suele realizarse mediante un Diagrama de Gantt, utilizando herramientas como Microsoft Project.

7. Definición de métricas. (Esto se ve más detalle más adelante)

- **Definición de Métricas:**
 - Métricas son medidas numéricas que proporcionan visibilidad sobre el estado y avance del proyecto, proceso o producto en un momento específico.
- **Claridad y Representatividad:**
 - En la planificación del proyecto, se deben definir de manera clara y no ambigua las métricas asociadas a cada dominio (producto, proyecto o proceso) y asegurarse de que sean representativas para el entorno.
- **Métricas del Proyecto y del Proceso:**
 - Las métricas del proyecto se consolidan para crear métricas de proceso, compartidas en toda la organización del software.
- **Enfoque no Ágil:**
 - A diferencia de las metodologías ágiles, se definen métricas que miden y exponen los avances en lugar de depender principalmente de incrementos entregados en cada iteración.
- **Clasificación de Métricas Básicas:**
 - Métricas básicas para un proyecto de software se clasifican en:
 - Tamaño del producto
 - Esfuerzo
 - Calendario (tiempo)
 - Defectos
 - Costos

8. Definición de controles.

La planificación de controles en el desarrollo se fundamenta en las métricas previamente definidas para asegurar que todas las actividades estén alineadas con lo establecido. Se establecen reuniones periódicas y la generación de reportes e informes como parte de la planificación de controles. Estos elementos son clave para evaluar y verificar el progreso del desarrollo.

- Seguimiento de proceso: se realiza un control por hitos o por tareas
- Seguimiento de producto: se realiza un control sobre los entregables

PLANES DE SOPORTE

- Planes de Testing:
- Planes de Mantenimiento:
- Planes de Subcontrataciones:
- Planes de Calidad:
- Planes de Capacitaciones:
- Planes de Iteraciones (en ciclos iterativos/incrementales):

FACTORES PARA EL ÉXITO DE UN PROYECTO

- ☺ Misión/objetivos claros
- ☺ Comunicación
- ☺ Monitoreo & Feedback

FACTORES PARA EL FRACASO DE UN PROYECTO

- ☹ Fallas al definir el problema
- ☹ Planificación basada en datos insuficientes
- ☹ Planificación realizada por grupo de planificaciones
- ☹ No hay seguimiento del plan de proyecto
- ☹ Mala planificación de recursos
- ☹ Estimaciones basadas en supuestos sin consultar datos históricos
- ☹ Nadie estaba a cargo

VINCULO PROCESO-PROYECTO-PRODUCTO EN LA GESTIÓN DE UN PROYECTO.

La idea en este punto es poder unir TODO lo que vimos hasta ahora, lo que se describe a continuación es una versión simplificada, pero teniendo claros y firmes todos los puntos anteriores se puede pilotear una banda. Si te toca este punto como tema de desarrollo/final recomiendo expandir.

En la **gestión de proyectos** de desarrollo de software, **el proceso, automatizado con herramientas, se adapta al proyecto, que incorpora personas, resultando en un producto.**

Enfoque tradicional: **se definen objetivos y ámbito del producto antes del proyecto** para realizar estimaciones. **El proceso**, teórico y adaptable, **guía la transformación de requerimientos** en productos.

Proporciona un marco conceptual y actividades. Disciplinas de gestión

se adaptan al proyecto, mientras que las de **soporte son independientes y cubren el modelo del proceso**. Se elige un **modelo de proceso** (secuencial, iterativo, recursivo), y el **equipo** sigue roles definidos basados en el proceso adoptado.

Las **personas** son **clave** en el desarrollo humano-intensivo. Necesitamos equipo capacitado que reúna las habilidades y herramientas necesarias para el desarrollo de este.

Herramientas automatizan actividades del proceso, como software de diseño para análisis y diagramación.



NO SILVER BULLET

Y por último, de esta unidad, las benditas balas de plata (que no hay), vamos a intentar resumir este libro:

El concepto **"No Silver Bullet"** destaca la **falta de soluciones mágicas** para superar las dificultades inherentes al desarrollo de software. **Se contrasta el progreso del hardware tangible con la complejidad inherente del software intangible**. Las **"balas de plata"** son soluciones que podrían abordar la crisis del software, pero **no han demostrado avances significativos en menos de una década**.

DIFICULTADES ESENCIALES

1. **Complejidad:** Aumenta exponencialmente con el tamaño del software, generando problemas de comunicación, deficiencias y errores.
2. **Conformidad:** El software debe cumplir con los **requerimientos del cliente**.
3. **Mutabilidad:** Sujeto a **cambios constantes** a lo largo del tiempo.
4. **Invisibilidad:** Dificulta el trabajo con representaciones geométricas, como por ejemplo estructuras.

DIFICULTADES ACCIDENTALES

- **Solucionadas a lo largo del tiempo** con avances como **lenguajes de programación** de alto nivel, **tiempo compartido** y **entornos de programación unificados**.

RESOLUCIÓN DE DIFICULTADES

- Enfoques como **programación orientada a objetos**, **inteligencia artificial**, **sistemas expertos**, **programación automática y gráfica**, **verificación de programas** y **entornos/tools**.
- **Estaciones de trabajo** poderosas y resolución de dificultades esenciales, como la **compra en lugar de construir** y el **desarrollo incremental**.
- Importancia de **personas capacitadas y creatividad** en el equipo.

Se destaca la necesidad de **reconocer la complejidad inherente del software** y la **importancia de la creatividad** en el equipo de desarrollo, sin perder de vista que **no hay soluciones mágicas**.

UNIDAD 2 | GESTIÓN LEAN-ÁGIL DE PRODUCTOS DE SOFTWARE

GESTIÓN DE PROYECTOS BASADOS EN PROCESOS DEFINIDOS.

La **gestión de proyectos** basada en **procesos definidos** es **esencial** para el desarrollo y mantenimiento efectivos de productos de software. Esto implica:

1. **Establecer pasos específicos** desde la **concepción** hasta la **entrega** del producto. Los procesos pueden incluir la **recopilación de requisitos, diseño, implementación, pruebas y despliegue**.
2. Asignar **roles y responsabilidades** es crucial. Cada miembro del equipo debe comprender claramente sus funciones para garantizar una colaboración eficiente. Los procesos definidos proporcionan estructura y claridad.
3. La gestión efectiva implica la **medición y control continuo del progreso**. Esto se logra mediante la implementación de **métricas específicas** y la **monitorización constante** para garantizar que el proyecto siga el camino correcto.
4. Los procesos definidos permiten la **identificación de áreas de mejora**. Mediante la **retroalimentación** y la **evaluación constante**, se pueden realizar ajustes para optimizar la eficiencia y la calidad del desarrollo. (no tanto como en los procesos empíricos pero it Works)

PLANIFICACIÓN PARA EL DESARROLLO Y EL MANTENIMIENTO DE PRODUCTOS DE SOFTWARE.

La **planificación** comienza con la **estimación de recursos necesarios**, incluidos el personal, el tiempo y los costos. Esto implica una **evaluación** detallada de los **requisitos** del proyecto y la **complejidad** del software a desarrollar o mantener.

La elaboración de un **cronograma** detallado es esencial. Define las **fechas de inicio** y **finalización** de **cada fase** del proyecto, permitiendo una **gestión del tiempo** efectiva y la identificación de posibles cuellos de botella.

La planificación debe abordar los posibles **riesgos** y **establecer estrategias de mitigación**. Identificar y planificar para posibles desafíos ayuda a minimizar impactos negativos en el desarrollo.

La planificación **puede** (énfasis en que depende del modelo que se elija) incorporar enfoques incrementales o iterativos, lo que significa entregar incrementos de funcionalidades o realizar iteraciones a lo largo del tiempo. Esto permite adaptarse a cambios en los requisitos y mejorar la satisfacción del cliente.

Incluir **planes para el mantenimiento** del software es fundamental. Esto abarca actualizaciones, correcciones de errores y mejoras continuas después de la implementación inicial.

Beneficios de la Gestión basada en Procesos Definidos y una Planificación Rigurosa

- **Mayor Eficiencia:** Los procesos bien definidos y una planificación detallada reducen la ambigüedad y mejoran la eficiencia en todas las etapas del proyecto.
- **Mejor Control:** Se facilita el seguimiento y control del progreso del proyecto, permitiendo una toma de decisiones informada y oportuna.

MÉTRICAS Y ESTIMACIONES DE SOFTWARE

Este tema lo vamos a tomar más adelante abarcando ambientes ágiles y tradicionales.

MANIFIESTO ÁGIL/FILOSOFÍA LEAN

OBJETIVO DEL ENFOQUE ÁGIL

El enfoque ágil tiene como **objetivo** principal la **construcción rápida de software** para **aprovechar oportunidades** actuales y hacer **frente a la competencia**, destacando la velocidad de entrega como requisito fundamental. Este método se adapta especialmente a **entornos con variabilidad de requisitos**.

MANIFIESTO ÁGIL

Es un documento redactado en 2001 por expertos de la Ingeniería en Software, que promueve la filosofía del desarrollo ágil del software.

VALORES ÁGILES

1. **Individuos e interacciones por sobre procesos y herramientas:** Destaca la importancia de **equipos motivados** y **comunicativos**, considerando las **herramientas** como **complemento** adaptable al equipo. Intentan capitalizar las **fortalezas de cada individuo** (como así también paliar las debilidades individuales), para así lograr una **armonía** que permita **fortalecer la unión** y **motivación** dentro del equipo.
2. **Software funcionando por sobre documentación extensiva:** Prioriza la **entrega rápida** y **continua** de software estable, obteniendo **retroalimentación** del cliente y generando **documentación solo cuando agrega valor**.
3. **Colaboración con el cliente por sobre negociación contractual:** Favorece la **integración activa del cliente** en el proceso, **evitando dependencias de documentos** extensos y **comprometiendo al cliente** en el desarrollo, esto permitirá no sólo tener una mejor noción de aquellos requerimientos de negocio que necesita el cliente, sino también tener a un cliente que esté comprometido con cada entrega, sobre todo al momento de probarlo y dar el feedback.
4. **Responder a cambios por sobre seguir un plan:** Reconoce la **imprevisibilidad de los requisitos** y promueve la **adaptación a cambios** para ofrecer el mayor valor posible al cliente, en contraste con rigidez en la planificación tradicional. Al finalizar cada iteración, la **experiencia** de **desarrollar** los incrementos, **mostrar** o **desplegar** estos a los usuarios finales, **genera cierto conocimiento** en el equipo que **puede inducir a cambios**, los cuales el equipo ágil deberá incluir en su planificación.

PRINCIPIOS DEL MANIFIESTO ÁGIL

Estos hay que saberlos como el padre nuestro:

1. **Satisfacer al cliente mediante la entrega continua de software valioso:**
 - * El equipo ágil sigue las prioridades del Product Owner para garantizar la **entrega de funcionalidades de alto valor al cliente en cada iteración**.
2. **Adaptarse a cambios incluso en etapas tardías del desarrollo:**
 - * La **flexibilidad** del equipo ágil **permite ajustes continuos** en respuesta a los cambios de requisitos, evitando planes rígidos.
3. **Entregar software funcional de manera frecuente:**
 - * El equipo **produce iterativamente** software completamente desarrollado y testeado, asegurando **entregas constantes y de calidad**.
4. **Colaboración constante entre individuos técnicos y no técnicos:**
 - * Destaca la importancia de la **colaboración** entre el **Product Owner y el equipo** de desarrollo para una **comunicación efectiva** y la **comprensión** de objetivos compartidos.
5. **Desarrollar proyectos en torno a individuos motivados:**
 - * Reconoce que la **motivación** y el **compromiso** individual son **esenciales** para el **éxito** del proyecto, fomentando la colaboración y la visión compartida.
6. **La comunicación cara a cara es crucial para una comprensión efectiva:**
 - * Se enfatiza la **comunicación directa** como el método **más efectivo**, reconociendo la importancia de los aspectos no verbales.
7. **La mejor medida de progreso es el software funcionando:**
 - * Destaca que el **progreso** real **se demuestra mediante entregas de software funcional**, contrastando con métricas tradicionales.
8. **Promover el desarrollo sostenible:**
 - * Favorece **equipos estables** a lo largo del tiempo **para mejorar continuamente** las **relaciones** y las **prácticas** de trabajo.
9. **Atención constante a la excelencia técnica y al buen diseño:**
 - * Enfatiza la **importancia** de la **calidad técnica y el diseño** para asegurar la entrega de un producto de alta calidad.
10. **La simplicidad es esencial:**
 - * Insta a **centrarse** en las **necesidades** que **aportan valor** al cliente, **evitando la complejidad** innecesaria.
11. **Las mejores soluciones surgen de equipos autoorganizados:**
 - * Destaca la **proactividad** de los **equipos** en la **generación de soluciones**, permitiendo que las mejores ideas emerjan de la colaboración.
12. **Evaluación regular y ajuste continuo:**
 - * Propone que el equipo **evalúe periódicamente** su **desempeño y ajuste** sus métodos **en respuesta** a cambios y oportunidades de mejora.

FILOSOFÍA ÁGIL

La **Filosofía Ágil** busca **entregar software de forma frecuente en entornos cambiantes**, con énfasis en la adaptabilidad. Utiliza metodologías en entornos con variabilidad de requerimientos, **involucrando al cliente** para obtener **retroalimentación rápida**. Adopta un enfoque **iterativo-incremental**, generando incrementos cada **2-3 semanas**, con el cliente inmerso para una rápida inspección y adaptación ante cambios.

A diferencia de enfoques como el PUD o RUP, la Filosofía Ágil **prioriza la adaptabilidad sobre la predictibilidad**. Busca equilibrar la rigurosidad del proceso con la entrega de software de calidad, evitando extremos. **Reduce la documentación** mediante comunicaciones frecuentes e informales.

Según Martin Fowler, la Filosofía Ágil es un **"compromiso entre nada de proceso y demasiado proceso"**. Se basa en el Manifiesto Ágil, que **se sustenta en procesos empíricos** con pilares de transparencia, inspección y adaptación. La esencia radica en **ser ágil y adaptable**, favoreciendo la entrega continua y la mejora constante.

Ser ágil, en el contexto de desarrollo de software y gestión de proyectos, se refiere a la adopción de los principios y valores delineados en el Manifiesto Ágil. La agilidad es una filosofía que promueve enfoques flexibles, colaborativos y adaptativos para la creación de productos y la gestión de proyectos.

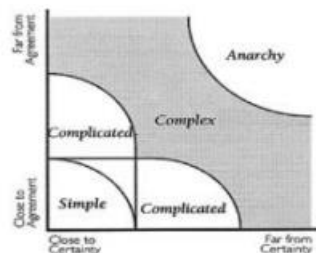
PILARES DEL EMPIRISMO

1. **Transparencia:** Facilita el crecimiento del equipo al **convertir el conocimiento individual en conocimiento colectivo**. Promueve la **colaboración** al enfocarse en la propiedad compartida de los desarrollos. La falta de transparencia puede llevar a decisiones perjudiciales.
2. **Inspección:** Es esencial **evaluar productos, procesos y aspectos humanos** para **identificar variaciones y posibles problemas**. La inspección proporciona **oportunidades de aprendizaje** continuo, asegurando la detección temprana de desviaciones.
3. **Adaptación:** Implica la **mejora continua y la capacidad de ajustar el proceso** según los resultados de la inspección. Los **ajustes** deben realizarse **de inmediato** para **minimizar desviaciones** y garantizar la adaptabilidad del equipo ante nuevos aprendizajes.

¿CUÁNDO ES APLICABLE ÁGIL?

Ágil no es una metodología específica, sino una ideología que guía el desarrollo del producto a través de valores y principios. Los frameworks ágiles son herramientas para implementar estos principios, mayormente centrados en la gestión de proyectos. (En el universo ágil hay categorías, XP, TDD, FDD, CM, etc. Scrum está en término medio, el cual representa en 55% del nivel de uso entre las metodologías ágiles.)

Destaca que la mayoría de los procesos ágiles se centran en prácticas de gestión en lugar de ingeniería. La aplicabilidad de las metodologías ágiles es más eficaz en contextos complejos, donde hay incertidumbre y nuevos productos emergen con entendimiento y certeza parciales.

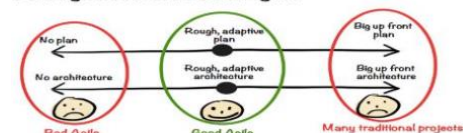


En el gráfico de áreas, Ágil destaca en el ámbito de lo complejo, brindando buenos resultados. La investigación actúa en el campo de la "anarquía", caracterizado por una gran incertidumbre. El mantenimiento, generalmente, se sitúa en lo relativamente simple, aunque esto no es una regla general.

Finalmente, se enfatiza que ser ágil no implica indisciplina, como dice en la imagen.

Adoptar el enfoque ágil ofrece beneficios como entregas rápidas, adaptabilidad a cambios, alineación de objetivos, reducción de riesgos, mayor visibilidad del proyecto, mejoras en relaciones de equipo y eficiencia costosa.

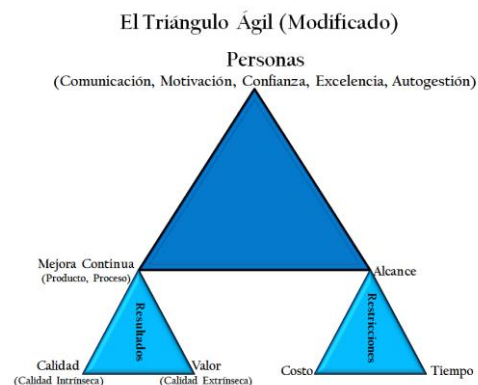
Don't go overboard with Agile!



EL TRIÁNGULO ÁGIL

El triángulo ágil se centra en el valor del software para el cliente, destacando la importancia de la calidad intrínseca y extrínseca del producto.

Coloca a las personas en el centro, priorizando la comunicación, motivación, confianza, excelencia y autogestión. La mejora continua, representada en la base del triángulo, se apoya en la calidad (intrínseca) y el valor (extrínseco). Del otro lado de la base podemos encontrar el alcance, el cual nos da cuáles serían las restricciones de nuestro producto, y encontramos el costo y el tiempo, los cuales suelen ser grandes limitantes a la hora de desarrollar software.



FILOSOFÍA LEAN

La filosofía Lean, originada en Japón con Toyota en los años 40, busca maximizar el valor para el cliente minimizando el uso de recursos. Se centra en eliminar cualquier esfuerzo que no añada valor, reduciendo la creación de artefactos innecesarios y enfocándose en aspectos esenciales/de valor (Siendo el cliente quien determina si el servicio o producto que la empresa entrega tiene valor o no.). Busca integrar cultura y estrategia para brindar al cliente calidad, bajo costo y entrega oportuna, produciendo lo que el cliente quiere, cuando lo quiere y donde lo quiere. Utiliza el método ✨ Just in Time ✨, intercambios y entregas justo a tiempo, evitando actividades innecesarias.

PRINCIPIOS LEAN

- 1. Eliminar desperdicios:** Es quizás el más importante. Enfocado en la eficiencia y la adición de valor en cada paso del proceso, todo paso o parte que no contribuye a la generación de valor produce un desperdicio. Este principio se relaciona con los principios agile 3 y 7 (entregar software funcionando frecuentemente y el software funcionando es la principal medida de progreso) y 10 (la simplicidad o maximizar el trabajo no realizado es esencial), porque en el software el trabajo parcialmente hecho y la funcionalidad extra son el desperdicio.

Desperdicios en Desarrollo de Software según Lean:

- ❖ **Características extras:** Evitar funcionalidades innecesarias que no aporten valor.
 - ❖ **Trabajo a medias:** Gestionar tareas como "listas" o "no listas" para evitar trabajos inacabados.
 - ❖ **Proceso extra:** Identificar y eliminar pasos que no contribuyan al valor del producto.
 - ❖ **Búsqueda de información:** Implementar SCM para facilitar la trazabilidad y la resolución de errores.
 - ❖ **Defectos:** Aplicar Aseguramiento de Calidad para prevenir errores y retrabajos.
 - ❖ **Esperas:** Favorecer equipos multidisciplinarios para evitar tiempos de espera.
 - ❖ **Cambios de tareas (Multitasking):** Limitar el WIP para reducir el multitasking y mejorar la eficiencia.
- 2. Amplificar el aprendizaje:** Promueve una cultura de mejora continua y compartición de conocimientos. Se relaciona con los principios ágiles de (2) recibir requerimientos aún en etapas finales y el principio de (4) personas técnicas y no técnicas trabajando juntos todo el proyecto.
 - 3. Embeber la integridad conceptual:** Prioriza la calidad del producto y la simplicidad, alineando cada decisión con las expectativas del cliente. Coherencia y consistencia son clave, reflejando principios ágiles.

4. **Diferir compromisos hasta el último momento responsable:** Postergar decisiones para tener la **máxima información**, pero antes de que sea tarde. Se relaciona con el principio ágil de recibir cambios de requerimientos aun en etapas finales (3).
5. **Dar poder al equipo:** Otorgar **libertad** y **poder de decisión** a equipos **multifuncionales** y **autogestionados**. Esto implica **no subordinar** por parte de los superiores, debido a que esto anula las capacidades intelectuales, **entrenar líderes, delegar decisiones y fomentar buena ética laboral**. Pero a la vez, implica un **compromiso** por parte de los miembros del equipo.
Relacionado con el principio 11 del Manifiesto Ágil ("Las mejores arquitecturas, requisitos y diseños emergen de equipos autoorganizados").
6. **Ver el todo:** Busca una **visión holística** (ver algo de forma completa, en su totalidad) para **alinear objetivos particulares con objetivos globales**, conectando con la *filosofía ágil* de trabajar hacia **metas comunes**.
7. **Entregar lo antes posible:** Prioriza la **entrega rápida** de software funcional, **adaptándose a cambios** en las necesidades del cliente. *Relacionado con principios ágiles de entregas tempranas y frecuentes de software de valor para el cliente (1 y 3).*

RELACIÓN LEAN-ÁGIL

Lean, anterior al Manifiesto Ágil, **comparte principios fundamentales con Agile**. Ambos se **centran en el cliente**, buscan **proporcionar máximo valor**, requieren **equipos motivados** y enfatizan la **sinergia como valor añadido**. La **adaptabilidad a cambios** y la entrega continua de calidad son aspectos comunes en ambas filosofías.

REQUERIMIENTOS EN AMBIENTES LEAN-AGILE

En el desarrollo de software, la **mayor dificultad** radica en **definir qué software construir**, más que en desafíos tecnológicos. Los proyectos fracasan debido a la **brecha entre las expectativas del cliente y lo entregado**. La etapa de **relevar requisitos**, a menudo **subestimada**, es **esencial** y **social**. **Problemas de comunicación** entre clientes y equipos también son comunes, ya sea por expectativas poco realistas o falta de comprensión mutua. La **colaboración efectiva** y la **visualización temprana** del progreso son **clave** para evitar desviaciones y garantizar el éxito del proyecto. La **comunicación bidireccional** entre ambas partes es esencial, ya que el dominio unilateral conduce a asignaciones de recursos ineficientes. La **flexibilidad** y la **adaptabilidad** son **fundamentales**, ya que no se pueden prever todas las actividades desde el principio.

FUNDAMENTACIÓN DE LOS REQUERIMIENTOS ÁGILES

Los requerimientos ágiles se fundamentan en los **principios del manifiesto ágil**, destacando la entrega temprana y frecuente de software valioso (1) y la adaptabilidad a cambios en etapas finales (2). Estos requerimientos se apoyan en **tres pilares**:

1. **Construcción de Valor:** Centrarse en construir un producto que **aporte valor** al negocio y **satisfaga al cliente**, haciéndole la vida más fácil.
2. **"Sólo lo Suficiente":** Encontrar los **requerimientos** de manera **incremental**, **empezando con lo mínimo** necesario para obtener retroalimentación y avanzar de manera continua.
3. **Uso de Historias y Modelos:** Desarrollar requerimientos en **colaboración con el cliente**, aplicando la idea de "técnicos y no técnicos trabajando juntos" (4) para construir el producto de manera conjunta.

La **comunicación cara a cara** (6) es esencial, y el enfoque de requerimientos ágiles debe alinearse con los valores y principios del manifiesto ágil. La **ingeniería de requerimientos** abarca el **desarrollo** y la **gestión**, **asegurando la integridad y actualización** constante de los requisitos a lo largo del trabajo con el producto de software.

LA TRIPLE RESTRICCIÓN – ENFOQUE ÁGIL

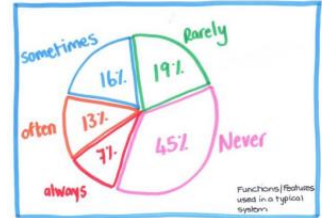
En el enfoque ágil, se reconoce la realidad de que **el alcance del producto puede cambiar** durante el desarrollo. Se **prioriza fijar el tiempo y los recursos**, adaptándose a "**cuánto software**" se puede entregar en ese tiempo con el equipo disponible.

En contraste, el enfoque tradicional fija el alcance, y los recursos y el tiempo se derivan a partir de ello. La resistencia a cambios en los requerimientos en este enfoque se debe a que alterar el alcance afecta directamente los recursos y el tiempo.



BRUF – BIG REQUIREMENTS UP FRONT

El concepto de **BRUF** (Big Requirements Up Front) destaca la importancia de desarrollar inicialmente unas **pocas funcionalidades de gran valor** para el cliente, **cubriendo** así una **parte significativa de sus necesidades**. La **priorización de requerimientos** se vuelve **crucial**, y el equipo, en colaboración con el **Product Owner**, debe identificar los requisitos indispensables para el negocio, permitiendo la entrega de versiones valiosas del producto.



PRODUCT BACKLOG

La gestión ágil aborda el problema de desarrollar software no utilizado al **priorizar características y requerimientos** a través del Product Backlog. El Product Owner, con claridad sobre las necesidades, establece el **Product Backlog**, una **lista priorizada basándose en el valor** para el negocio. A diferencia del enfoque tradicional, donde el cliente no se compromete y el equipo decide la prioridad, en ágil, el cliente lidera la priorización, asegurando la construcción del producto deseado. Esta metodología **compensa la falta de documentos formales de requerimientos** al enfocarse en la comunicación cara a cara y la participación del cliente en el equipo.

Los requisitos cambiantes son una ventaja competitiva si puede actuar sobre ellos



REQUERIMIENTOS JUST IN TIME

La gestión ágil de requerimientos sostiene los requerimientos **justo a tiempo**, esto se alinea con la filosofía Lean, **eliminando desperdicios y difiriendo compromisos**. Se inicia con una visión y algunos requerimientos, construyendo el producto con la participación continua del cliente. El **Product Backlog nunca está completamente definido**, a diferencia del enfoque tradicional que emplea "Up Front Specification". En ágil, **los requerimientos se consensuan continuamente** con el cliente, reduciendo el impacto de posibles errores en la captura de requerimientos. El producto se desarrolla de manera incremental y **los requerimientos se identifican y describen a medida que surgen**, conforme haga falta, "ni antes, ni después", evitando la especificación completa desde el principio.

FUENTE DE REQUERIMIENTOS

En el enfoque ágil, donde la comunicación **cara a cara es clave**, los requerimientos del software se obtienen mediante **conversaciones directas con el cliente**. A diferencia de los ambientes tradicionales, **no se elabora** un extenso documento de especificación de requerimientos (ERS), pero **se compensa con la disponibilidad del cliente** para aclarar dudas, **reduciendo la necesidad de documentación formal**. El desarrollo ágil no descarta la documentación, pero **prioriza registrar y especificar solo lo que realmente agrega valor** para el cliente, evitando la creación de documentos extensos y potencialmente erróneos.

MOMENTO DE LA CAPTURA DE REQUERIMIENTOS

En el enfoque tradicional, los requerimientos se definen al inicio del proyecto, con un énfasis inicial en el proceso. En cambio, en el enfoque ágil, la captura de requerimientos es **un proceso continuo que se extiende a lo largo de todo el proyecto**, permitiendo iniciar la construcción del software incluso cuando los requerimientos no están completamente definidos, pero se cuenta con una visión general.

CONTENEDOR Y PERSISTENCIA DE LOS REQUERIMIENTOS

En el enfoque tradicional, la Especificación de Requerimientos de Software (ERS) contiene los requerimientos a lo largo del ciclo de vida del producto, requiriendo actualizaciones en caso de cambios. En el enfoque ágil, el **Product Backlog sirve como un contenedor temporal de requerimientos**, y una vez implementados, estos quedan "persistentes" en el producto funcionando, con detalles de valor conservados en casos de prueba construidos durante el testing.

TIPOS DE REQUERIMIENTOS

La parte baja de la pirámide, corresponde a los requerimientos del Dominio de la solución (esto no le interesa al cliente, es cómo el equipo transforma sus necesidades en código ejecutable). Lo que está por encima, es decir, Requerimientos de Usuario y Requerimientos de Negocio, corresponden al Dominio del Problema, y es lo que al cliente sí le interesa.

En la **gestión ágil** de requerimientos, se manejan **dos niveles**:

1. **Requerimientos de Negocio:**
 - Representan el **valor de negocio**.
 - Son fundamentales para el cliente.
2. **Requerimientos de Usuario:**
 - Relacionados con las **necesidades del usuario**.
 - Las **User Stories** son un medio para identificar estos requerimientos de usuario.
 - **El Product Owner**, en enfoques como Scrum, se centra en **identificar las User Stories** para satisfacer las necesidades de negocio del usuario.



En la gestión ágil de requerimientos, debemos asumir:

1. **Cambios Constantes:** Inevitables y constantes.
2. **Stakeholders Involucrados:** Identificar y comprender a todos los stakeholders (con interés en el producto).
3. **Comunicación Efectiva:** Entre el PO y los stakeholders es esencial para transmitir las necesidades y expectativas
4. **Validación del Usuario:** La comprensión de los requisitos se logra cuando el usuario experimenta la entrega
5. **Adaptación de Herramientas:** No hay una única herramienta o técnica aplicable en todos los casos.
6. **Entrega de Valor:** Ofrecer soluciones valiosas en lugar de simplemente cumplir con requisitos específicos.

PRINCIPIOS ÁGILES RELACIONADOS CON REQUERIMIENTOS ÁGILES

(Esta es una pregunta de parcial y lo veo re factible en un final)

Los principios ágiles se relacionan con los requerimientos ágiles de la siguiente manera:

1. **La prioridad es satisfacer al cliente a través de releases tempranos y frecuentes:**
 - Relación con Requerimientos Ágiles: Prioriza la entrega continua de valor al cliente, lo cual implica **identificar y satisfacer sus requerimientos de manera constante** a lo largo del proyecto.
2. **Recibir cambios de requerimientos, aun en etapas finales:**
 - Relación con Requerimientos Ágiles: Alineado con la **flexibilidad de los requerimientos ágiles**, permite adaptarse a cambios en cualquier fase del proyecto para cumplir con las necesidades cambiantes del cliente.
4. **Técnicos y no técnicos trabajando juntos TODO el proyecto:**
 - Relación con Requerimientos Ágiles: Favorece la **comunicación** constante entre el **equipo técnico** y los **stakeholders no técnicos**, facilitando la **comprensión y refinamiento continuo** de los requerimientos para construir el producto correcto.
6. **El medio de comunicación por excelencia es cara a cara:**
 - Relación con Requerimientos Ágiles: Destaca la importancia de la **comunicación directa** en la **captura efectiva de requerimientos**, fomentando conversaciones cara a cara para comprender y validar necesidades.
11. **Las mejores arquitecturas, diseños y requerimientos emergen de equipos autoorganizados:**
 - Relación con Requerimientos Ágiles: Equipos autoorganizados están más capacitados para identificar y comprender los requerimientos del cliente, pueden adaptarse rápidamente a los cambios. Tienen la libertad de proponer y experimentar con diferentes enfoques de diseño, fomentando la innovación y la creatividad. Equipos autoorganizados están más enfocados en construir productos que realmente agreguen valor al cliente. Esto se traduce en la identificación y priorización efectiva de los requerimientos que impactan directamente en los objetivos del cliente.

USER STORIES

Las **User Stories** (US para más referencias) son una **técnica** esencial en entornos ágiles para **abordar requerimientos de usuario**. Estas historias contienen **descripciones breves** de las **funcionalidades** deseadas, valoradas por los usuarios. Funcionan como **recordatorios** para que el equipo y el Product Owner **discutan las necesidades** del negocio y diferir **conversaciones** detalladas.

Son requerimientos a **nivel de usuario**, no a nivel de sistema, lo que las hace más abstractas y las coloca en un nivel de planificación más alto. **Escritas por el Product Owner** (si justo), son consideradas **verticales** dentro del producto, abarcando desde interfaces hasta la base de datos. Son **multipropósito**, sirviendo como **representaciones de necesidades** de usuario, **descripciones del producto**, **ítems de planificación** y **recordatorios para futuras conversaciones**. Además, junto con el código ejecutable, **contribuyen a la documentación** del producto.

Surgen para **superar la dificultad de comunicar las acciones** del sistema a todas las partes interesadas, ya que cada una tiene un interés específico. En contraste con la elicitación tradicional, las User Stories **permiten tomar decisiones** a lo largo del proyecto, adaptándose a la **reducción de incertidumbre** y el **aumento de información**.

PARTES DE LA USER STORY

La User Story consta de tres partes esenciales: (TCC)

1. **Tarjeta:**
 - **Frase Verbal:** Expresión que sigue la estructura <<Frase Verbal>>, describiendo la actividad.
 - **Nombre del Rol:** Indica quién realiza la acción o recibe el valor.
 - **Actividad:** Representa la acción realizada por el sistema.
 - **Valor de Negocio:** Justificación esencial escrita por el Product Owner, priorizando el desarrollo.
 - **Criterios de Aceptación:** Establecen límites y facilitan la comprensión para desarrolladores y testers v.
 - **Pruebas de Aceptación:** Detalles de la conversación que definen cuándo la historia está completa.
2. **Conversación:**
 - Es la **parte central**, destacando la **comunicación** entre el Product Owner y el Equipo de Desarrollo.
 - **Compensa la falta de especificación** y detalle mediante discusiones que desarrollan los detalles US.
 - Puede registrarse en grabaciones, minutas o software de gestión.
3. **Confirmación:**
 - **Acuerdo** entre el Product Owner y el Equipo **que demuestra la implementación** de la característica del producto.
 - Define la **conformidad con las pruebas de aceptación**.
 - Las pruebas son utilizadas para **determinar la finalización exitosa** de la User Story.

Ventajas

1. **Prioridad a la Comunicación Verbal:** Se reconoce que la conversación es clave.
2. **Comprensibles para Todos:** Son entendibles tanto para el cliente/usuario como para el equipo de desarrollo.
3. **Tamaño Apropiado para Planificación:** Poseen un tamaño adecuado que facilita la planificación del desarrollo.
4. **Desarrollo Iterativo:** Permiten el desarrollo iterativo al dividir todas las funcionalidades del producto en US.
5. **Flexibilidad para Detalles:** Permiten postergar los detalles hasta obtener una comprensión más clara.

Desventajas

1. **Dificultad en Proyectos Grandes:** Puede ser difícil entender la relación entre US, ya que son independientes.
2. **Inadecuadas para Equipos Muy Grandes:** Debido a la priorización de la comunicación verbal sobre la escrita.

PRODUCT BACKLOG Y LAS USER STORIES

El **Product Backlog** es una **lista dinámica de ítems**, conocidos como Product Backlog Ítems (PBIs), que contiene estimaciones de tamaño. Esta lista **está organizada de manera prioritaria**, con **las US de mayor importancia en la parte superior** y las de menor importancia en la parte inferior. La responsabilidad de la priorización recae en el **Product Owner**, quien realiza este proceso de **forma continua** a lo largo del proyecto. Se añaden nuevas US y actividades a medida que se identifican nuevos requerimientos durante el desarrollo del proyecto. **Para eliminar una US** del Product Backlog, **se planifica una iteración y se decide cuántas se quitarán**, comenzando por las de mayor prioridad.

- **Relación con la Arquitectura en Capas:** Las US **se extienden a través de todas las capas de la arquitectura** de software, incluyendo la capa de datos, la lógica de negocio y la capa de presentación. Este enfoque vertical **permite entregar características totalmente desarrolladas y funcionales**, abarcando todos los aspectos.

TAMAÑO

Es aconsejable tener un mayor número de historias con **tamaños relativamente pequeños** en lugar de historias muy grandes. El tamaño de las User Stories se mide en **Story Points**. Vamos a retomar y profundizar este tema más adelante.

NIVELES DE ABSTRACCIÓN

1. **User Story:**
 - ★ Cumple con el **criterio de Listo (DoR)** para ingresar a una iteración. (lo vemos más adelante)
2. **Épica:**
 - ★ Una **US de gran tamaño**, determinada por si puede o no entrar en una iteración.
 - ★ Comparte la **misma sintaxis** que una US estándar.
 - ★ Se identifica cuando **no cumple con** el modelo de calidad de **INVEST**. (lo vemos más adelante)
 - ★ **Requiere ser dividida** en US más pequeñas para su implementación.
 - ★ Utilizada para **requerimientos** que **no son prioritarios** en las **primeras iteraciones**, permitiendo una comprensión general de las funcionalidades necesarias. La implementación posterior implica dividirla en User Stories más pequeñas.
3. **Temas:**
 - ★ **Conjunto de US** agrupadas debido a su relación conceptual.
 - ★ Agrupación que **facilita la estimación y planificación** al considerarlas como una entidad única.
 - ★ Tanto Épicas como Temas ayudan a **simplificar los esfuerzos** del equipo durante la estimación.

MODELADO DE ROLES

En el desarrollo ágil, el modelado de roles se realiza **a través de las tarjetas de rol de usuario**, donde se **registran** las **características** generales que los usuarios tendrán en relación con el producto. Esta práctica se centra en comprender **quiénes utilizarán** la **funcionalidad** y es **crucial** para **entender la naturaleza** de esta.

Además de las tarjetas de rol, se emplean **técnicas adicionales** para un análisis más detallado:

1. Personas:

- **Descripciones específicas** de las personas concretas que utilizarán el sistema.
- Análisis exhaustivo y detallado para obtener información específica sobre los usuarios.
- Se puede pensar en analogía, donde si los roles son clases, las personas serían los objetos.

2. Personajes Extremos:

- Explora cómo **personas en situaciones extremas** podrían interactuar con el software.
- Proporciona una comprensión más completa de los posibles escenarios de uso al considerar casos límite. (se prende fuego el sistema?)

3. Proxies (Usuarios Representantes): (Me hace acordar a los Pixies de los padrinos mágicos)

- Involucra a una **persona** que actúa como **representante** de todos los usuarios.
- Esta persona debe tener la capacidad de **tomar decisiones desde la perspectiva del negocio**.



CRITERIOS DE ACEPTACIÓN

Los criterios de aceptación, aunque no son obligatorios, **establecen un acuerdo sobre el comportamiento necesario** del software **para que el Product Owner (PO) acepte la implementación**. Es esencial que estén redactados de manera **objetiva** y **concreta**, proporcionando un **marco claro** para que los desarrolladores evalúen si la implementación es satisfactoria.

Importancia y Funciones de los Criterios de Aceptación:

- ☺ Definen **límites** para una User Story (US).
- ☺ Facilitan que el PO especifique **lo necesario** para que la US aporte valor (requerimientos funcionales mínimos).
- ☺ Contribuyen a una **visión compartida** de la US dentro del equipo.
- ☺ **Ayudan a desarrolladores y testers** a derivar las pruebas.
- ☺ **Orientan a los desarrolladores** sobre cuándo detener la adición de funcionalidades en una US.

Características de Criterios de Aceptación Buenos:

- Definen una **intención**, no una solución específica. Ej. "El usuario debe elegir al menos una cuenta para operar".
- Son **independientes** de la implementación.
- Tienen un **nivel de abstracción** relativamente **alto**, sin necesidad de detallar cada aspecto.

Manejo de Detalles en Criterios de Aceptación:

- Detalles específicos, como el formato del saldo o el uso de una lista desplegable, se derivan de conversaciones.
- Pueden ser almacenados en la documentación interna del equipo o en pruebas de aceptación automatizadas.
- La elección de dónde y cómo mantener estos detalles depende de la decisión del equipo.

PRUEBAS DE ACEPTACIÓN

Las pruebas de aceptación son **declaraciones que deben probarse**, incorporando el conocimiento del negocio. **Evalúan escenarios exitosos y fallidos**, siendo **cruciales** ya que es impracticable cubrir todas las posibles pruebas.

Importancia y Funciones:

- **Decisivas** para la aceptación del Product Owner.
- **Reflejan detalles** de conversaciones entre clientes y desarrolladores.
- **Utilizadas para complementar** los detalles de la User Story.

Recomendaciones:

- Deben **redactarse antes del inicio de la programación**.
- Idealmente, el cliente las redacta (sí, ajá).

DOR (DEFINITION OF READY) – DEFINICIÓN DE LISTO

Es una **medida de calidad** que determina si la **US** está **en condiciones de entrar a una iteración** de desarrollo. **Para que la US pueda ser implementada**, mínimamente **debe satisfacer el INVEST Model**.

Entre el **equipo de desarrollo** y el **PO** pueden **definir características o condiciones extras al INVEST** para que una US se considere lista para entrar a la iteración de desarrollo.

INVEST MODEL

Como dijimos, el modelo **INVEST** es una **guía de calidad** que **verifica si una US cumple con los requisitos mínimos** para ingresar a una iteración de desarrollo.

Principios de INVEST:

- **Independiente:** La US es calendarizable e implementable **sin depender de otras**, permitiendo flexibilidad en la priorización por el Product Owner.
- **Negociable:** Las User Stories **no son contratos estrictos**, sino descripciones de funcionalidades que **se negocian en conversaciones futuras**, centrándose en "qué" se necesita, no en "cómo" se implementa.
- **Valuable:** Debe **aportar valor** de negocio a los usuarios, no solo al equipo, excluyendo detalles tecnológicos.
- **Estimable:** Contiene **información suficiente para estimar tamaño, complejidad y esfuerzo**, eliminando incertidumbres antes de la implementación.
 - Hay tres razones por las cuales una US puede no ser estimable:
 1. El equipo tiene incertidumbre acerca del dominio: Discutir con el PO para esclarecer las dudas.
 2. El equipo tiene incertidumbre técnica: Deberá destinarse miembros del equipo a uno o más Spikes.
 3. La US es demasiado grande: En este caso, deberá dividirse en US más pequeñas.
- **Pequeña (Small):** Debe ser lo suficientemente **pequeña para completarse en una iteración**, evitando trabajos a medias y aplicando una gestión binaria.
- **Testable:** **Cumple con pruebas de aceptación** para demostrar su implementación y ser probada.
-

DOD (DEFINITION OF DONE) – DEFINICIÓN DE HECHO

La Definición de Hecho (**DoD**) **establece los criterios** que determinan **cuándo una US está completa y lista para ser mostrada al Product Owner**. Es un **conjunto de condiciones** que, cuando se cumplen, **indican** que la US ha sido **implementada de manera correcta y completa**. La DoD asegura transparencia al proporcionar al equipo una comprensión clara del trabajo realizado en un incremento. Puede constar de un **checklist de condiciones específicas** y **puede variar entre organizaciones o equipos**. Su objetivo es garantizar que la implementación cumpla con los estándares establecidos antes de pasar a producción.

SPIKES

Los Spikes son **User Stories especiales** que surgen debido a la **incertidumbre** que presentan, **impidiendo su estimación e incumpliendo la definición de listo**. Una vez que se resuelve la incertidumbre, el Spike se transforma en una o más User Stories. Estas características **requieren tiempo para que el equipo se familiarice** con nuevas tecnologías o dominios, investigue y prototipe para superar riesgos tecnológicos o funcionales. Los Spikes se dividen en técnicos y funcionales y se utilizan para diversas finalidades, como la familiarización con nuevas tecnologías, el análisis de historias complejas y la mitigación de riesgos tecnológicos y funcionales.

- **Spikes técnicos:** Evaluación de **enfoques técnicos y tecnológicos**, asociadas a la **implementación** de funcionalidades, y dependientes de los conocimientos técnicos del equipo.
- **Spikes funcionales:** Utilizadas cuando hay **incertidumbre sobre la interacción del usuario** con el sistema, generalmente evaluadas con prototipos y dependientes de definiciones del negocio y del Product Owner.

Lineamientos para Spikes

- **Estimables, demostrables y aceptables.**
- **Excepción, no regla;** se reservan para incertidumbres críticas y grandes.
- **Implementación en una iteración separada**, salvo casos pequeños y simples donde se puede incluir en la misma iteración que las historias resultantes.

INVESTMENT THEMES (TEMAS DE INVERSIÓN)

Los Investment Themes son **conjuntos de propuestas de valor** que **guían la inversión** de una empresa en sistemas, productos, aplicaciones y servicios, con el propósito de diferenciarse en el mercado y/u obtener ventajas competitivas. Estos temas abarcan diversas áreas:

1. **Inversión en productos existentes:** Mejoras, soporte y mantenimiento.
2. **Inversión en nuevos productos y servicios:** Mejorar beneficios y ganar porciones de mercado a corto plazo.
3. **Inversión a futuro:** Dirigida a productos y servicios avanzados.
4. **Inversión a largo plazo:** Se invierte hoy con beneficios proyectados para el futuro.
5. **Inversión en reducción:** Estrategia de retiro para ofertas cercanas al final de su vida útil.

ESTIMACIÓN

La estimación es un **proceso crucial** para **prever la completitud** y **gestionar riesgos** en proyectos de software, según McConnell. Aunque **compleja**, se busca **predecir la duración y el costo del proyecto**. Sin embargo, las estimaciones **no son planes ni compromisos definitivos**.

Errores Comunes en las Estimaciones

1. **Información imprecisa** sobre el **software** o la **capacidad** para realizar el proyecto.
2. **Caos en el proyecto** debido a una **definición deficiente**.
3. **Imprecisión** generada durante el proceso **de estimación**.
4. **Omisión de actividades** necesarias para la estimación, como requerimientos faltantes.
5. **Confusión entre planear y estimar**; son la base del plan, pero no son idénticos.
6. A mayor **diferencia entre plan y estimación**, mayor riesgo.
7. Las estimaciones **no son compromisos**.

Razones para Estimar

1. **Predecir Completitud**: Saber cuánto trabajo se realizará, los recursos necesarios y el tiempo llevará.
2. **Administrar Riesgos**: Proporcionar información cuantitativa para compromisos, aunque la información cualitativa sea limitada.

Al inicio del proyecto, las estimaciones **suelen ser imprecisas** debido a la **falta de información** y la **incertidumbre**. A medida que avanza el proyecto, **con más información**, las estimaciones **se vuelven más certeras**. La primera estimación puede diferir hasta un 400% (ke), ya que **es optimista** y se recalcula a medida que progresa el proyecto. La máxima: "el que no sabe estimar, estima muchas veces, y el que no sabe planificar, planifica muchas veces".

PROCESO DE ESTIMACIÓN

Para ver cómo y qué se estima en el *ambiente tradicional* ver el punto de **Estimación**. De la U1

MÉTODOS UTILIZADOS PARA ESTIMAR

Se aconseja utilizar **diversos métodos de estimación** y **contrastarlos** para obtener un "Factor de Ajuste". Los principales métodos incluyen:

1. **Basados en la Experiencia**:
 - **Datos Históricos**: Utilización de **información de proyectos anteriores** para construir una base de conocimientos.
 - i. **Industry Data**: datos de otras organizaciones que aportan al mercado y desarrollan productos con algún grado de semejanza y que permite una comparación básica.
 - ii. **Historical Data**: datos de la organización de proyectos que se desarrollaron y ya se cerraron.
 - iii. **Project Data**: datos del proyecto, pero de etapas anteriores a la que se está estimando.
 - **Juicio Experto**: Estimaciones basadas en la **experiencia individual o grupal de expertos**. Puede ser propenso a errores si se usa como única técnica.
 - i. **Juicio Experto Puro**: Un experto evalúa las especificaciones y realiza una estimación basada principalmente en su experiencia. La organización pierde su capacidad de estimación si el experto se va.
 - ii. **Juicio Wideband Delphi**: Enfoque grupal similar al juicio experto. Un grupo de personas se reúne con el objetivo de llegar a una estimación común en esfuerzo y duración. Un líder reúne las estimaciones individuales, se discuten de forma anónima, se revisan y se repiten iterativamente hasta llegar a un acuerdo razonable.
2. **Analogía**:
 - **Comparación relativa de factores estimativos** con proyectos similares (como tamaño, complejidad o usuarios). Aunque común, puede tener un alto grado de error debido a la no extrapolación del conocimiento entre proyectos.
3. **Otros Métodos**:
 - **Basados en Recursos**: Estimaciones centradas exclusivamente en el personal y el tiempo disponible.
 - **Basados en el Cliente o Mercado**: Se enfocan en la capacidad de pago del cliente. A veces se ofrece un precio más bajo para superar a la competencia.
 - **Basados en Componentes del Producto o Proceso de Desarrollo**:
 - **Bottom-up**: **Descomposición del producto en unidades pequeñas** para estimar cada componente.
 - **Top-down**: **Descomposición en bloques grandes** antes de realizar estimaciones detalladas.
 - **Métodos Algorítmicos**: Utilización de **algoritmos con parámetros cuantitativos** para obtener una estimación de esfuerzo. A pesar de ser matemático, la precisión no está garantizada.

PROBLEMAS DE ESTIMACIONES TRADICIONALES

- Estimar demasiado pronto.
- Resistencia a las reestimaciones.
- Estimaciones que buscan precisión.
- Estimaciones hechas por gente distinta a la que hace el trabajo.
- Estimaciones que pretenden ser absolutas (inflexibles).

ESTIMACIONES EN AMBIENTES ÁGILES

Las estimaciones ágiles son **suposiciones numéricas** vinculadas a la **probabilidad de que ocurra un evento** específico. En metodologías ágiles, el equipo de desarrollo, de manera colectiva, realiza estas estimaciones, fomentando debates para una mayor precisión.

- **Enfoque Colaborativo:** Quien estima es quien realiza el trabajo, promoviendo una perspectiva más precisa y colectiva. El Product Owner no participa en las estimaciones.
- **Importancia del Diálogo:** Diálogo abierto y escucha activa son fundamentales para enriquecer las estimaciones. Evita influencias que puedan sesgar opiniones.
- **Riesgos en Estimaciones Ágiles:**
 - **Subestimación:** Simplificar el trabajo por miedo al rechazo del cliente.
 - **Sobreestimación:** Inflar la complejidad del producto como medida preventiva.
- **Estimaciones vs. Compromisos:** Las estimaciones informan la planificación, pero no son compromisos rígidos.
- **Proceso Evolutivo:** A medida que avanza el proyecto, la info aumenta y las estimaciones son más precisas.
- **Beneficios del Proceso de Estimación:**
 - Enriquece el trabajo del equipo.
 - Mejora futuros procesos de estimación.
 - Evalúa la factibilidad del trabajo planificado en etapas tempranas.
- **Enfoque Ágil en Estimación:** Se destina un esfuerzo bajo en la estimación, y para esto el tiempo de estimación se define con antelación para limitar el esfuerzo. Se busca acercarse a la realidad sin una precisión excesiva.
- **Consideraciones Importantes:**
 - Medidas relativas, no absolutas.
 - No basada en tiempo.
 - Comparaciones más efectivas que estimaciones absolutas.
 - Mayor aprovechamiento del tiempo de análisis de las User Stories.
- **Inicio con Estimación Revisable:**
 - Ideal comenzar con una estimación que pueda revisarse según la evolución del proyecto.
 - Enfoque en el producto y adaptabilidad.

¿QUÉ SE ESTIMA?

En entornos ágiles, la estimación se centra exclusivamente en **el tamaño de las User Stories (US) o Features**, utilizando **Story Points** como unidad de medida. Se tienen en cuenta tres aspectos clave: incertidumbre, complejidad y esfuerzo. Dado que la **complejidad tiende a aumentar exponencialmente**, se emplean **series numéricas**, como la serie de **Fibonacci**, para reflejar estas variaciones.

Estas estimaciones son de **tamaños relativos**, enfocándose en **comparaciones** entre las US. La importancia recae en la **consistencia dentro del equipo**; no tanto en el valor asignado a cada US, sino en que las estimaciones sean coherentes en el conjunto. Aunque el valor de una US puede variar entre equipos, la clave es mantener la coherencia interna para facilitar la comprensión.

Es fundamental destacar que **la estimación no aborda el tiempo**, ya que la duración de los Sprints es constante (Timebox). Por esta razón, desaconseja estimar en horas para evitar confusiones entre el tamaño y el esfuerzo asociado.

TAMAÑO

El tamaño se refiere a la **medida de la complejidad y magnitud** de una User Story (US) o Feature, considerando la **cantidad de trabajo necesario** para su desarrollo. **También se relaciona con la incertidumbre**, entendida como la falta de información sobre la US.

Formas de Estimar el Tamaño: Se emplean diversas escalas comparativas para estimar el tamaño, como:

1. **Número del 1 al 10.**
2. **Talles de remeras (S, M, L):** Utilizado como escala para ítems en el Product Backlog.
3. **Serie exponencial de base 2:** 2, 4, 8, 16, 32, 64, etc.
4. **Serie de Fibonacci:** 0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, etc. (Esta es la más usada ya que decimos que la complejidad es exponencial)

Una vez definida la escala, se mantiene constante. Cambiarla implica ajustar el patrón de medición correspondiente. Estas escalas facilitan la comparación relativa del tamaño de las US durante la estimación.

TAMAÑO VS. ESFUERZO

Como el **esfuerzo (horas de trabajo)** es el factor más significativo del costo de un proyecto, tenemos una gran tendencia a confundir tamaño con esfuerzo y esfuerzo con calendario. **La complejidad es independiente de quien haga el proyecto, y el esfuerzo depende específicamente de la persona** que lo va a llevar a cabo, de su capacitación, conlleva horas lineales (horas únicamente de trabajo, sin tener en cuenta almuerzo, baño, leer emails, etc. Para una jornada de trabajo de 8 horas, usualmente solo 5.5 o 6 son lineales). Por otro lado, **no se debe confundir esfuerzo con calendario**, ya que yo puedo decir que puedo tener un trabajo para el viernes, pero en el medio pueden suceder millones de cosas que evitan que cumpla mi objetivo. Un trabajo puede requerir un esfuerzo de x horas, pero no se puede asegurar que para tal día va a estar terminado

STORY POINTS

El Story Point es una **unidad de estimación** que **refleja la complejidad, incertidumbre y esfuerzo** percibidos por el **equipo** en relación con una **User Story (US)**. Sirve para **asignar un "peso"** relativo a cada US, determinando su tamaño y complejidad **en comparación con otras**. La complejidad tiende a aumentar exponencialmente.

Componentes:

- **Complejidad:** Refiere a la **dificultad inherente a la funcionalidad**, que puede impedir su división en User Stories más pequeñas. Ejemplos de complejidad incluyen algoritmos complicados, cálculos extensos, gran cantidad de campos, numerosas validaciones y filtros.
- **Esfuerzo:** Necesita ser estimado por la persona encargada del trabajo, ya que el esfuerzo varía entre individuos. Representa las **horas persona lineales** requeridas para completar la US.
- **Incertidumbre:**
 - **Técnica:** Surge cuando existen **dudas sobre el dominio de la solución**, como el uso de tecnologías menos familiares para el equipo.
 - **De negocio:** Ocurre cuando hay incertidumbre sobre la interacción del usuario con el sistema debido a la **falta de conocimiento en el dominio del negocio**
 -

VELOCIDAD

La velocidad, una **métrica clave en ágil (es LA métrica)**, **representa el progreso del equipo** y se calcula sumando los **Story Points** asignados a las US **completadas en cada iteración**. Es una **medida binaria**: o se completa una US según la Definición de Hecho (DoD) o no. Enfoques en el promedio de velocidades a lo largo de iteraciones sucesivas brindan una visión más precisa del progreso del equipo en lugar de centrarse en velocidades individuales.

REESTIMACIÓN

La **reestimación** se aconseja **solo cuando hay un cambio en la percepción del tamaño relativo de una US**, basado en descubrimientos durante el proceso de estimación. No se debe realizar reestimaciones simplemente debido a discrepancias en la velocidad del equipo; la consistencia de tamaños en estimaciones sucesivas se ajusta naturalmente a lo largo del tiempo.

PROPÓSITO DE LA ESTIMACIÓN EN DESARROLLO ÁGIL

1. **Compromiso en la Iteración:**
 - Determinar **cuántas User Stories (US)** puede asumir el equipo en una **iteración** inicial.
2. **Medición y Velocidad del Sprint:**
 - Al finalizar la iteración, contar los Story Points de las US aceptadas por el Product Owner **para calcular la velocidad del sprint**, proporciona predictibilidad si se mantiene constante en el tiempo.

ESTIMACIÓN DE LA DURACIÓN DEL PROYECTO

Utilizando **Story Points para medir la complejidad** relativa de las User Stories, se estima la duración del proyecto **dividiendo el número total de Story Points por la velocidad del equipo**.

POKER PLANNING O POKER ESTIMATION

Es una técnica de estimación ágil basada en **juicio experto, analogía y desagregación**, desarrollada por Mike Cohn. **Los miembros del equipo estiman** su propio trabajo, fomentando la **mezcla de experiencia y conocimiento**. El Product Owner participa, pero no realiza estimaciones.

Escala de Fibonacci:

- **1 o 1/2:** Funcionalidad pequeña.
- **2-3:** Funcionalidad pequeña a mediana.
- **5:** Funcionalidad mediana.
- **8:** Funcionalidad grande (posible división).
- **13 o más:** Requiere división, no cabe en el sprint.

Procedimiento:

1. Se define una **lista de actividades, módulos, casos de uso, US**, etc.
2. **Se elige una referencia canónica** (US canónica) de baja complejidad.
3. Cada miembro elige una carta con su estimación.
4. Se **revelan las estimaciones** simultáneamente y **se justifican** discrepancias.
5. **Si hay diferencias** notables, **se repite** el proceso.
6. Si no hay convergencia, se llega a una estimación intermedia.

Consideraciones:

1. Considerar el **criterio de Done** y todas las actividades relacionadas.
2. El **tiempo** para estimar es **limitado** (time-boxing).
3. **Estimar las mínimas US posibles**, siguiendo el principio de diferir compromisos.

Ventajas:

- **Opiniones multidisciplinarias** enriquecen la estimación.
- **Debates** mejoran la comprensión del dominio.
- **Actividad divertida** para el equipo.

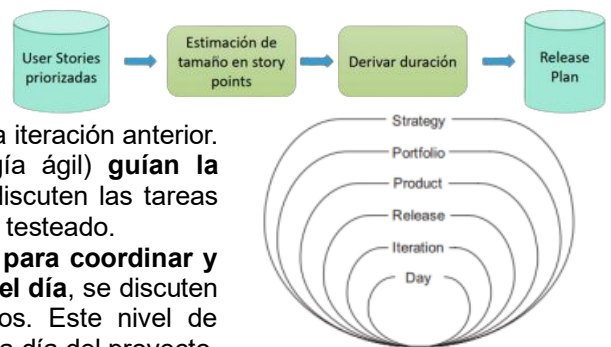
PLANIFICACIÓN

En entornos ágiles, la **planificación es un proceso continuo y adaptable**. Más que centrarse en documentación extensa, se valora el proceso de planificación en sí mismo. Los equipos ágiles abordan la planificación en **tres niveles clave**:

1. Planificación de Release: Inicia al **comienzo del proyecto** y **evoluciona** a lo largo del tiempo. Se **analizan y determinan** las **User Stories, épicas o Temas a desarrollar** en una entrega (**Release**) del producto. La **priorización y estimación** con Story Points, considerando la velocidad del equipo, determinan la **duración de la Release** o proyecto. **Se actualiza periódicamente** para reflejar las expectativas actuales.

2. Planificación de Iteración: Realizada **al inicio de cada iteración**, se basa en la **priorización del Product Owner** según la iteración anterior. La **velocidad del equipo y el tiempo constante** (metodología ágil) **guían la selección de US** para la iteración. Durante la planificación, se discuten las tareas necesarias para transformar las User Stories en código funcional y testeado.

3. Planificación del Día: Se lleva a cabo en **reuniones diarias para coordinar y sincronizar el trabajo** del equipo. Enfocada en las **actividades del día**, se discuten las tareas esenciales para cumplir con los objetivos específicos. Este nivel de planificación asegura una ejecución efectiva y adaptativa en el día a día del proyecto.



ESTIMACIONES EN LEAN

En Lean, una metodología **más extrema que las ágiles**, se considera que las estimaciones **son opcionales y a veces** pueden ser un **desperdicio**, ya que tienen **alta probabilidad de no coincidir** con la realidad debido a cambios en variables.

ESTIMACIONES EN PROYECTOS PEQUEÑOS

En proyectos pequeños, se sugiere utilizar **datos históricos internos**, ya que los datos de la industria pueden no ser apropiados. Las estimaciones para proyectos pequeños **dependen en gran medida de las habilidades individuales**, y el SEI propone el uso del Personal Software Process, especialmente para proyectos muy pequeños o "mini proyectos".

DIFERENCIAS CON LAS ESTIMACIONES EN AMBIENTES TRADICIONALES

1. Dimensiones en Agile vs. Tradicional:

- **Agile:** Tiempo y recursos fijos, alcance variable. Producto no completamente definido, visión general.
- **Tradicional:** Alcance fijo, producto definido. Se estima primero el tamaño como base.

2. Precisión:

- **Agile:** Busca rapidez y flexibilidad, considera la precisión como un desperdicio.
- **Tradicional:** Busca mayor precisión, base para la planificación.

3. Relatividad vs. Absolutismo:

- **Agile:** Estimaciones relativas, medidas de tamaño relativo para características del producto.
- **Tradicional:** Estimaciones absolutas, busca precisiones detalladas.

4. Responsabilidad de Estimación:

- **Agile:** El equipo realiza las estimaciones, cada miembro estima su propio trabajo.
- **Tradicional:** Líder del proyecto o experto realiza las estimaciones, potencial desviación.

5. Confusión con Planificación:

- **Agile:** Estimaciones no son compromisos, están en el ámbito de las probabilidades.
- **Tradicional:** Estimaciones asumidas como compromisos, llevan a promesas.

6. Momentos para Estimar:

- **Agile:** Estimaciones al inicio de cada sprint, ajuste en alcance, evolución certera a lo largo del proyecto.
- **Tradicional:** Estimaciones al inicio, tras requerimientos y diseño, ajustes al modificar el alcance.

SCRUM

Scrum es un **marco ligero** diseñado para **generar valor** en **soluciones adaptables** para **problemas complejos**. En resumen, requiere un **Scrum Master** para facilitar un entorno donde:

1. El Product Owner organiza el trabajo en un **Product Backlog**.
2. El equipo convierte parte del trabajo en un **Incremento de valor durante un Sprint**.
3. El equipo y los stakeholders **inspeccionan los resultados** y **realizan ajustes** para el próximo Sprint.
4. Se repite el ciclo.

Scrum es **simple** y se basa en la **inteligencia colectiva** de quienes lo utilizan. Es **deliberadamente incompleto**, definiendo solo las **partes esenciales** y **guiando relaciones e interacciones** en lugar de proporcionar instrucciones detalladas. Puede emplear diversos procesos y técnicas, haciendo visibles la eficacia y permitiendo mejoras.

Basado en **empirismo** y **pensamiento Lean**, Scrum utiliza un enfoque **iterativo e incremental** para optimizar previsibilidad y controlar riesgos. Involucra a grupos con **todas las habilidades necesarias** y combina **eventos formales** dentro de un Sprint, implementando los pilares empíricos: transparencia, inspección y adaptación. (en pág. 14)

VALORES DE SCRUM

El éxito con Scrum depende de vivir cinco valores fundamentales:

Compromiso: El equipo se compromete a alcanzar objetivos

Enfoque: se enfoca en el progreso durante el Sprint

Apertura: mantiene apertura sobre el trabajo y desafíos

Respeto: se respeta mutuamente como individuos capaces e independientes

Coraje: tiene el coraje de abordar problemas complejos.

Estos valores **guían las decisiones, acciones y comportamiento** del equipo de Scrum, y deben ser reforzados en todas las medidas y acciones tomadas. Asimilar estos valores construye confianza y da vida a los pilares empíricos de Scrum: transparencia, inspección y adaptación.

EL EQUIPO SCRUM (SCRUM TEAM)

El equipo Scrum, **unidad fundamental** en Scrum, consta de un **Scrum Master**, un **Product Owner** y **desarrolladores**, **sin jerarquías** ni subequipos. Es **multifuncional y autogestionado**, con **habilidades completas** para crear valor en cada Sprint. Tienen un **tamaño óptimo**, generalmente **10 o menos** personas, para una **comunicación efectiva**. El equipo es responsable de todas las actividades relacionadas con el producto y trabaja en Sprints a un **ritmo sostenible**. Todos comparten la responsabilidad de crear un incremento valioso en cada Sprint, con roles definidos (3): desarrolladores, Product Owner y Scrum Master.

DESARROLLADORES

Los desarrolladores en Scrum son **miembros del equipo** que se comprometen a **crear un Incremento funcional y útil** en cada Sprint. Son responsables de **planificar el Sprint (sprint backlog)**, mantener la **calidad** según la Definición de Hecho, **adaptar el plan** hacia el **Objetivo Sprint** y **responsabilizarse mutuamente**.

PROPIETARIO DEL PRODUCTO (PRODUCT OWNER)

El Product Owner **maximiza el valor** del producto, gestionando el **Product Backlog** (PB para los amigos). Sus responsabilidades incluyen **desarrollar y comunicar el Objetivo** del Producto, **ordenar el PB** y asegurar la **transparencia** y **comprensión del trabajo pendiente** del producto. El Product Owner es una **persona**, **toma decisiones** respetadas por toda la organización.

SCRUM MASTER

El Scrum Master **establece Scrum**, ayuda a **comprender la teoría y práctica** de Scrum en el equipo y la organización. Su enfoque es **la efectividad del equipo** Scrum, **facilitando mejoras** y **eliminando impedimentos**. El Scrum Master desempeña múltiples roles:

Para el equipo de Scrum:

1. Capacita a los miembros en autogestión y multifuncionalidad.
2. Ayuda al equipo a enfocarse en crear incrementos de alto valor que cumplan con la DoD.
3. Promueve la eliminación de impedimentos para el progreso del equipo.
4. Asegura que todos los eventos de Scrum sean positivos, productivos y se respeten los tiempos establecidos.

Para el Propietario del Producto (Product Owner):

1. Ayuda a encontrar técnicas para una definición eficaz de los objetivos del producto y la gestión de los retrasos.
2. Ayuda a comprender la necesidad de elementos (de trabajo pendiente) claros y concisos.
3. Ayuda a establecer la planificación empírica de productos para un entorno complejo.
4. Facilita la colaboración de las partes interesadas según sea necesario.

Para la organización:

1. Lidera y capacita a la organización en la adopción de Scrum.
2. Planifica y asesora sobre la implementación de Scrum dentro de la organización.
3. Ayuda a las personas y partes interesadas a comprender y promulgar un enfoque empírico.
4. Elimina las barreras entre las partes interesadas y los equipos de Scrum.

EVENTOS DE SCRUM

Los eventos de Scrum están **encapsulados en el Sprint**, sirviendo como **oportunidades formales** para **inspeccionar y adaptar los artefactos** del proceso. La **ejecución adecuada** de estos eventos es **crucial**, ya que perderlos implica perder oportunidades valiosas para la mejora continua. Estos eventos se utilizan para **establecer regularidad y transparencia** en el proceso, y **minimizar la necesidad de reuniones no definidas** en Scrum. La coordinación óptima se logra cuando todos los eventos se llevan a cabo en el **mismo tiempo y lugar**, **reduciendo la complejidad** del proceso.

SPRINT

El Sprint en Scrum es el **motor central** (como DePaul) donde **las ideas se materializan** en valor (que poético). Durante un **período fijo de tiempo**, generalmente sugerido en 30 días o dos semanas, se lleva a cabo un conjunto de **eventos clave**, como la Planificación del Sprint, Daily Scrums, Revisión del Sprint y Retrospectiva. Estos eventos ocurren **ininterrumpidamente** y se centran en **producir un incremento** utilizable basado en el producto anterior.

Durante el Sprint, **se congela el tiempo, los costos y la calidad**, pero **la funcionalidad puede variar siempre que se alcance el Objetivo** del Sprint. **No se permiten cambios** que pongan en peligro este objetivo, y se enfoca en **refinar continuamente el trabajo pendiente** del producto. El equipo puede descubrir y abordar más trabajo durante el Sprint, y los cambios en requisitos se ponen en el Product Backlog y se vuelve a priorizar. La flexibilidad en la funcionalidad es posible, pero solo el Product Owner puede eliminarla. El Scrum Master juega un papel crucial al identificar y resolver impedimentos durante el Sprint. Los Sprints **garantizan la previsibilidad** al permitir la **inspección y adaptación** del progreso hacia el objetivo del producto.

Para mantener la consistencia, un nuevo Sprint comienza **de inmediato** después del anterior. Se utilizan diversas **prácticas para pronosticar el progreso**, como gráficos de burn-downs, burn-ups o flujos acumulativos, aunque se destaca que ninguna sustituye la importancia del **empirismo**, especialmente en entornos complejos. Si el **Objetivo** del Sprint se vuelve **obsoleto**, solo el Propietario del Producto tiene la autoridad para **cancelar el Sprint**, lo cual debe considerarse cuidadosamente dada su implicación organizacional y costosa naturaleza.

1. Planificación de Sprint (Sprint Planning)

La Planificación de Sprint en Scrum **inicia el Sprint** al **establecer el trabajo a realizar**, siendo un esfuerzo colaborativo del equipo. Dirigida por el **Product Owner**, aborda tres temas clave: la **valoración del Sprint**, la **selección de elementos del Product Backlog** y la **planificación detallada del trabajo**. La definición del **Objetivo Sprint** debe concluirse durante esta planificación. Los **desarrolladores eligen cómo realizar el trabajo** seleccionado, desglosándolo en tareas manejables. El conjunto **resultante**, que incluye el **Objetivo Sprint** y el trabajo seleccionado, se denomina el **Sprint Backlog**. La duración máxima del Sprint Planning es de ocho horas para un Sprint de un mes, siendo más breve para Sprints más cortos.

2. Scrum diario (Daily Scrum)

El Daily Scrum, **un evento de 15 minutos** para los **desarrolladores**, tiene como objetivo **inspeccionar el progreso** hacia el **Objetivo Sprint** y **ajustar el Sprint Backlog** según sea necesario. Se realiza diariamente durante los días laborables del Sprint, en el mismo lugar y hora. El PO y el Scrum Master participan como desarrolladores si están trabajando en elementos del Trabajo pendiente de Sprint. Los desarrolladores **eligen la estructura y técnicas**, centrando la reunión en el progreso hacia el Objetivo Sprint y **generando un plan accionable** para el siguiente día de trabajo. Este enfoque **mejora la autogestión** y, al promover una **comunicación eficiente**, **identifica impedimentos**, **acelera la toma de decisiones** y elimina la necesidad de otras reuniones. Además, los desarrolladores pueden ajustar su plan en otras reuniones detalladas a lo largo del día.

3. Revisión del Sprint (Sprint Review)

La Revisión del Sprint tiene como objetivo **inspeccionar los resultados del Sprint** y **planificar adaptaciones** futuras. Durante este evento, el **equipo de Scrum presenta los logros** a las partes interesadas clave, **discutiendo el progreso hacia el Objetivo** de Producto. La revisión incluye la **evaluación de lo alcanzado** en el Sprint y los **cambios en el entorno**. Con base en esta información, los participantes colaboran en **decisiones futuras** y pueden ajustar el trabajo pendiente del producto para aprovechar nuevas oportunidades. Es esencial que la Sprint Review sea una **sesión de trabajo** y no solo una presentación, evitando limitarla a ese propósito. Este evento, el **penúltimo del Sprint**, tiene un límite máximo de cuatro horas para un Sprint de un mes, siendo más breve para sprints más cortos.

4. La retrospectiva del Sprint (Sprint Retrospective)

La Retrospectiva del Sprint tiene como propósito **planificar mejoras** en la **calidad y eficacia**. Durante este evento, el equipo de Scrum **analiza el último Sprint**, **evaluando aspectos** como **individuos, interacciones, procesos, herramientas** y su DoD. Se **identifican suposiciones desviadas**, explorando sus orígenes. Se **revisa lo que funcionó bien, los problemas encontrados y cómo se resolvieron** o no. El equipo **identifica cambios** beneficiosos para mejorar su eficacia, abordando las mejoras de manera inmediata. Estas mejoras **pueden agregarse al Sprint Backlog** para el próximo Sprint. La Retrospectiva concluye el Sprint y tiene un límite de hasta tres horas para un Sprint de un mes, siendo más breve para sprints más cortos.

5. Story Time/Grooming

Existe la opción de realizar **"Story Time/Grooming"** (de repente estoy en TikTok) o Refinamiento del Product Backlog, una reunión opcional donde el equipo se encuentra con el PO para discutir ítems de alto valor en el backlog. Aquí se establecen criterios de aceptación y se asigna prioridad a los nuevos ítems. Esta actividad puede ocurrir antes del desarrollo inicial y de manera iterativa en cada Sprint.

ARTEFACTOS DE SCRUM

Los artefactos de Scrum **representan trabajo o valor** y buscan **maximizar la transparencia** de la **información clave**. Cada artefacto contiene un **compromiso** que asegura que proporciona información para mejorar la transparencia y el enfoque medible del progreso. Estos compromisos incluyen el objetivo del producto para el trabajo pendiente del producto, el Sprint Goal para el Sprint Backlog y la DoD para el Incremento. Estos compromisos **refuerzan el empirismo y los valores de Scrum** para el equipo y sus partes interesadas.

Pila del Producto (Product Backlog)

La Pila del Producto **representa el trabajo necesario para mejorar el producto** y es la **única fuente de trabajo** para el equipo Scrum. Los **elementos listos para ser seleccionados** en un Sprint se determinan en la Planning, después de **actividades de refinamiento** que descomponen y definen los elementos. Este refinamiento es **continuo y agrega detalles**, como **descripciones, prioridades y tamaños**, con los **desarrolladores responsables del tamaño**. El Propietario del Producto puede influir en las decisiones proporcionando información.

Compromiso: Objetivo del Producto (Product Goal): El Objetivo del Producto describe un **estado futuro del producto**, sirviendo como **meta a largo plazo** para el equipo Scrum. Se encuentra en el trabajo pendiente del producto (Product Backlog). Este objetivo **guía la planificación y debe cumplirse o abandonarse** antes de asumir el siguiente. Un producto se define como un vehículo para entregar valor, con límites claros, partes interesadas conocidas y usuarios o clientes bien definidos, que puede ser un servicio, un producto físico o algo más abstracto.

Pila del Sprint (Sprint Backlog)

La Pila del Sprint comprende el **Objetivo Sprint (por qué)**, los **elementos seleccionados** del trabajo pendiente de producto para el Sprint (**qué**) y un **plan accionable** para lograr el incremento (**cómo**). Es un **plan en tiempo real** para los desarrolladores y **se actualiza durante el Sprint** a medida que se aprende más, proporcionando una imagen visible del trabajo planificado para **alcanzar el Objetivo Sprint**. Debe ser lo suficientemente **detaillado** para la **inspección diaria** en el Scrum Diario.

Compromiso: Objetivo del Sprint (Sprint Goal): El Objetivo del Sprint es la **única meta para el Sprint** y **proporciona flexibilidad** en la ejecución del trabajo necesario. Aunque es un compromiso de los desarrolladores, **crea coherencia y enfoque**, fomentando la **colaboración** en lugar de iniciativas separadas. Se establece durante la Sprint Planning. Si el trabajo difiere, el equipo colabora con el PO para ajustar el alcance sin afectar el Objetivo Sprint.

Incremento (Incremento)

Un Incremento representa un **avance concreto hacia el Objetivo del Producto** y es **acumulativo** con los Incrementos previos. Cada Incremento es completamente **verificado** y debe ser **utilizable** para **proporcionar valor**. Pueden crearse **múltiples Incrementos en un Sprint**, y su suma se presenta en la Revisión Sprint, respaldando así el empirismo. Un Incremento PUEDE entregarse a las partes interesadas antes del final del Sprint, la Revisión Sprint no se considera una puerta para liberar valor. Para ser parte de un Incremento, el trabajo debe cumplir con la **DoD**.

Compromiso: Definición de Hecho (Definition of Done): La Definición de Hecho es una **descripción formal del estado del Incremento** cuando **cumple con los estándares** de calidad requeridos. Cuando un elemento del trabajo pendiente del producto satisface la Definición de Hecho, **se considera un Incremento**. Esta definición **brinda transparencia** al proporcionar una comprensión compartida (debe ser seguida por todos los equipos de Scrum en un producto, ya sea un estándar organizativo o creado por el equipo) de lo que se ha completado en el Incremento.

TIMEBOX

El Timebox es una **práctica que evita la pérdida de tiempo** innecesario en tareas, fomentando la **negociación basada en el alcance** del producto en lugar de tiempo o costos (vinculado a la triple restricción). En caso de no completar un Sprint, en lugar de extender el tiempo, se entrega menos. Esta metodología busca **aumentar la confiabilidad y mantener un ritmo** de trabajo sostenible. **Todas las ceremonias** en Scrum, incluido el refinamiento del Product Backlog, **son Timebox**, asegurando **eficiencia y consistencia** en las actividades, aunque el tiempo para el refinamiento se expresa en términos porcentuales en relación con la definición del Sprint.



CAPACIDAD DEL EQUIPO EN UN SPRINT

La capacidad en Scrum es una **métrica** que ayuda a determinar **cuánto trabajo puede asumir un equipo** durante un Sprint. Esta estimación y medición se realizan durante el **Sprint Planning**. La capacidad del equipo ya sea en **Story Points** para equipos **más experimentados** o en **horas ideales** para **equipos menos maduros**, se contrasta con la **cantidad de elementos** de trabajo del Product Backlog que se pueden incorporar al Sprint Backlog. Este proceso permite una **planificación más precisa y ajustada a la capacidad real** del equipo.



NIVELES DE PLANIFICACIÓN

1. Portfolio:

- *Horizonte:* 1 año o más.
- *Responsables:* Stakeholders y Product Owners.
- *Enfoque:* Administración de un Portfolio de Producto.
- *Entregable:* Backlog de Portfolio.

2. Producto:

- *Horizonte:* Arriba de varios meses o más.
- *Responsables:* Product Owner y Stakeholders.
- *Enfoque:* Visión y evolución del producto a través del tiempo.
- *Entregable:* Visión de Producto, roadmap y características.

3. Release:

- *Horizonte:* 3 (o menos) a 9 meses.
- *Responsables:* Equipo Scrum entero y Stakeholders.
- *Enfoque:* Balancear continuamente el valor del cliente y la calidad global con las restricciones de alcance, cronograma y presupuesto.
- *Entregable:* Plan de release.

4. Iteración:

- *Horizonte:* Cada iteración (de 1 semana a 1 mes).
- *Responsables:* Equipo Scrum entero.
- *Enfoque:* Qué aspectos entregar en el siguiente Sprint.
- *Entregable:* Objetivo del Sprint y Sprint Backlog.

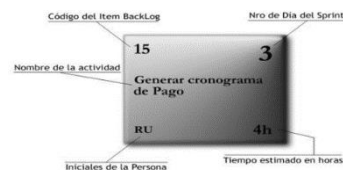
5. Día a Día:

- *Horizonte:* Diaria.
- *Responsables:* Scrum Master y Equipo de desarrollo.
- *Enfoque:* Cómo completar lo comprometido.
- *Entregable:* Inspección del progreso.

HERRAMIENTAS DE SCRUM

1. Tarjeta de Tarea:

- **Identificador del ítem** en el Product Backlog.
- Nombre de la actividad.
- Iniciales del responsable.
- Número de día del sprint.
- Estimación del tiempo para finalizar la tarea.



2. Tablero:

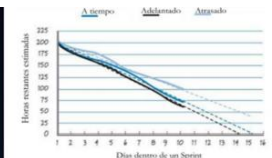
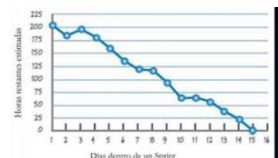
- Dividido en 5 secciones: **Story, To Do, Work in Process, To Verify, Done.**
- Detalla tareas por hacer, en proceso, para verificar y terminadas, representadas en tarjetas de tarea.
- Lo ideal es que el nivel de granularidad de las tareas sea lo **más fino posible**.

3. Gráficos del Backlog:

- Información sobre el **progreso del Sprint, release o producto**.
- Backlog de trabajo en horas y tendencia del backlog en días.

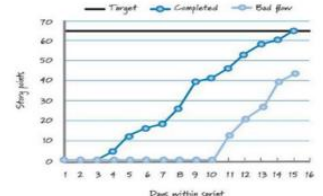
- **Sprint Burn-Down Chart:**

- Curva descendente que muestra **trabajo restante** en función del tiempo del sprint.
- Eje X: tiempo desde el inicio del sprint.
- Eje Y: puntos de historia por realizar en el sprint.



- **Sprint Burn-Up Chart:**

- Curva ascendente que visualiza **trabajo completado** a lo largo de la release.
- Muestra puntos de historia terminados en cada sprint.



- **Combined Burn Chart:**

- Visualiza trabajo restante y trabajo realizado en conjunto.

FRAMEWORKS PARA ESCALAR SCRUM

Dado que Scrum impone un límite en la cantidad máxima de integrantes del Equipo de Desarrollo, surge la necesidad de escalar este Framework para **poder gestionar productos de mayor envergadura**, ya que las prácticas ágiles se utilizan en **ambientes complejos** para reducir tal complejidad. Existen **diferentes frameworks** para escalar como **Nexus** (El que vimos en clase), **scale scrum**, **LESS**, **SAFe**.

NEXUS

Nexus es un framework que **coordina de tres a nueve equipos Scrum** que colaboran en un **único Product Backlog** para construir **un incremento integrado** de producto "Terminado". Se utiliza para **abordar la complejidad** que surge cuando múltiples equipos Scrum trabajan en un mismo Product Backlog. Nexus **gestiona las dependencias** entre equipos, especialmente en áreas **como requisitos superpuestos y conocimiento del dominio**, con el objetivo de lograr una **entrega eficiente y coordinada**. Cuenta con roles, eventos, artefactos y técnicas para facilitar esta colaboración. **Un solo Product Owner** lidera la entrega del producto.

RESPONSABILIDADES

-  **Nexus Integration Team:** Garantiza la producción de un **incremento integrado** en cada sprint, abordando **restricciones** técnicas y no técnicas que puedan obstaculizar la entrega consistente. **Incluye a miembros multifuncionales.**
-  **Product Owner:** Maximiza el valor del producto y del trabajo integrado por los equipos Scrum en el Nexus, gestionando eficazmente el **Product Backlog**.
-  **Scrum Master:** Asegura que el marco de trabajo **Nexus se comprenda y aplique según la Guía de Nexus**, pudiendo desempeñar el rol en uno o más equipos Scrum en el Nexus.
-  **Miembros del Nexus Integration Team:** Colaboran con los **Scrum Teams** para adoptar herramientas y prácticas que mejoren su capacidad para entregar un Incremento Integrado útil y valioso, cumpliendo con la DoD.

EVENTOS

Nexus **agrega o extiende los eventos definidos por Scrum**. La duración de los eventos Nexus se guía por la duración de los eventos correspondientes en la Guía de Scrum. Tienen definido un bloque de tiempo adicional a sus correspondientes eventos de Scrum.

- **Nexus Sprint:** Equipos Scrum producen un **Integrated Increment** en un sprint coordinado.
- **Refinamiento entre Equipos:** Reduce dependencias entre equipos, descomponiendo el Product Backlog para hacer transparentes y abordar dependencias. **Es continuo.** La frecuencia, duración y participación del Refinamiento Entre Equipos varía para optimizar estos dos propósitos.
- **Nexus Sprint Planning:** Coordina actividades de todos los **Scrum Teams** para un solo Sprint, estableciendo **objetivos** de Sprint para cada equipo y un **objetivo común** para el Nexus.

Resultados:

- Objetivo de Sprint para cada Scrum Team.
- Objetivo de Nexus.
- Un Nexus sprint backlog.
- Un Sprint backlog.
- **Nexus Daily Scrum:** Reunión diaria donde **representantes** de cada **Scrum Team** discuten **problemas de integración y revisan el progreso** hacia el objetivo del Nexus.
- **Nexus Sprint Review:** Al final del Sprint, proporciona **retroalimentación** al **Integrated Increment** y **determina adaptaciones futuras**, *reemplazando los sprint reviews* individuales de cada Scrum Team.
- **Nexus Sprint Retrospective:** Planificación para **mejorar la calidad y eficacia** en todo el Nexus, inspeccionando el último Sprint y complementando las Retrospectivas de los Scrum Teams para abordar problemas del Nexus.

ARTEFACTOS

Representan **trabajo o valor** y están diseñados para **maximizar la transparencia**. El Nexus Integration Team trabaja con los Scrum Teams dentro de un Nexus para garantizar que se logre la transparencia en todos los artefactos y que el estado del Integrated Increment se entienda.

1. Product Backlog:

- **Descripción:** Lista única que contiene todo lo necesario para mejorar el producto en el Nexus.
- **Responsabilidad:** Product Owner.
- **Compromiso:** Objetivo del Producto, que describe el estado futuro del producto.

2. Nexus Sprint Backlog:

- **Descripción:** Compuesto por el Objetivo del Sprint de Nexus y elementos del Product Backlog de cada Scrum Team. Destaca dependencias y flujo de trabajo.
- **Responsabilidad:** Scrum Teams en conjunto.
- **Compromiso:** Objetivo de Sprint de Nexus, un objetivo único para el Nexus.

3. Integrated Increment:

- **Descripción:** Suma del trabajo integrado completado por el Nexus en relación con el Objetivo del Producto.
- **Responsabilidad:** Nexus Integration Team.
- **Compromiso:** Definición de Terminado, establece el estado del trabajo integrado cuando cumple con la calidad y medidas requeridas.

LESS

Este framework solo lo mencionaron, es más de lo mismo, voy a intentar sintetizar al máximo. Cuando hay un * es porque eso es distinto a Nexus.

LeSS (Large-Scale Scrum) es un marco de trabajo diseñado para escalar Scrum, aplicable a productos que involucran de **dos a ocho equipos**. Propone un **único Product Owner, un Scrum Master** que puede atender de uno a tres equipos, y la gestión de **un solo Product Backlog compartido** entre todos los equipos. En caso de trabajar con **más de ocho equipos**, se recomienda la implementación de **LeSS Huge**.

Principios de LeSS (literal es más de lo mismo, ya saben lo que significa: 🤪)

1. **Scrum a gran escala es Scrum;**
2. **Más con LeSS;**
3. **Pensamiento sistémico;**
4. **Pensamiento Lean;**
5. **Control empírico de procesos;**
6. **Transparencia;**
7. **Mejora continua hacia la perfección;**
8. **Centrado en el consumidor;**
9. **Enfoque de producto completo;**
10. **Teoría de colas.**

ROLES

1. **Scrum Master:**
 - Enseña Scrum y LeSS a la organización.
 - Entrena en la adopción de Scrum.
 - Guía a todos para contribuir mejor al producto.
 - Dedicado a tiempo completo al rol.
 - Sirve entre uno a tres equipos.
 - Se enfoca hacia los equipos, Product Owner, organización y prácticas de desarrollo.
2. **Product Owner:**
 - Visión clara y compartida del producto a lo largo de la organización.
 - Capaz de priorizar efectivamente las func., considerando las necesidades del cliente y del mercado.
 - Trabaja activamente con los equipos de desarrollo, participando en la planificación, revisión y retroalimentación para garantizar la construcción de las funcionalidades correctas.
 - Facilita una comunicación clara y transparente entre los equipos y stakeholders.
 - Maneja las expectativas de los stakeholders y equilibra sus necesidades con las metas del producto.
3. **Equipos:**
 - El mismo concepto que en Scrum de un solo equipo.
 - Objetivo: agregar ítems del Product Backlog al producto durante el Sprint.
 - Trabajo en estrecha colaboración con clientes/usuarios y Product Owner.
 - Coordinación e integración con otros equipos para producir un incremento único al final del Sprint.
 - Responsabilidad de manejar relaciones con otros equipos.
 - Autogestionados y multifuncionales.

EVENTOS

- 🕒 **Sprint Planning One:** *
 - Atendido por el **Product Owner** y **equipos o representantes**.
 - Seleccionan **tentativamente** los ítems para el Sprint.
 - **Identifican oportunidades** de colaboración y **aclaran preguntas finales**.
- 🕒 **Sprint Planning Two:** *
 - Equipos deciden **cómo abordar los ítems seleccionados**.
 - Involucra **diseño y creación del Sprint Backlog**.
- 🕒 **Daily Scrum:**
 - **Cada equipo tiene su propia Daily Scrum**.
 - Duración de 15 minutos.
 - Se responden **tres preguntas clave**: ¿Qué hice ayer?, ¿En qué voy a trabajar hoy? y ¿Qué queda por hacer?
- 🕒 **Sprint Review:**
 - **Revisión común** para todos los equipos.
 - **Punto de inspección y adaptación** al final del Sprint.
 - Enfoque en **todo el producto**.
 - Involucra a clientes y stakeholders para inspección y adaptación efectiva.
- 🕒 **Retrospective:** *
 - **Cada equipo realiza su propia Sprint Retrospective** al final del Sprint.

🕒 Overall Retrospective: *

- **Nueva reunión** en LeSS para discutir problemas entre equipos organizacionales y sistémicos.
- Enfoque en el trabajo conjunto de los equipos y aprendizaje compartido.

🕒 Product Backlog Refinement:

- Realizado dentro de cada Sprint para **refinar elementos** y prepararse para futuros Sprints.
- Actividades: **dividir elementos grandes, aclarar elementos y estimar tamaño, valor y riesgos.**

🕒 LeSS Sprint:

- Un **único Sprint** a nivel de producto para todos los equipos.
- Todos los equipos comienzan y terminan el Sprint simultáneamente.

ARTEFACTOS

▪ Incremento de Producto Potencialmente Entregable

- **Salida de cada Sprint** es un incremento potencialmente entregable.
- Integración del trabajo de **todos los equipos** al finalizar el Sprint.
- **"Potencialmente entregable" se refiere a la calidad** del software, no al valor o comerciabilidad.
- La capacidad de envío real depende del Definition of Done (DoD).
 - **DoD común** para todos los equipos.
 - **Cada equipo puede tener una DoD más detallada**, pero siempre expandiendo la común.
 - Objetivo: Mejorar la DoD para lograr el envío del producto en cada Sprint.

▪ Product Backlog

- Todos los equipos construyen **un solo producto** desde un único **Product Backlog**.
- **Ítems no preasignados** a equipos.
- Definición de producto amplia y centrada en el usuario final/cliente.
- El Product Backlog de LeSS es equivalente al de un entorno Scrum de un solo equipo.

▪ Sprint Backlog

- **Cada equipo tiene su propio Sprint Backlog.**
- Lista de trabajo para completar ítems del Product Backlog.
- **No hay diferencia entre un LeSS Sprint Backlog y un Sprint Backlog estándar.**

LESS HUGE

Igual a LeSS:

- Un Product Backlog para todos.
- Un Definition of Done (DoD).
- Un Definition of Ready (DoR).
- Un Product Manager.
- El Sprint es común para todos los equipos.
- Un Incremento potencialmente entregable.

Diferente a LeSS:

- Ahora hay un Área Product Manager.
- Hay un Área Product Owners.
- Hay un Área Product Backlogs.
- Hay un Área Product Backlogs.
- Hay un Área Product Visión.
- Hay más de 8 equipos.

KANBAN

KANBAN - **No es:**

- Un proceso de desarrollo de software.
- Una metodología de administración de proyectos.

KANBAN es un **enfoque para la gestión de formas de trabajo (PROCESO)** con el objetivo de **lograr mejora continua**. Se basa en el principio de "**empieza por donde estés**", introduciendo **mejoras graduales** en **procesos existentes** de desarrollo de software o metodologías de administración de proyectos. Esta estrategia **minimiza la resistencia al cambio**, fomentando la evolución gradual.

Inspirado en estudios de **Toyota** para mejorar técnicas de almacenamiento en supermercados. Su aplicación en el **desarrollo de software requirió una adaptación**, aprovechando principios de **Lean** como la definición de **valor desde la perspectiva del cliente**, la limitación del trabajo en proceso (**WIP**), la **identificación y eliminación de desperdicios**, la **eliminación de barreras en el flujo**, y la **promoción de una cultura de mejora continua**.

PRINCIPIOS FUNDACIONALES

GESTIÓN DE CAMBIOS

1. **Comenzar con lo que haces ahora:** Comprender y trabajar con los procesos actuales tal como se realizan en la actualidad y, respetar roles y responsabilidades actuales
2. **Acordar la mejora a través del cambio evolutivo:** Mejoras con cambios graduales y evolutivos en lugar de revolucionarios.
3. **Fomentar el liderazgo en todos los niveles:** Promover el liderazgo y la toma de decisiones en todos los niveles de la organización.

ENTREGA DE SERVICIOS

1. **Comprender y enfocarse en las necesidades del cliente:** Centrarse en comprender y satisfacer las necesidades y expectativas del cliente.
2. **Gestionar el trabajo y permitir la autoorganización:** Administrar el trabajo y permitir que los trabajadores se organicen en torno a él.

3. **Revisar periódicamente la red de servicios y políticas:** Evaluar regularmente la red de servicios y sus políticas para mejorar los resultados entregados.

PRINCIPIOS DIRECTORES

1. **Sostenibilidad:** relativo a encontrar un foco sostenible y un foco en la mejora.
2. **Orientación al servicio:** enfocado a conseguir rendimiento y satisfacción del cliente.
3. **Supervivencia:** relativa al mantenimiento de la competitividad y adaptabilidad.

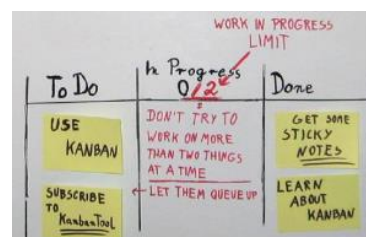
PRÁCTICAS GENERALES DE KANBAN

VISUALIZAR

Kanban se basa en la **visualización de trabajo a través de tableros visuales**. Estos tableros representan **el flujo de trabajo** y **muestran tareas** pendientes, en progreso y completadas, lo que proporciona una **visión clara del proceso**. Utiliza **tarjetas señalizadoras (kan-bans)** para dividir el trabajo en piezas y señalar cuándo se puede comenzar una tarea y en qué etapa se encuentra. El tablero KANBAN muestra el flujo de trabajo de izquierda a derecha y utiliza columnas (**Producción:** piezas sobre las que se está trabajando. **Acumulación:** piezas que están listas para pasar a la siguiente etapa) para representar etapas. Visualizar el trabajo en un tablero KANBAN promueve la transparencia, facilita la cooperación y agiliza la toma de decisiones.

LIMITAR EL TRABAJO EN PROGRESO (WIP)

Kanban enfatiza la importancia de **limitar el trabajo en progreso (WIP)** para **mantener un flujo de trabajo** equilibrado y constante. Esto **evita la sobrecarga** del equipo y garantiza un **rendimiento óptimo**. Limitar el WIP se basa en asignar límites explícitos a cuántos ítems pueden estar en progreso en cada etapa del flujo de trabajo. Esta limitación crea un **sistema de arrastre**, donde el trabajo se "arrastra" al sistema cuando se libera capacidad en lugar de "empujar" trabajo al siguiente paso. Esto **optimiza la utilización de recursos** y garantiza un flujo eficiente. Tener demasiado trabajo en progreso se considera un desperdicio de tiempo y recursos, y limitar el WIP es esencial para **mejorar la calidad, los tiempos de entrega y la tasa de entrega**.



GESTIONAR EL FLUJO.

El enfoque central de Kanban es **alcanzar un flujo de trabajo continuo y eficiente**. Esto implica **identificar y solucionar los cuellos de botella** y problemas que puedan afectar el flujo de trabajo, con el objetivo de **optimizarlo y garantizar entregas rápidas y constantes**. La meta es **completar el trabajo** de manera **fluida y predecible**, manteniendo un **ritmo sostenible**. El **seguimiento** y la **medición** del flujo de trabajo proporcionan información valiosa para gestionar las expectativas de los clientes, realizar predicciones y continuar mejorando.

HACER EXPLÍCITAS LAS POLÍTICAS.

Kanban enfatiza la necesidad de hacer **explícitas las políticas de trabajo y procedimientos**. Esto implica **definir** claramente los **criterios de finalización, prioridades y otras reglas** que afectan el proceso de trabajo. Estas políticas deben ser acordadas entre todas las partes involucradas, incluyendo clientes, interesados y miembros del equipo. Deben estar **visibles** y ser **sencillas, aplicables** en todo momento y **fácilmente modificables** para adaptarse a las necesidades del proceso. Además, las políticas de calidad o Definición de Hecho (DoD) deben ser **definidas, publicadas y promovidas** para buscar mejoras continuas. La **flexibilidad** para cambiar las políticas **es importante** si están teniendo efectos negativos o no son aplicables en ciertos momentos.

IMPLEMENTAR CICLOS DE RETROALIMENTACIÓN O FEEDBACK

Implementar ciclos de **retroalimentación o feedback** es esencial en Kanban. Esto implica establecer momentos regulares para la **colaboración, el aprendizaje y la mejora** basados en **datos recopilados**. La cadencia de estas reuniones **debe definirse considerando el contexto**, ya que reuniones demasiado frecuentes pueden generar cambios innecesarios, mientras que reuniones poco frecuentes pueden resultar en un bajo rendimiento a largo plazo.

MEJORAR DE MANERA COLABORATIVA, EVOLUCIONAR EXPERIMENTALMENTE

Kanban promueve la **mejora continua** a través de la **colaboración y el aprendizaje**. Los equipos deben buscar constantemente formas de **mejorar el proceso y adaptarlo** a medida que surgen nuevas ideas y desafíos. Esto implica **crear una cultura de mejora continua**, donde se identifica, visualiza y propone mejoras en el proceso de desarrollo de forma gradual a lo largo del tiempo, sin introducir cambios significativos de manera repentina.

CÓMO APLICAR KANBAN

Para implementar Kanban en un proyecto de manera efectiva, es fundamental seguir un proceso estructurado que abarque los siguientes pasos:

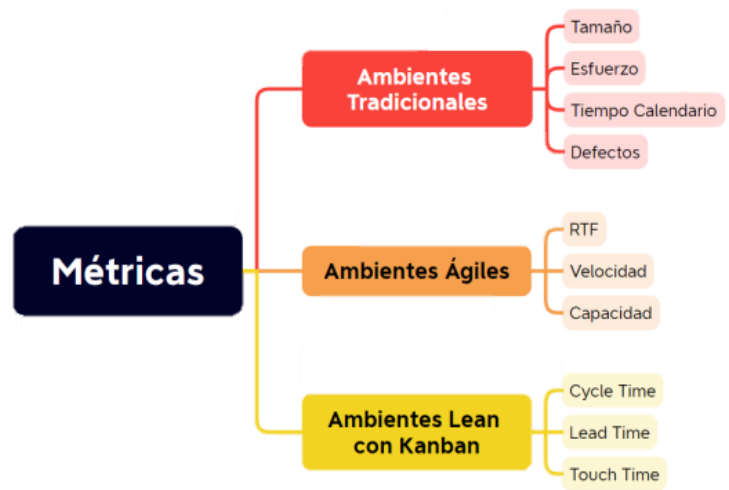
- 1. Comprensión del Contexto y Objetivos del Proyecto:** Antes de aplicar Kanban, es crucial **comprender a fondo el contexto del proyecto y los objetivos** que se desean lograr con su implementación. Esto implica identificar los problemas o desafíos específicos que se espera abordar.
- 2. Formación de un Equipo:** El siguiente paso es **formar un equipo** que incluya a todos los miembros involucrados en el proyecto. Esto asegura que todos tengan una comprensión completa de lo que implica Kanban y participen activamente en su implementación.
- 3. Modelado del Proceso:** El proceso se modela en un **tablero Kanban**, que puede ser físico o digital. En este tablero, se **representan las etapas** del flujo de trabajo a través de columnas. Estas columnas abarcan desde las tareas pendientes hasta las tareas completadas. La clave es que el tablero sea **claro y comprensible** para todos los miembros del equipo y permita una visualización transparente del flujo de trabajo.
- 4. Definición de la Unidad de Trabajo:** Se debe **definir** claramente la **unidad de trabajo** que se utilizará en el proyecto. Esto podría ser un requerimiento, una User Story, una tarea o cualquier otra unidad de trabajo relevante para el proyecto. También **es posible categorizar y priorizar** estas unidades de trabajo utilizando colores o **clases de servicio**. Son categorías utilizadas para definir cómo se gestionarán las unidades de trabajo. No siguen un enfoque de "primero en entrar, primero en salir" (FIFO). Ejemplos de algunas clases de servicio comunes:
 - Alta Prioridad (Expreso):** Las tareas con alta prioridad deben **resolverse de inmediato**. Se permite exceder los límites de trabajo en curso (WIP) para estas tareas y, si es necesario, se realizan entregas especiales para ponerlas en producción.
 - Fecha Fija:** Estas tareas tienen **fechas de entrega específicas**. No se pueden modificar los límites de WIP para esta clase de servicio, pero se permite que las tareas esperen en cola hasta que sea conveniente que ingresen. Si se retrasan y la fecha de entrega está en riesgo, pueden promoverse a la clase de servicio "Alta Prioridad".
 - Estándar:** Las tareas siguen un enfoque **FIFO**, es decir, se gestionan siguiendo el orden en que ingresaron al proceso. Deben adherirse a los límites de WIP definidos y se priorizan según el valor de negocio. Se entregan en entregas programadas.
 - Intangible:** Esta clase de servicio aborda tareas que **no generan entregables directos** para el cliente, como trabajos de deuda técnica, actualizaciones, refactorizaciones y mantenimientos.
- 5. Establecimiento de Políticas de Calidad:** Cada columna del tablero **debe tener políticas de calidad** claramente definidas. Estas políticas especifican los **criterios** que deben cumplirse **para mover una tarea** de una columna a la siguiente. También **se establecen límites de trabajo en curso (WIP)** para cada columna, lo que contribuye a mantener un flujo de trabajo equilibrado y eficiente y evita la sobrecarga del equipo.
- 6. Implementación Gradual de Kanban:** Kanban **se implementa gradualmente**. En lugar de realizar cambios radicales en los procesos de desarrollo, Kanban identifica el proceso actual, lo visualiza y propone mejoras de manera progresiva a lo largo del tiempo. Se comienza con algunas columnas y tarjetas que representan el flujo de trabajo actual y se agregan más columnas y **se refina el proceso a medida que el equipo se familiariza** con Kanban.
- 7. Capacitación del Equipo:** Durante todo el proceso, es importante **capacitar al equipo sobre los principios y prácticas de Kanban**. Se fomenta la colaboración y la comunicación abierta, animando a los miembros del equipo a compartir ideas y sugerencias para mejorar el flujo de trabajo y resolver problemas de manera conjunta.
- 8. Monitoreo Continuo del Flujo de Trabajo:** Es fundamental **monitorear continuamente el flujo de trabajo** en el tablero Kanban. Esto ayuda a **identificar cuellos de botella** o etapas donde se acumula demasiado trabajo. En esos casos, se considera mover recursos de una etapa a otra para equilibrar el flujo.
- 9. Establecimiento de Cadencia para Actividades Clave:** Se deben **programar y llevar a cabo actividades clave** del proyecto, como **releases, planificaciones y revisiones**, según un calendario definido. Esto contribuye a mantener un ritmo constante y permite una mejor planificación y seguimiento del proyecto.

Cadencia	Ejemplo de frecuencia	Propósito
Team Kanban Meeting	Diaria	Observar y seguir el estado y flujo del trabajo (no de los trabajadores). ¿Cómo podemos entregar los elementos de trabajo más rápido en el sistema? ¿Hay capacidad disponible? ¿Qué debemos tomar a continuación?
Team Retrospective	Quincenal o mensual	Reflexionar sobre cómo el equipo gestiona su trabajo y cómo pueden mejorar
Team Replenishment Meeting	Semanalmente o a demanda	Seleccionar los elementos de la lista de trabajo para realizar a continuación

MÉTRICAS

Las métricas **son números** que **representan atributos medibles** en procesos, productos o proyectos, permitiendo una **evaluación objetiva**. Su uso implica un **costo**, requiriendo **planificación y análisis**, lo que hace crucial un análisis de **costo-beneficio**. En el software, donde medir ciertas características es difícil, se recurre a conceptos asociados (Medidas indirectas).

En un proyecto de software, las métricas son esenciales para **monitorear** y **corregir desviaciones**. Sin embargo, nunca deben usarse para medir a las personas, ya que esto puede generar problemas y competencia no saludable. Las métricas **simplifican** el trabajo pero pueden carecer de contexto y generar sesgos. En cambio, se debe enfocar en la **colaboración, comunicación abierta y confianza**, **evaluando el rendimiento y mejorando continuamente** sin centrarse en métricas individuales, sino en el éxito colectivo y la mejora del proceso.



MEDICIONES EN EL SOFTWARE

Medidas Directas del Proceso: Incluyen aspectos fácilmente cuantificables como costo, esfuerzo, líneas de código, velocidad de ejecución, tamaño de memoria y defectos informados en un período establecido.

Medidas Indirectas: Engloban conceptos más abstractos como funcionalidad, calidad, complejidad, eficiencia, fiabilidad y facilidad de mantenimiento, siendo más desafiantes de medir.

Principio de Medición:

- **Formulación:** Derivación de medidas y métricas adecuadas para representar el software.
- **Recolección:** Acumulación de datos necesarios para derivar las métricas formuladas.
- **Análisis:** Cálculo de métricas y aplicación de herramientas matemáticas.
- **Interpretación:** Evaluación de las métricas para comprender la calidad de la representación.
- **Retroalimentación:** Recomendaciones derivadas de la interpretación de las métricas transmitidas al equipo de software.

MÉTRICAS ORIENTADAS AL TAMAÑO

Estas métricas surgen de la normalización de medidas de calidad y productividad considerando el tamaño del software producido.

Posibles Variables Por Medir:

- Líneas de código.
- Complejidad de Casos de Uso (CU).
- Complejidad de User Stories.
- Cantidad de Puntos de Función.
- Número de páginas web.
- Errores, defectos, personas.

Ejemplos de Métricas:

- Errores por KLDC (miles de líneas de código).
- Defectos por KLDC, Esfuerzo por KLDC.
- Páginas de documentación por KLDC.
- Errores por persona/mes.
- LDC por persona/mes.
- Costo por página de documentación.

MÉTRICAS ORIENTADAS A LA FUNCIÓN

Puntos de Función (PF): Derivados de medidas de funcionalidad, se definen mediante variables multiplicadas por su complejidad y sumadas.

Variables para Calcular Puntos de Función:

- Número de entradas de usuario.
- Número de salidas por usuario.
- Número de archivos.
- Número de interfaces externas.

Consideraciones sobre Puntos de Función: Controversiales, considerados independientes del lenguaje de programación, pero critican su base subjetiva.

MÉTRICAS DE CALIDAD

Corrección: Grado en que el software cumple su función. Medida común: "defectos por KLDC" donde defecto se define como la falta verificada de conformidad de requisitos.

Facilidad de Mantenimiento: Facilidad de corregir, adaptar o mejorar. Métrica indirecta: "tiempo medio de cambio".

Integridad: Capacidad del sistema para resistir ataques. Se mide considerando amenaza y seguridad.

Facilidad de Uso: Medida en términos de habilidad requerida, tiempo de eficiencia, aumento de productividad y valoración subjetiva del usuario.

INTEGRACIÓN DE MÉTRICAS

Establecimiento de una Línea Base: Se utiliza para comprender el estado actual del desarrollo de software, evaluando mejoras a través de medidas de productividad y calidad. La línea base se forma con datos exactos y consistentes recolectados de múltiples proyectos anteriores.

MÉTRICAS EN EL ENFOQUE TRADICIONAL

En el enfoque tradicional, la gestión de métricas **es más amplia y permite una mayor flexibilidad** en la definición y uso de métricas. Estas métricas se pueden dividir en **tres dominios**, que son **áreas específicas** a las que se aplican las métricas y que varían según el líder del proyecto. Esto contrasta con los enfoques Agile o Lean, que tienden a utilizar un conjunto más limitado y estandarizado de métricas. Los 3 conjuntos son:

1. **Métricas de Producto:** Estas métricas **se centran en el producto de software** en sí. Proporcionan **información sobre la calidad y el rendimiento** del producto. Por lo general, se utilizan para **evaluar defectos** (lo mismo que en proyecto) y problemas en el software, y no deben utilizarse para evaluar el rendimiento de las personas. Algunos ejemplos incluyen la cantidad de defectos o errores en el software y la cobertura de pruebas. Por ejemplo:
 - ♥ **Tamaño del Producto:** Esta métrica se refiere a la **magnitud y complejidad del producto** de software que se está construyendo. Inicialmente, se midió en líneas de código sin comentarios (jjsjsj), pero esta métrica es poco precisa ya que depende del lenguaje de programación y la experiencia del programador. Actualmente, se utiliza la estimación del tamaño del producto en función de los **requisitos**, que puede medirse mediante casos de uso, características, clases o puntos de historia según su complejidad.
2. **Métricas de Proyecto (Tácticas):** Estas métricas se aplican **durante la ejecución del proyecto** y están destinadas a **evaluar el estado y la salud del proyecto** en sí. Ayudan a **identificar desviaciones y problemas** específicos del proyecto y son útiles para el líder del proyecto y el equipo para tomar decisiones informadas. Algunas métricas comunes son:
 - ♥ **Esfuerzo:** Esta métrica está relacionada con los **recursos invertidos** en el proyecto. El tamaño del producto tiene una gran influencia en la cantidad de esfuerzo necesario para completar un proyecto.
 - ♥ **Tiempo (Calendario):** Esta métrica hace referencia al **tiempo empleado** para finalizar el proyecto, se mide en términos de calendario.
 - ♥ **Defectos:** Las métricas de defectos evalúan la **cantidad de problemas** o errores en el software. La cantidad de defectos en un proyecto de software es una función del esfuerzo invertido y el tamaño del proyecto. Algunas métricas comunes incluyen la cantidad de defectos por punto de función. Por lo general, proyectos más grandes tienden a generar más defectos por la línea de código.
 - **Cobertura:** La cobertura se relaciona con la cantidad del **código** del software que ha sido **probado**. Proporciona confianza para liberar el producto de software. Las métricas de cobertura evalúan qué porcentaje de clases o líneas de código se han sometido a pruebas.
 - **Defectos por Severidad:** Esta métrica **clasifica los defectos y los riesgos según su severidad**. Puede expresarse en cantidad de defectos por nivel de severidad o en porcentaje de defectos de un nivel de severidad específico.
 - **Densidad de Defectos:** La densidad de defectos se mide en términos de la **cantidad de errores por líneas de código**. Para su análisis, es importante considerar el tamaño del proyecto. Esta métrica indica cuántos defectos se encuentran en una unidad de prueba específica.
3. **Métricas de Proceso (Organizacionales o Estratégicas):** Estas métricas **ofrecen una visión más amplia** del funcionamiento **de la organización** y cómo se gestionan los proyectos en el conjunto de la empresa. Se enfocan en **medir aspectos estratégicos y organizacionales**, como el rendimiento promedio de los proyectos y la eficiencia de las operaciones. Pueden ayudar a identificar áreas de mejora en la organización y permiten realizar análisis a nivel de toda la empresa.

Es importante seleccionar métricas que sean **relevantes y útiles** para el contexto actual. Las métricas **no deben consumir demasiado tiempo y esfuerzo** en su recopilación, y deben **proporcionar información práctica**. Es fundamental encontrar un equilibrio entre recopilar datos significativos y no sobrecargar al equipo con mediciones innecesarias. El enfoque selectivo, centrándose en unas **pocas métricas clave**, puede ser más eficaz que recopilar un exceso de datos. En última instancia, las métricas deben ser una **herramienta para la mejora** continua y la toma de decisiones informadas en el desarrollo de software.

MÉTRICAS EN AMBIENTES ÁGILES

En la filosofía Agile, se adopta un enfoque **minimalista en la medición** y **se prioriza lo esencial**. Se eligen métricas que agreguen valor para el cliente, y no se consideran las métricas como una actividad en sí misma, sino como un **resultado**. Esto se basa en **dos principios ágiles clave**:

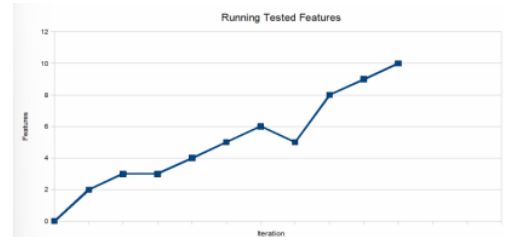
1. **El software que funciona es la principal medida del progreso:** Este principio destaca que el progreso de un proyecto se evalúa mejor a través del software real y en funcionamiento que proporciona valor. En lugar de enfocarse únicamente en métricas teóricas, se valora el software tangible y funcional como el indicador principal de progreso.
2. **La mayor prioridad es satisfacer al cliente a través de entregas tempranas y continuas de software valioso y funcional:** Este principio enfatiza la importancia de entregar rápidamente software de alta calidad que sea valioso para el cliente. En lugar de esperar a tener un producto completo, se realizan entregas frecuentes y tempranas de software funcional. Esto permite obtener retroalimentación temprana del cliente y ajustar el producto en consecuencia.

En base a lo especificado se definen las métricas de:

RUNNING TESTED FEATURES (RTF)

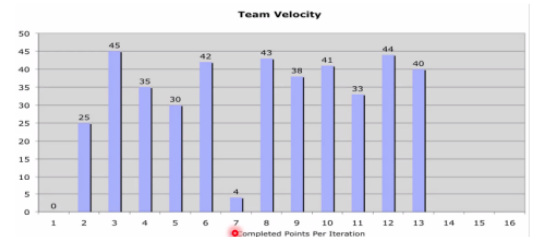
Esta métrica se centra en **medir la funcionalidad entregada** en un producto de software. Se desglosa en tres partes: **"Running"** se refiere a la funcionalidad que está realmente en funcionamiento, **"Tested"** indica que la funcionalidad ha pasado con éxito las pruebas de aceptación establecidas, y **"Features"** se relaciona con las características específicas requeridas por el cliente. RTF mide la **cantidad de estas características que están terminadas y funcionando**. Sin embargo, una limitación de esta métrica es que **no considera la complejidad** de las características.

Esta métrica es útil para **evaluar la estabilidad del software** y **observar tendencias** a lo largo del tiempo. Por ejemplo, si la curva de RTF muestra una tendencia constante o negativa, podría indicar un problema, como la obsolescencia de una funcionalidad en el sistema.



VELOCIDAD

La velocidad es una **métrica clave** en Agile. Se trata de una **medida empírica** de la **capacidad del equipo** para completar trabajo en una iteración o sprint. No se basa en estimaciones, sino en observaciones reales del equipo. En un sprint, la velocidad se calcula tomando en cuenta la **cantidad de Story Points** que el equipo terminó y que el PO aprobó en la Review. Estos puntos de historia son una medida del trabajo que el equipo puede abordar en un período de tiempo. La velocidad es **vital para la planificación** y estimación de proyectos Agile, ya que **permite predecir cuánto trabajo** se puede realizar en futuras iteraciones. Es normal que en los primeros sprint la velocidad sea 0 o muy baja, ya que el equipo se está conociendo. A medida que los sprints avanzan, se espera que la velocidad se estabilice, lo que indica que el equipo ha adquirido un ritmo constante y predecible en la finalización de trabajo. Esta métrica suele representarse con un **gráfico de barras** para medir la estabilidad del equipo.



CAPACIDAD

La capacidad es una **estimación teórica** de **cuánto trabajo** puede completarse en un período de tiempo. Se calcula al principio de un sprint y se utiliza para predecir cuántos Story Points (o cualquier otra unidad de medida) el equipo podrá abordar en el próximo sprint. La capacidad se estima considerando la **cantidad de tiempo ideal disponible** para el equipo. Puede expresarse en horas ideales o Story Points. La capacidad es una métrica **útil en la planificación** de proyectos, ya que proporciona una guía sobre cuánto trabajo es realista asumir para la próxima iteración.

$$\begin{aligned} \text{Horas de Trabajo Disponibles por día (WH) X Días} \\ \text{Disponibles Iteración (DA)} &= \text{Capacidad} \\ \text{WH x DA} &= \text{Capacidad} \end{aligned}$$

MÉTRICAS EN KANBAN

1. **Cycle Time (Tiempo de Ciclo):** Registra el **tiempo entre el inicio y el final del proceso** para un ítem de trabajo. Se mide en días de trabajo o esfuerzo y proporciona una visión mecánica de la capacidad del proceso. También se conoce como el *"Ritmo de Terminación."*
2. **Lead Time o Elapsed Time (Tiempo de Entrega):** Registra el **tiempo entre la solicitud de un ítem de trabajo y su entrega final**. Se mide en días de trabajo y se conoce como el *"Ritmo de Entrega."*
3. **Touch Time (Tiempo de Tocado):** Representa el **tiempo en el que un ítem de trabajo fue activamente procesado** por el equipo, en contraposición a estar en cola o en estado de espera. El objetivo es que el Touch Time se asemeje lo más posible al Cycle Time.
4. **Eficiencia del ciclo:** La Eficiencia del Ciclo de Proceso se mide:

$$\begin{aligned} \% \text{ Eficiencia Ciclo Proceso} &= \frac{\text{Touch Time}}{\text{Elapsed Time}} \\ \text{Touch Time} &\leq \text{Cycle Time} \leq \text{Lead Time} \end{aligned}$$

MÉTRICAS ORIENTADAS A SERVICIO

Las métricas orientadas a servicio se centran en la **calidad del servicio** proporcionado a los clientes y pueden incluir:

1. **Expectativa de Nivel de Servicio:** Esto se refiere a las expectativas que los clientes tienen con respecto a la calidad del servicio que van a recibir.
2. **Capacidad del Nivel de Servicio:** Se refiere a la capacidad del sistema o equipo para entregar un nivel de servicio específico. Esto implica evaluar si el sistema tiene la capacidad técnica y los recursos necesarios.
3. **Acuerdo de Nivel de Servicio (SLA):** Estos son acuerdos formales entre el proveedor de servicios y el cliente que establecen los niveles de servicio esperados. Las métricas se utilizan para medir si se están cumpliendo los términos del SLA.
4. **Umbral de Adecuación del Servicio:** Este es el nivel mínimo aceptable de servicio para el cliente. Cuando el servicio cae por debajo de este umbral, se considera inaceptable.

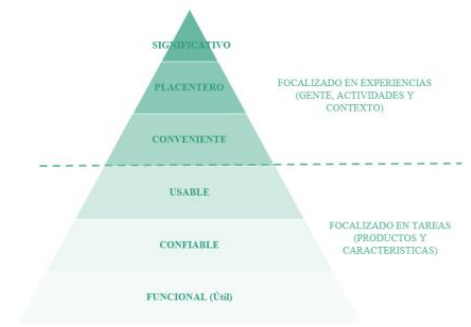
GESTIÓN DE PRODUCTOS DE SOFTWARE

La **gestión de productos** implica tener una **visión clara** de por qué creamos productos de software, y esto va más allá de la **satisfacción del cliente**, la **generación de ingresos** o la **popularidad**. También puede estar relacionado con la ambición de **cambiar el mundo** o **mejorar nuestra diaria**. Sin embargo, a menudo invertimos tiempo en construir características que no se utilizan o se utilizan raramente, lo que implica un esfuerzo desperdiciado en algo que no aporta valor. Por lo tanto, es crucial eliminar el desperdicio y centrarse en lo que realmente es útil.

EVOLUCIÓN DE LOS PRODUCTOS DE SOFTWARE

Podemos organizar la **evolución de los productos de software** en una especie de **pirámide** con respecto a sus características. En la **base** de la pirámide se encuentran las características fundamentales que deben existir para que el software **funcione correctamente**, sea **confiable** y tenga cierta **usabilidad básica**. Estos **aspectos son esenciales** para lograr el propósito esperado y son lo que todos esperamos de la funcionalidad del software.

En la **cima** de la pirámide se ubican otras características relacionadas con la **visión del producto** que son más difíciles de alcanzar. Estas incluyen la **conveniencia**, la **satisfacción** del usuario y la **significatividad** del producto. Sin embargo, muchas veces los productos se quedan en la base de la pirámide y no logran abarcar estos aspectos más elevados.



PLANIFICACIÓN DEL PRODUCTO ÁGIL

En las metodologías ágiles, la planificación del **producto se basa en el Product Backlog**, que representa la visión del producto con todas sus características. Esta visión **se desglosa en releases**, entregas frecuentes al cliente construidas a través de iteraciones sucesivas.

VALOR VS DESPERDICIO

En el contexto ágil, la **productividad** de un startup **se mide** no por cuánto se construye diariamente, sino **por construir lo correcto** cada día. El "correcto" se define como lo que **agrega valor**. Construir sin agregar valor se considera un desperdicio de esfuerzo.

MÍNIMO PRODUCTO VIABLE (MVP)

El MVP, proveniente de **Lean Startup**, destaca la **importancia del aprendizaje** en el desarrollo de nuevos productos. Se define como la versión que permite recopilar la máxima cantidad de aprendizaje validado sobre los clientes con el **menor esfuerzo**. La premisa clave es observar el comportamiento real de los usuarios con el producto.

Características Principales

1. **Valor Suficiente:** Debe tener valor para que las personas lo usen o lo compren.
2. **Beneficios Futuros:** Demuestra beneficios a futuro para retener usuarios.
3. **Retroalimentación:** Proporciona un ciclo de retroalimentación para guiar el desarrollo futuro.

Errores Comunes

1. **Confusión con MMF o MMP:** Enfoque en el aprendizaje, no en ganar (más detalle de esto más adelante).
2. **Énfasis Incorrecto:** No descuidar la calidad; debe ser viable para obtener retroalimentación.
3. **Producto Final:** No considerarlo definitivo; cambios basados en retroalimentación son esenciales.

Validación de Hipótesis

1. **Hipótesis del Valor:** Ver si entrega valor a los clientes.
2. **Hipótesis de Crecimiento:** Explorar cómo nuevos clientes descubrirán el producto.

Preparación del MVP

1. **Encontrar Nicho de Mercado.**
2. **Roadmap:** Definir características del producto a alto nivel para inversores.
3. **Investigar Competencia.**
4. **Pre-Venta del MVP.**
5. **Testear Suposiciones.**

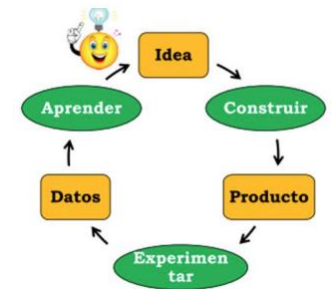
6. Resolver el Problema Correcto.
7. Enfocarse en Funcionalidades Principales.

LEAN UX Y CONSTRUCCIÓN DEL MVP

La filosofía Lean UX se basa en un **proceso iterativo**, Build-Measure-Learn, que **prioriza la construcción rápida, medición y aprendizaje**. Enfoca la **validación temprana, permitiendo ajustes** cuando el usuario no aprueba el producto inicial. El éxito se define por entregar algo que el cliente realmente usará, alineando esfuerzos continuamente con sus necesidades.

La fase de construir el MVP busca ingresar **rápidamente un producto mínimo viable**, variando en complejidad. El objetivo es **validar hipótesis** y **obtener retroalimentación** en el ciclo de aprendizaje. La simplificación, evitar excesos y reconocer el trabajo necesario son clave para minimizar desperdicio.

En la construcción del MVP, la **audacia desde el principio** es crucial. Postergar experimentos con el MVP puede resultar en pérdida de trabajo y datos esenciales. La fase de **experimentación**, especialmente con los primeros usuarios, verifica hipótesis y prueba elementos riesgosos. Abordar supuestos arriesgados desde el inicio y probarlos con el MVP es esencial para el éxito en el desarrollo de nuevos productos o startups.



DINOSAURIO LEAN/AGILE

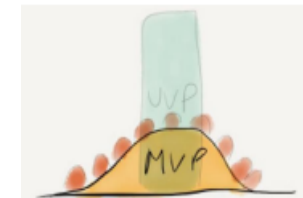
El enfoque aquí se centra en desarrollar un **producto mínimo** o una característica mínima para **probar si cumple** con las **expectativas del usuario** o las propias al desarrollar el producto. Se parte de una **hipótesis** sobre el valor que el producto puede ofrecer y se busca validarla con un esfuerzo mínimo, **evitando gastar recursos** en algo que no aportará valor al producto.



1. La técnica utilizada es la **UVP (User Value Proposition o Propuesta de Valor para el Usuario)**, que se basa en **comprender las necesidades y problemas de los usuarios** para crear soluciones efectivas que las satisfagan. En lugar de desarrollar un producto completo con muchas funcionalidades desde el principio, se busca **minimizar el esfuerzo** para construir un producto que los usuarios realmente demanden, si la hipótesis inicial resulta ser válida. Esto **ayuda a evitar el desperdicio** de recursos en un producto que no cumple con las expectativas del mercado.



2. El **MVP (Minimum Viable Product)** es un concepto de **Lean Startup** que se centra en **desarrollar un producto con la cantidad mínima de funcionalidades** necesarias para que los usuarios potenciales puedan **evaluarlo y proporcionar retroalimentación**. El objetivo principal del MVP es **validar la hipótesis** sobre el producto y aprender de la participación de los usuarios para determinar si se deben hacer ajustes en el producto. No se trata de vender el MVP, sino de comprobar si el producto cumple con las expectativas de los usuarios.



3. Pueden ocurrir **dos situaciones**:

1. Los usuarios **encuentran el MVP satisfactorio** y expresan que lo necesitan tal como está. En este caso, la hipótesis original se valida, y el producto puede continuar desarrollándose en la misma dirección.
2. **Cada usuario** que prueba el MVP **proporciona retroalimentación** y **solicita funcionalidades** diferentes. Esto indica que el **mercado** y las **necesidades** de los usuarios **no están claros**, y la visión del producto **necesita ajustes** para encontrar un enfoque más adecuado.

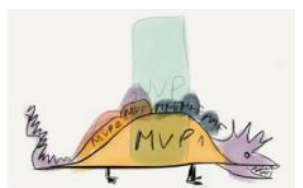
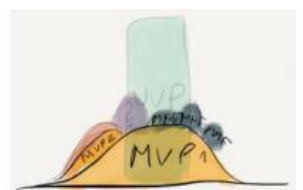
En el desarrollo de un producto, la **hipótesis inicial puede evolucionar o modificarse** en función de las demandas y necesidades del mercado. En este proceso, el **MVP se ajusta y se mueve** hacia el enfoque que atrae más a los **clientes**. Aunque la hipótesis inicial puede ser acertada en su enfoque general, algunas de sus características pueden requerir ajustes o cambios para

satisfacer mejor las expectativas de los usuarios y adaptarse al mercado.

4. Cuando la hipótesis inicial necesita **modificaciones o ajustes**, es necesario **redefinir el MVP** en función del conocimiento adquirido. Esto implica la creación de un **nuevo MVP**, que podríamos llamar **MVP2**. El proceso de **retroalimentación se repite** con este nuevo MVP, y este ciclo puede ocurrir varias veces hasta encontrar un feedback exitoso y llegar a la versión final del producto que queremos construir.

5. Supongamos que MVP2 es exitoso y está viendo una atracción real de los primeros usuarios. Si desea aumentar el crecimiento y busca una penetración más profunda de sus primeros usuarios además de **atraer nuevos clientes**, algunos de ellos más allá de la multitud de usuarios pioneros. En función del feedback recopilado y la investigación de su gestión de productos, tiene un par de **áreas que potencialmente pueden traer este crecimiento**. Algunos de ellos por cierto amplían su propuesta de valor única y algunos hacen que su producto actual sea más robusto.





6. En el caso de áreas con una fuerte **indicación de valor**, puede directamente definir un **MMF (Características mínimas comercializables)**. Para encontrar la pieza mínima que pueda empezar a traer crecimiento. **El objetivo del MMF es aportar valor**. Supone una alta certeza de que existe valor en esta área y que sabemos cuál debe ser el producto para proporcionar este valor. La razón para **dividir una característica grande en MMF** más pequeños es principalmente el tiempo de comercialización (**Time to market**) y la **capacidad de aportar valor en muchas áreas**. Una indicación de que está trabajando en MMF es que cuando al liberarse uno se siente cómodo trabajando en el próximo MMF en esa área. El MMF se refiere a una **"Feature" o funcionalidad extremadamente pequeña que representa lo mínimo que se puede lanzar al mercado** para evaluar su viabilidad.

7. Si vamos a esperar para ver si el primer MMF no funciona... entonces está de vuelta en la tierra de la hipótesis. Ahora tu hipótesis se centra en una característica en lugar del producto. Tienes un área con **alto potencial** pero también **alta incertidumbre**.

8. La forma de afrontarlo es crear una función "pionera": **MVF (Característica Mínima Viable)**. La **característica mínima** que aún **puede ser viable** para uso real y aprendizaje de los usuarios reales. **Si el MVF resulta exitoso (hit gold), puede desarrollar más MMF en esa área** para tomar ventaja (si eso tiene sentido). Si no es así, puede cambiar a otro enfoque hacia esa área de características, o en algún momento buscar una ruta de crecimiento alternativa. Esencialmente, **el MVF es una versión mini del MVP**.

9. **El producto se cultiva en mercados inciertos al intentar varios MVP**. Cuando se logra ajustar el producto en el mercado de productos se combinan MMF y MVF según el nivel de incertidumbre del negocio / requisitos en las áreas en las que se está enfocando. Si bien los MVP / MMF / MVF son **atómicos desde una perspectiva empresarial** (no puede implementar y aprender de algo más pequeño) **pueden ser bastante grandes** desde la perspectiva de la implementación. El **dinosaurio carpaccio** se obtiene cortando cada una de esas piezas en pequeñas porciones destinadas a **reducir el riesgo de ejecución / tecnología**. Esas porciones más pequeñas **pueden tener un valor comercial tangible** o no. Para otros puede ser importante proporcionar una **temprana retroalimentación** de decisiones a lo largo del camino.

PRODUCTOS MÍNIMOS

1. Producto Mínimo Viable (MVP):

- Definición: Versión de un nuevo producto con el menor esfuerzo, utilizado para obtener aprendizaje validado.
- Enfoque: Dirigido a un subconjunto específico de clientes potenciales.
- Concepto: Más cercano a prototipos que a una versión real.
- Premisa: Énfasis en el impacto del aprendizaje en el desarrollo de nuevos productos.

2. Característica Mínima Viable (MVF):

- Definición: Versión mini del MVP, pieza pequeña de funcionalidad.
- Valor: Significativo para la organización y los usuarios.
- Implementación: Construcción rápida con recursos mínimos.
- Relación: Parte de un MMR o MMP.

3. Característica Mínima del Release (MRF):

- Definición: Características mínimas que debe tener un producto para salir al mercado.
- Publicación: Versión del producto que el cliente puede usar.

4. Característica Mínima Comercializable (MMF):

- Definición: Pieza más pequeña de funcionalidad liberada, versión mini del MVP.
- Valor: Importante tanto para la organización como para los usuarios.
- Implementación: Construida e implementada rápidamente con recursos mínimos.

RELACIÓN ENTRE MVP, MMF, MMP, MMR

1. MVP:

- Descripción: Versión de nuevo producto para aprendizaje validado con bajo esfuerzo.
- Foco: Dirigido a subconjunto específico de clientes.
- Método: Énfasis en observar comportamiento real del cliente.

2. MMF:

- Definición: Pieza más pequeña de funcionalidad, versión mini del MVP.
- Valor: Significativo para la organización y los usuarios.
- Implementación: Rápida, parte de un MMR o MMP.

3. MMP:

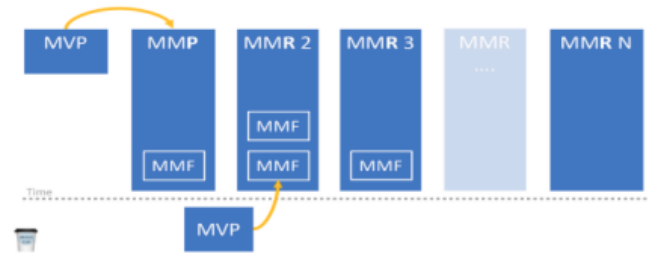
- Descripción: Primer release de un MMR para early adopters.
- Foco: Características clave para satisfacer a un grupo clave.

4. MMR:

- Definición: Release del producto con conjunto mínimo de características.
- Valor: Ofrece nuevo valor a usuarios, MMP = MMR1.

Los **errores más comunes** relacionados con MVP, MMF y MMP son:

1. Confundir un MVP, que se enfoca en aprender y validar hipótesis, con una Característica Comercial Mínima (MMF) o un Producto Comercializable Mínimo (MMP), que buscan obtener un retorno de inversión.
2. Correr el riesgo de entregar algo sin considerar si realmente satisface las necesidades del cliente.
3. Enfocarse excesivamente en hacer el MVP lo más mínimo posible, descuidando su viabilidad. Un MVP debe ser tanto mínimo como viable para proporcionar una evaluación precisa.
4. Entregar un producto como MVP y luego no realizar cambios, incluso si se reciben comentarios negativos. Es importante estar dispuesto a adaptar el producto en función de los comentarios y aprendizajes obtenidos.

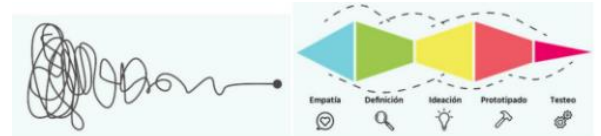


DESIGN THINKING

Es una metodología centrada en usuarios para crear **servicios digitales**, enfocada en **cumplir expectativas** mediante **mejora continua y comprensión de necesidades** reales.

Proceso:

1. **Empatizar:** Comprender necesidades del cliente con razonamiento intuitivo.
2. **Definir:** Determinar lo valioso para el cliente, guiando el trabajo.
3. **Idear:** Generar muchas soluciones posibles en equipos multifuncionales.
4. **Prototipar:** Materializar ideas en prototipos para visualizar soluciones.
5. **Probar:** Etapa crucial para validar prototipos con usuarios, obtener retroalimentación y permitir iteraciones.



Utiliza pensamiento divergente (amplio espectro de soluciones) y convergente (seleccionar mejor idea) en un proceso iterativo, volviendo a pasos anteriores según necesidad.

UNIDAD 3 | GESTIÓN DEL SOFTWARE COMO PRODUCTO

CONCEPTOS INTRODUCTORIOS DE LA GESTIÓN DE CONFIGURACIÓN

Los **sistemas de software siempre cambian** durante su desarrollo y uso. Conforme se hacen cambios al software, se crean nuevas **versiones** del sistema. Los **sistemas** pueden considerarse como un **conjunto de versiones** donde cada una de ellas debe **mantenerse y gestionarse**. Esto es necesario para no perder la **trazabilidad** de los cambios que se incorporan a cada versión.

GESTIÓN DE CONFIGURACIÓN

La **gestión de configuración de software (SCM)** es fundamental para **manejar todos los aspectos** relacionados con el software, incluyendo **herramientas, procesos y entorno**. El software se compone de **información estructurada** con **propiedades lógicas y funcionales**, y se crea y mantiene en diversas formas y representaciones para ser procesado por computadoras.

Los sistemas de software **experimentan cambios** durante su desarrollo y uso, lo que da lugar a la creación de nuevas **versiones** del sistema. Estas versiones **deben gestionarse y mantenerse** para mantener la **trazabilidad** de los cambios realizados en cada una de ellas. Los cambios en el software pueden originarse en **modificaciones en el negocio, nuevos requisitos, alteraciones** en productos asociados, **reorganización de prioridades, cambios presupuestarios, corrección de defectos y oportunidades de mejora**.

La gestión de configuración de software asegura la **integridad del producto** de software a lo largo de su ciclo de vida. Es una **actividad de soporte** que abarca todo el proyecto y que implica **dirección y monitoreo** administrativo y técnico. Su objetivo es **garantizar que el producto se mantenga íntegro y de alta calidad**.

ACTIVIDADES

La gestión de configuración de software (SCM) tiene como **propósito establecer y mantener la integridad** de los productos de software a lo largo de su ciclo de vida. Para lograr esto, se llevan a cabo diversas **actividades** que incluyen:

1. Identificar características técnicas y funcionales de ítems de configuración.
2. Documentar estas características técnicas y funcionales de los ítems de configuración.
3. Controlar los cambios que afectan a las características identificadas y documentadas.
4. Registrar y reportar los cambios realizados.
5. Verificar la correspondencia de los cambios con los requerimientos, manteniendo la trazabilidad.

APLICACIONES

La SCM encuentra aplicaciones en diversas disciplinas, como el **control de calidad de proceso**, el **control de calidad de producto** y la **prueba de software**. Su objetivo es resolver problemas de diferente índole mediante el establecimiento y el mantenimiento de la integridad del producto de software en todas sus etapas.

PROBLEMAS

Algunos de los **problemas** que la SCM ayuda a abordar incluyen:

- ✖ **pérdida de componentes**
- ✖ **pérdida de cambios** (cuando no se mantiene la última versión del componente)
- ✖ **regresión de fallas, doble mantenimiento** (realizar cambios en varios lugares)
- ✖ **superposición de cambios** (varios cambios realizados simultáneamente sin validación).

INTEGRIDAD

La integridad asegura la calidad y la confiabilidad del producto de software. La calidad es subjetiva y difícil de medir debido a la naturaleza intangible del software. La integridad del producto implica hacer explícitas las expectativas del cliente sobre el software.

La integridad de un producto de software se mantiene cuando cumple con los siguientes aspectos:

1. **Satisface las necesidades del usuario** y realiza las funciones esperadas.
2. **Permite la trazabilidad durante todo su ciclo de vida**, lo que implica mantener vínculos entre los ítems de configuración para analizar el impacto de los cambios.
3. **Cumple con criterios de rendimiento y requisitos no funcionales**.

4. **Satisface las expectativas de costos**, incluyendo la rentabilidad del desarrollo del producto y la gestión de configuración que es responsabilidad de todo el equipo, con un enfoque adicional del Gestor de Configuración en tareas específicas.

CONCEPTOS GENERALES

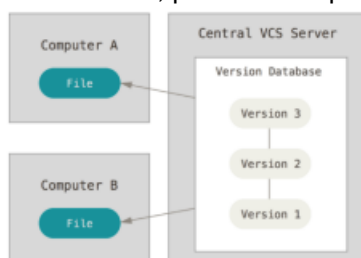
ÍTEM DE CONFIGURACIÓN

Los ítems de configuración son **elementos esenciales** en un **proyecto** o **producto** de software que pueden tener **diferentes tipos**, como código fuente, documentos, datos de prueba, etc. Estos ítems **se almacenan** en un **repositorio** y **pueden cambiar** con el tiempo. Su objetivo es **mantener un registro** de su **estado** y **evolución** a lo largo del ciclo de vida del proyecto o producto. Ejemplos de ítems de configuración incluyen prototipos de interfaz, manuales de usuario, código fuente y más. Su duración puede variar según el proyecto o sprint. Son fundamentales para la gestión y control de la configuración del software.

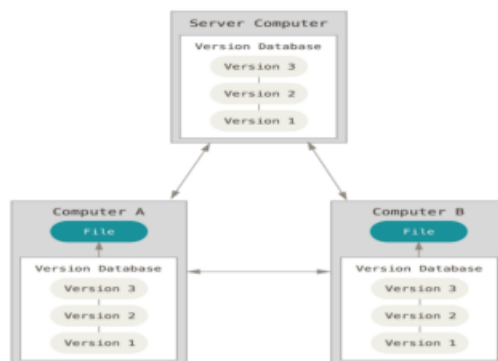
REPOSITORIO

Un repositorio es un **lugar donde se almacenan ítems de configuración** junto con su **historial**, **atributos** y **relaciones**. Puede ser una base de datos o varias bases de datos organizadas con una estructura que asegura orden e integridad. Hay dos tipos principales de repositorios:

1. **Repositorio Centralizado:** Un servidor contiene todos los archivos con sus versiones. Proporciona un mayor control a los administradores, pero si el servidor falla, puede haber problemas.



2. **Repositorio Descentralizado:** Cada cliente tiene una copia idéntica del repositorio completo. Resuelve problemas de servidores centralizados y permite ciertos flujos de trabajo no disponibles en el modelo centralizado. Sin embargo, puede ser más difícil de controlar.



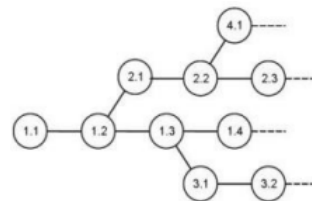
VERSIÓN

Una versión es una **instancia de un ítem de configuración** que se diferencia de otras instancias de ese **mismo ítem**. Cada versión **se identifica de manera única** y representa un punto específico en el tiempo o contexto de ese ítem de configuración. La gestión de configuración permite a los usuarios seleccionar las versiones adecuadas para especificar configuraciones alternativas del sistema de software. **Cada versión de software comprende una colección de elementos de configuración**, como código fuente, documentos y datos. El **control de versiones** se enfoca en la evolución de un ítem de configuración particular y puede representarse gráficamente en forma de grafo para visualizar los cambios a lo largo del tiempo.



VARIANTE

Una variante es una **versión separada de un ítem de configuración** o de la configuración en sí misma. Estas variantes representan **configuraciones alternativas** que permiten que un producto de software pueda **adaptarse** a diferentes contextos, como plataformas de hardware y sistemas operativos específicos, así como funciones opcionales que pueden activarse o desactivarse según las necesidades.



CONFIGURACIÓN DE SOFTWARE

Una configuración de software es un **conjunto que incluye todos los ítems de configuración con sus versiones específicas en un momento particular**. Esta configuración representa una instantánea que abarca todos los componentes fuentes, documentos y la información estructural que definen una **versión específica del producto en un punto en el tiempo**.

LINEA BASE

Una línea base en la gestión de configuración de software es un **conjunto de ítems de configuración que han sido construidos y revisados formalmente**. Estos ítems se consideran como **referencia** para demostrar que han alcanzado un cierto **nivel de madurez** y sirven como **base para desarrollos futuros**. Las líneas base deben tener una referencia única, generalmente a través de **"tags"**.

El propósito principal de las líneas base es proporcionar un **punto de referencia sólido** en el ciclo de vida del software. Además, se utilizan para realizar **"Rollback"** en caso de necesidad, es decir, volver a una versión anterior del software. También son útiles para **determinar qué versión se va a poner en producción**. Permiten rastrear la evolución del software a lo largo del tiempo y reproducir el entorno de desarrollo en momentos específicos del proyecto. Existen dos tipos de líneas base: **las de especificación**, que incluyen requerimientos y diseño, y **las operacionales**, que contienen una versión ejecutable del producto que ha pasado por un control de calidad previo.

La primera línea base operacional corresponde a la primera **"Release"**, que es una entrega del sistema que se libera para su uso por parte de los clientes u otros usuarios de la organización.



RAMA (BRANCH)

Las ramas en la gestión de configuración de software son **conjuntos de ítems de configuración con sus respectivas versiones** que permiten **dividir el desarrollo de un software en diferentes direcciones**. Esto puede hacerse por varias razones, como **experimentación**, **corrección** de errores en el desarrollo o para **adaptar** el software a diferentes plataformas.

La **integración de ramas o "merge"** se produce cuando se combinan dos ramas para fusionar la configuración de los ítems que las componen, incluyendo sus respectivas versiones. Es una buena práctica mantener la rama principal como la versión estable y fusionar los cambios hacia ella. En caso de que no se integren en la rama principal, se recomienda descartar las ramas para mantener la organización y evitar confusiones.

PLANIFICACIÓN DE LA SCM

La planificación de la gestión de configuración de software se realiza **al inicio del proyecto** y abarca diversos **aspectos esenciales** para su implementación eficiente. Esto incluye la **definición de reglas de nombrado de ítems de configuración**, la **elección de herramientas** para el Control de Configuración de Software (SCM), la **asignación de roles** en el **Comité de Control de Cambios**, el **establecimiento de procedimientos formales para cambios**, **procesos de auditoría**, la **estructura del repositorio**, consideraciones sobre **SCM para software externo** (si es aplicable), **métodos de control de cambios**, **registros obligatorios** y **tipos de documentos**.

Consideraciones clave:

- Debe llevarse a cabo **tempranamente** en el proyecto.
- Es crucial **definir** los **documentos** que serán gestionados.
- **Todos los productos** generados en el proceso deben estar **sujetos a administración** mediante la SCM.

ACTIVIDADES RELACIONADAS A LA SCM

Todas las actividades se mencionan se realizan en **ambas metodologías** (tradicional y ágil), menos las auditorías, ya que el concepto de ser auditado y controlado por alguien externo no es compatible con una metodología ágil pura.

IDENTIFICACIÓN DE ÍTEMES DE CONFIGURACIÓN

La identificación de ítems de configuración es un proceso que implica **asignar una identificación única a cada ítem** y establecer **reglas de nomenclatura** y **versionado** en el equipo. También incluye **definir la estructura del repositorio** y dónde se ubicarán los ítems de configuración dentro de esa estructura.

Este proceso es esencial porque **proporciona una conexión** a lo largo de todas las etapas del **ciclo de vida del software**, lo que permite a los desarrolladores mantener el control y la integridad del producto, al tiempo que permite a los clientes evaluar esa integridad.

Es importante considerar **la duración de la integridad** de los ítems al identificarlos, ya que **varía** según el tiempo que sea necesario mantenerlos. Los ítems de configuración se clasifican en tres **categorías**:

1. **Ítems de producto:** Tienen un **ciclo de vida más largo** y se mantienen mientras el producto exista. *Ejemplos* incluyen documentos de arquitectura, casos de uso, código fuente, manuales de usuario, casos de prueba y bases de datos.
2. **Ítems de proyecto:** Tienen un **ciclo de vida que coincide con el proyecto** y se mantienen durante su duración. *Ejemplos* son el plan de proyecto y listados de defectos encontrados.
3. **Ítems de iteración:** Su **ciclo de vida corresponde a una iteración** específica del proyecto. *Ejemplos* incluyen planes de iteración, cronogramas de iteración y reportes de defectos.

CONTROL DE CAMBIOS

El control de cambios es un **procedimiento formal** que surge a partir de un **requerimiento de modificación** en uno o varios ítems de configuración que se encuentran en una **línea base establecida**. Una vez que se define una línea base, no se pueden realizar cambios en ella sin seguir un proceso formal de control de cambios. Este control se aplica a los ítems de configuración que forman parte de la línea base, ya que todos los involucrados en el desarrollo de software los utilizan como referencia.

El proceso de control de cambios es llevado a cabo por un **comité de control de cambios**, compuesto por diversos actores, que evalúa y decide si autoriza o no los cambios propuestos. Este comité puede incluir al **líder del proyecto, analistas, arquitectos e incluso el cliente**, dependiendo de la naturaleza del cambio propuesto.

Las **etapas del proceso de control de cambios** incluyen:

1. **Recepción de la propuesta** de cambio, que se refiere a una línea base en lugar de un ítem específico.
2. **Análisis de impacto** del cambio por parte del comité, **evaluando aspectos técnicos, gestión de recursos, efectos secundarios** y el **impacto global** en la funcionalidad y arquitectura del producto.
3. En caso de autorizarse el cambio, se **genera una orden de cambio** que detalla lo que se debe realizar, las restricciones a considerar y los criterios para revisar y auditar.
4. Después de **aplicar el cambio**, el comité vuelve a intervenir para **aprobar la modificación** de la línea base y marcarla nuevamente como línea base, realizando una revisión de partes.
5. **Se notifica a los interesados** sobre los cambios realizados en la línea base.

El control de cambios proporciona **trazabilidad** entre los ítems de configuración, lo que permite identificar qué ítems se ven afectados por un cambio y garantiza que los cambios sean gestionados de manera formal y documentada.

AUDITORÍAS DE CONFIGURACIÓN

La auditoría de configuración es un **conjunto de controles autorizados** que se realizan en un momento específico durante el desarrollo del software. En este proceso, **un auditor externo e imparcial** revisa las líneas base para evaluar el estado del software y asegurarse de que se cumplan las normas y directrices establecidas por la gestión de configuración.

La auditoría de configuración **es objetiva** cuando el auditor es independiente de cualquier influencia o prejuicio con respecto a lo que está auditando. Además, **es independiente** en el sentido de que el auditor no tiene ninguna relación jerárquica, funcional o de empleo con el equipo o el elemento que está siendo auditado.

Este proceso es importante para garantizar la calidad e integridad del producto mientras se encuentra en construcción, y se considera una medida preventiva, ya que es más eficiente y económico identificar y corregir problemas en esta etapa. La auditoría de configuración complementa las revisiones técnicas y verifica si las tareas se están realizando según lo planeado en el plan de gestión de configuración.

Existen dos tipos de auditorías de configuración: **la auditoría física de configuración (PCA)**, que se asegura de que lo que se establece en la línea base se haya logrado de manera efectiva, y **la auditoría funcional de configuración (FCA)**, que evalúa la funcionalidad y el rendimiento de los elementos de configuración para garantizar que coincidan con los requisitos especificados.

La auditoría de configuración sirve a **dos procesos** fundamentales: **validación y verificación**. **La validación** se asegura de que cada elemento de configuración resuelva el problema adecuado según las necesidades del cliente, mientras que **la verificación** verifica que un producto cumple con los objetivos definidos en la documentación de las líneas base.

(Después vemos mejor este tema)-

REPORTES E INFORMES DE ESTADO

Los **informes de gestión de configuración** proporcionan un mecanismo para **rastrear la evolución del sistema y la ubicación del software en comparación con la última versión publicada en la línea base**. Esto garantiza que todo el equipo trabaje con la versión más reciente y no con una versión obsoleta.

Estos informes **registran todos los cambios realizados** en las líneas base a lo largo del ciclo de vida del software. Dado que estos informes suelen contener grandes cantidades de datos, **se utilizan herramientas automatizadas para generarlos**. El objetivo principal es asegurarse de que la información sobre los cambios llegue a todos los involucrados en el proyecto.

Uno de los informes más comunes es el **informe de inventario**, que proporciona una **copia del contenido del repositorio** con la estructura de directorios. Estos informes **permiten responder preguntas** como el estado actual de un ítem, si una propuesta de cambio fue aceptada o rechazada por el comité, qué versión de un ítem implementa un cambio aprobado y cuál es la diferencia entre dos versiones de este ítem.

GESTIÓN DE CONFIGURACIÓN EN AMBIENTES ÁGILES

En metodologías ágiles, la **gestión de configuración** adquiere un **enfoque distinto** al ser de **utilidad para** los miembros del **equipo de desarrollo**. Se centra en **posibilitar el seguimiento y coordinación del desarrollo**, en contraposición al control directo de los desarrolladores, aprovechando la autoorganización de los equipos ágiles. En este contexto, las Auditorías de configuración no son propias de la gestión de configuración en entornos ágiles, y los procesos definidos en enfoques tradicionales son relativizados.

La gestión de configuración en ágil se alinea con el **manifiesto ágil** al **responder a los cambios y preservar la integridad** del producto. A diferencia del manifiesto ágil, que se centra en proyectos, la **gestión de configuración se enfoca en el producto**, trascendiendo el ámbito del proyecto. Mientras el manifiesto ágil aborda el contexto del proyecto de desarrollo, la gestión de configuración se convierte en una práctica específica del producto, garantizando su calidad.

Características de la Gestión de Configuración en Metodologías Ágiles:

1. **Adaptabilidad a Cambios:** La SCM **responde a los cambios** en lugar de evitarlos, alineándose con la naturaleza ágil de los requerimientos.
2. **Automatización Priorizada:** Se enfatiza la aplicación de **automatización** para facilitar la **entrega frecuente y rápida** de software (integración continua).
3. **Retroalimentación Continua:** Proporciona **retroalimentación constante** sobre **calidad, estabilidad e integridad** del software en desarrollo.
4. **Integración en Tareas del Sprint:** Las actividades de SCM están **integradas en las demás** tareas para alcanzar los objetivos del sprint.
5. **Responsabilidad Colectiva:** Es **responsabilidad de todo el equipo**, lo que implica que cada miembro requiere capacitación en gestión de configuración.
6. **Adopción de Prácticas Continuas:** Se promueve la adopción de **prácticas continuas** para mantener la **coherencia y calidad** del producto a lo largo del tiempo.

La diferencia principal entre las tres prácticas que vamos a ver radica en el nivel de automatización de las pruebas:

CONTINUOUS INTEGRATION

La integración continua es una **práctica de desarrollo** que insta a los **desarrolladores a integrar su código** en un **repositorio compartido múltiples veces al día**. Cada integración es sometida a **pruebas automatizadas** para **identificar y corregir errores** de manera temprana. El objetivo es asegurar que el software sea **desplegable en cualquier momento**, garantizando que el código compile y mantenga altos estándares de calidad.

Cada desarrollador realiza **pruebas unitarias**, preferiblemente **automatizadas**, empleando el enfoque del **Desarrollo Conducido por Pruebas (TDD)** (Ya lo vamos a ver mejor en testing). Una vez **completado y probado un componente de código** en el entorno de trabajo del desarrollador, **se sube al repositorio** de integración. Este proceso asegura que la versión del producto esté lista para someterse a pruebas de aceptación de usuario sin contratiempos.

CONTINUOUS DELIVERY

La entrega continua es una **disciplina de desarrollo de software** que **permite la liberación del software en producción en cualquier momento**. Se basa en la **automatización de pruebas** de aceptación, asegurando que el producto esté **listo para su despliegue**. Aunque no implica necesariamente la liberación con cada cambio, la versión está preparada inmediatamente para su lanzamiento, con la **decisión** de pasar a producción **tomada por el Product Owner**.

La integración continua es **un prerequisite para la entrega continua**, donde los artefactos producidos en el servidor de integración continua pueden ser desplegados en producción con un solo clic. La clave es tener un **despliegue automatizado** para agilizar el proceso y permitir despliegues más frecuentes, reduciendo así el riesgo de fallos. La meta es mantener el software siempre en un **estado "entregable"**, es decir, con el código compilado, pruebas pasadas y listo para su implementación.

CONTINUOUS DEPLOYMENT

El despliegue continuo implica **llevar el producto al ambiente de producción sin intervención humana**, utilizando **pipelines** con pasos secuenciales para asegurar una instalación exitosa (El despliegue continuo es como **Taylor Swift** dirían en mi curso, entrega temas constantemente).

Las estrategias, como Canary Deployment y Blue-Green Deployment, buscan hacer transparente para el usuario la implementación de nuevas versiones del producto.

Canary Deployment: (Canario?)

- Implementa **actualizaciones en etapas**, primero para un pequeño grupo de usuarios.
- **Permite la prueba y recolección de comentarios** antes de extender la actualización a todos los usuarios.
- **Muestra la interacción de los usuarios con los cambios** en el mundo real.
- Ofrece **actualizaciones sin tiempo de inactividad** y reversiones sencillas.

Blue-Green Deployment:

- **Transfiere gradualmente** el tráfico de una versión anterior a una nueva, ejecutándose en producción de manera casi idéntica.

- Entorno **"azul"** representa la **versión anterior**, **"verde"** la nueva.
- **Facilita la reversión** o retirada de producción al mantener la versión anterior en espera.
- **Permite la actualización continua** con **mínimos impactos** y **mayor flexibilidad**.

UNIDAD 4 | ASEGURAMIENTO DE CALIDAD DE PROCESO Y DE PRODUCTO

CONCEPTOS GENERALES SOBRE CALIDAD

CALIDAD

La calidad se define formalmente como el **cumplimiento de requerimientos funcionales y de rendimiento** explícitamente establecidos, así como de **estándares de desarrollo documentados**. La calidad engloba todas las **características de un producto o servicio** que impactan su capacidad para **satisfacer necesidades explícitas o implícitas**. Se distinguen dos tipos de calidad al examinar una entidad:

- **Calidad de Diseño:** Se refiere a las **características especificadas por los ingenieros**. En software, abarca **requisitos, especificaciones y el diseño** del sistema.
- **Calidad de Concordancia:** Representa el **grado de cumplimiento de las especificaciones de diseño** durante la implementación. Alta calidad de concordancia significa que la ejecución sigue el diseño y cumple con los objetivos de requisitos y rendimiento.

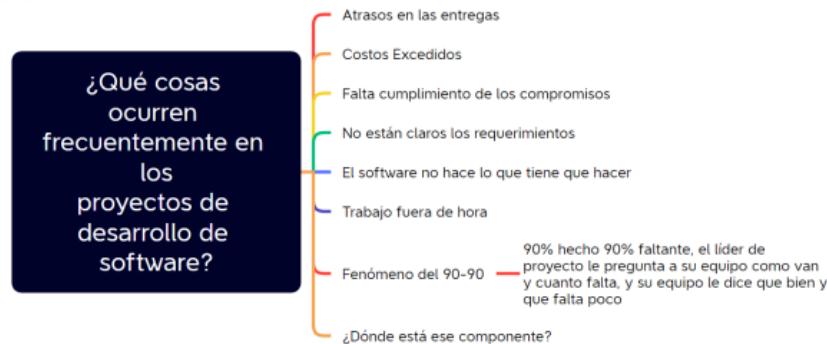
Es importante tener en cuenta que la percepción de la calidad puede variar entre personas y en diferentes momentos, ya que **es relativa**.

GESTIÓN Y ASEGURAMIENTO DE CALIDAD

El **aseguramiento de calidad** de procesos y productos se enfoca en la **prevención de problemas** y en **garantizar la máxima calidad en todas las etapas** del desarrollo de un producto o servicio. Esto significa **tomar medidas anticipadas** y realizar actividades en las **fases tempranas** del proceso para **evitar errores o defectos** en lugar de detectarlos y corregirlos después de que hayan ocurrido.

La **gestión de calidad** del software para los sistemas de software tiene **tres intereses** fundamentales:

- 👉 A **nivel de organización**, la gestión de calidad se ocupa de establecer un **marco de proceso y estándares** de organización que conducirán a software de mejor calidad.
- 👉 A **nivel del proceso**, la gestión de calidad implica la **aplicación de procesos específicos de calidad** y la verificación de que continúen dichos procesos planeados.
- 👉 A **nivel del proyecto**, la gestión de calidad se ocupa también de establecer un **plan de calidad para un proyecto**.



Estas situaciones concretas evidencian la falta de calidad o la NO calidad. El **aseguramiento de calidad** no se limita al producto, sino que también **involucra al proyecto** y al **equipo de trabajo**. Implica establecer **estándares y procedimientos** claros para llevar a cabo las actividades de manera consistente y eficiente, incluyendo la definición de **mejores prácticas, documentación de procesos, capacitación del personal y controles** para monitorear el desempeño.

El **aseguramiento de calidad de productos** se enfoca en **garantizar** que los productos cumplan con los **estándares de calidad establecidos**. Esto involucra **inspecciones, pruebas y evaluaciones** durante el proceso de fabricación para identificar y **corregir defectos** antes de la entrega al cliente. Además, se pueden implementar **sistemas de gestión de calidad**, como ISO 9001, para garantizar el cumplimiento de requisitos y mejores prácticas.

La **falta de calidad** en el software a menudo se manifiesta a través de

- ✖ demoras en la entrega
- ✖ costos excesivos
- ✖ incumplimiento de compromisos
- ✖ la no conformidad del software con los requerimientos

Por otro lado, el **software de calidad** cumple con las expectativas y necesidades de:

- ✓ El cliente
- ✓ El usuario
- ✓ La gerencia
- ✓ El equipo de desarrollo y mantenimiento.

IMPORTANCIA DE TRABAJAR PARA Y CON CALIDAD.

PRINCIPIOS DE ASEGURAMIENTO DE CALIDAD

1. La calidad debe estar **presente desde el inicio** del proceso, **no** se puede **agregar o comprar** después.
2. Asegurar la calidad es **responsabilidad de todos** los miembros del equipo.
3. La **capacitación del equipo** en calidad es esencial.
4. Contar con un **patrocinador o sponsor** a nivel gerencial que **respalde el enfoque en calidad**.
5. Los líderes deben **liderar con el ejemplo** y promover la cultura de aseguramiento de calidad.
6. Las **pruebas son fundamentales** para medir y controlar la calidad del software. **No se puede controlar lo que no se mide.**
7. Se debe **priorizar la simplicidad** y comenzar por lo **básico** en lugar de complicar los procesos.
8. Es crucial **planificar el aseguramiento de calidad** con objetivos, actividades y recursos definidos.
9. **Aumentar la cantidad de pruebas no garantiza** automáticamente una mayor **calidad**.
10. El aseguramiento de calidad debe ser **realista y adaptarse** a las necesidades y recursos de cada negocio.

VENTAJAS Y DESVENTAJAS.

Ventajas: 😊

1. **Competitividad:** Mejora la posición en el mercado al cumplir con estándares de calidad, lo que es esencial para competir internacionalmente.
2. **Lealtad del Cliente:** Aumenta la satisfacción del cliente al cumplir con sus expectativas, fomentando la retención y generando recomendaciones positivas.
3. **Eficiencia Operativa:** Reduce los costos y el retrabajo al minimizar errores, lo que resulta en un proceso más eficiente y rentable.
4. **Mejora Continua:** Facilita la identificación y corrección constante de áreas de mejora, promoviendo la evolución y adaptación continua del proceso.
5. **Credibilidad:** Certificaciones de calidad aumentan la credibilidad de la organización, generando confianza tanto interna como externamente.
6. **Costos a largo plazo:** Establece un equilibrio en términos de costo-efectividad al reducir los costos y el retrabajo, asumiendo que las fallas serán menores.

Desventajas: ☹️

1. **Costos Iniciales:** La implementación de estándares de calidad puede requerir inversiones iniciales significativas en formación, tecnología y procesos.
2. **Tiempo Adicional:** Seguir procesos rigurosos puede llevar más tiempo, lo que podría generar presiones en proyectos con plazos ajustados.
3. **Resistencia al Cambio:** Algunos empleados pueden resistirse a nuevos procedimientos y estándares, afectando la implementación eficaz de prácticas de calidad.
4. **Complejidad:** La gestión de la calidad puede añadir complejidad al proceso de desarrollo, especialmente en entornos grandes y complejos.
5. **Rigidez:** Demasiado enfoque en la calidad puede llevar a rigidez en los procesos, lo que podría limitar la agilidad y flexibilidad en la adaptación a cambios rápidos.

VISIONES DE CALIDAD - ¿CALIDAD PARA QUIÉN?

Cuando hablamos de calidad debemos saber desde que perspectiva y tenemos que integrar todas desde cuáles son las expectativas de calidad que pretende el usuario, hasta las del equipo de desarrollo (que muy probablemente no sean las mismas).

1. **Visión del usuario:** Relacionada con las **expectativas y satisfacción del usuario**. Incluye la facilidad de uso y rendimiento del software. El agilismo trata de incluir la visión de calidad del usuario a través del rol de **Product Owner**, el cual es ocupado por una persona con capacidad de decisión del cliente.
2. **Visión del producto:** Se enfoca en la **satisfacción de los requisitos** específicos del producto.
3. **Visión del proceso (manufactura):** Se evalúa **si el proceso** de desarrollo **es adecuado y agrega valor** al producto, evitando desperdicios.
4. **Visión del valor:** Busca **equilibrar el costo y los beneficios**, proporcionando el mayor valor posible al cliente.
5. **Visión Trascendental:** Asociada a **objetivos más allá de lo imaginado**, como alcanzar **cero defectos**, que impulsan la mejora continua. (como si esas cosas pasaran).

El aseguramiento de calidad se enfrenta al **desafío** de asegurarse de que la **calidad planeada**, la calidad **necesaria** y la calidad **lograda** estén alineadas en la medida de lo posible.

La **calidad programada** se refiere a las expectativas y estándares de calidad que se establecen **durante la planificación y el diseño del producto**. Es esencialmente la calidad que se busca alcanzar, y se convierte en el objetivo a lo largo del proceso de desarrollo.



La **calidad necesaria** es el **nivel mínimo de calidad requerido** para que el producto sea considerado adecuado y pueda satisfacer los requisitos y expectativas del usuario. Esto es lo que el usuario necesita y espera que el producto entregue.

Por último, la **calidad lograda** se refiere a la **calidad real que se ha alcanzado** en el producto final una vez que se ha completado el proceso de desarrollo y las pruebas correspondientes. Es el **resultado concreto y medible** de todas las actividades de desarrollo, aseguramiento y control de calidad.

Cuando todas **coinciden, se minimiza el riesgo de desperdicio y de insatisfacción del cliente.**

CALIDAD EN EL SOFTWARE

Para desarrollar software eficazmente, se necesita un **proceso como guía**. Este proceso se basa en **modelos de referencia** que ofrecen **mejores prácticas y estándares** para garantizar la calidad. Los modelos de calidad dicen: **hace tu proceso de manera que tenga calidad para vos (empresa), pero este proceso debe ser compatible con lo que yo te digo (modelo de referencia)**. Se pueden utilizar **modelos de mejora de procesos** para la mejora continua. La formalización y certificación del proceso se logra mediante **modelos de evaluación**.

Los **proyectos son la unidad de trabajo que da vida al proceso**, y es esencial incorporar **actividades de aseguramiento de calidad**, como revisiones técnicas y auditorías, en el contexto del proyecto. El aseguramiento de calidad de **proceso** se centra en **mejorar la forma de trabajar**, mientras que el aseguramiento de calidad de **producto** se enfoca en **evaluar y garantizar la calidad del producto resultante**. Ambos son cruciales para un desarrollo de software exitoso y de alta calidad.

Los **procesos se instancian en proyectos**, y es en los proyectos donde se añaden actividades para evaluar si se están siguiendo adecuadamente. Las **revisiones técnicas** son **evaluaciones entre compañeros** que se centran en los productos, no en las personas. Las **auditorías**, realizadas por **terceros**, son menos comunes en enfoques ágiles. La **calidad del producto** en desarrollo **se garantiza mediante tareas como revisiones técnicas, auditorías de configuración funcional** (la que valida una versión de la línea base) **y física** a (la que verifica una versión de la línea base), **y pruebas (Testing)**. Los modelos de calidad sugieren que se siga un proceso que sea tanto efectivo para la empresa como compatible con los estándares definidos.

El **Aseguramiento de Calidad de Proceso** se enfoca en mejorar y optimizar el proceso de desarrollo de software, estableciendo buenas prácticas y estándares. Implica actividades como definir estándares, realizar revisiones técnicas, auditorías de proceso y gestionar la configuración del software.

ESTÁNDARES

La implementación de estándares en el desarrollo de software es **crucial** por varias razones fundamentales. En primer lugar, estos estándares **encapsulan la sabiduría acumulada** que resulta **valiosa** para la organización, reflejando las **mejores prácticas y lecciones aprendidas**. Además, proporcionan un **marco objetivo** para definir la "calidad", estableciendo criterios claros y medibles.

Tipos de Estándares

1. **Estándares de Producto:** Documentación, estructura de documentos, codificación.
2. **Estándares de Proceso:** Especifican buenas prácticas en el desarrollo de software.

Importancia:

- 👉 Encapsulan mejores prácticas, evitando la repetición de errores.
- 👉 Proporcionan un marco para el aseguramiento de calidad.
- 👉 Aseguran continuidad y comprensión organizacional para el nuevo personal.

Problemas:

- ☹ No siempre son vistos como relevantes por los Ingenieros de Software.
- ☹ Pueden ser considerados burocráticos y consumir tiempo.
- ☹ Requieren mantenimiento manual si no están respaldados por herramientas automatizadas.

CALIDAD DE PRODUCTO¹

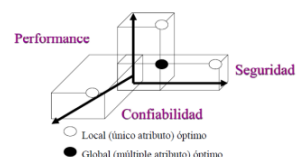
El **Aseguramiento de Calidad de Producto** se concentra en garantizar que el producto de software cumpla con los **requisitos y estándares de calidad**.

La **calidad del producto no se puede sistematizar, no hay modelos** en la industria **generalizados** para evaluar calidad de producto que se puedan aplicar como una plantilla a todos los productos igualmente. Su **aseguramiento** se hace **mediante las Revisiones Técnicas** (Revisiones de pares), **Auditorías y validaciones de usuarios**, mientras que su **control** se hace mediante el **Testing**.

MODELOS DE CALIDAD DE PRODUCTO

Los modelos de calidad del software, como el Modelo de Barbacci, el Modelo de McCall y la ISO 25.010, **permiten evaluar aspectos de calidad del producto** de software, como **confiabilidad, rendimiento y seguridad**. Sin embargo, debido a la **variación de requerimientos** en diferentes proyectos de software, **no existe un modelo único** que pueda certificar la calidad en todos los aspectos.

El **Modelo de Barbacci** se centra en **equilibrar la confiabilidad, el rendimiento y la seguridad** para evaluar la calidad del producto.



¹ Citado más adelante

Por otro lado, el **Modelo de McCall** considera tres factores determinantes de calidad: **revisión del producto** (capacidad de verificación del producto. Es la facilidad de mantenimiento, flexibilidad y prueba), **operación del producto** (Es la calidad del producto en uso, lo que más le importa al usuario final. Corrección, fiabilidad y usabilidad) y **transición del producto** (Es la facilidad con la que el producto puede expandirse y crecer).



Por último la **ISO 25000**, centrada en Requisitos No Funcionales (RNF), establece **estándares para evaluar características no relacionadas con la funcionalidad** en sistemas de software. Se enfoca en aspectos como **eficiencia, fiabilidad, seguridad y usabilidad**. Proporciona un marco para la especificación, medición y evaluación de estos requisitos no funcionales, permitiendo una gestión efectiva para asegurar la calidad global del software.

Cada uno de estos modelos **se adapta a las necesidades específicas de un proyecto**.

Es esencial tener en cuenta que la evaluación de calidad de un producto de software se realiza en función de los **requerimientos específicos del cliente y no únicamente comparándolo con modelos teóricos**.

CALIDAD EN EL PROCESO

La **calidad en el proceso de desarrollo** de software es **fundamental para mejorar la calidad del producto** y el rendimiento de la organización. El **proceso** es el **único factor** que **se puede controlar y optimizar** para lograr un software de calidad.

El aseguramiento de calidad de software implica la **incorporación de acciones en todas las etapas del proceso** para detectar oportunidades de mejora tanto en el producto como en el proceso en sí. Esto incluye la **definición de estándares y procesos** de calidad **adecuados** y asegurar su cumplimiento. Además, promueve una cultura de calidad en la que todos son responsables de la calidad.

A raíz de un **proceso que adopte la calidad**, se puede lograr un **proyecto que tenga la calidad** como factor **embebido**, lo que lleva a un **software de calidad**.

Es importante tener en cuenta que **no existe un proceso único que sirva para todas las organizaciones y proyectos**. La calidad del proceso es un medio para lograr la calidad del producto, y se debe adaptar a las necesidades y características específicas de cada proyecto. Además, factores **como la tecnología, las personas, las características del cliente y las condiciones de negocio** son **difíciles de controlar y no se pueden modificar**.

DEFINICIÓN DE UN PROCESO DE SOFTWARE

Un **proceso** se define como una secuencia de pasos ejecutados con un propósito específico.

En el **contexto del desarrollo de software**, es un conjunto de actividades, métodos, prácticas y transformaciones que las personas utilizan para crear o mantener software y sus productos relacionados.

Además de las **tareas** escritas, un **proceso de software** requiere **personas** con habilidades, **entrenamiento y motivación**, así como el apoyo de **herramientas automatizadas y equipamiento** para ser efectivo. Es como una mesa de tres patas que necesita tanto **lineamientos** como **recursos humanos y tecnológicos** para funcionar adecuadamente.

¿Cómo es un proceso para Construir Software?

1. **Definir el proceso de forma explícita y comunicarlo** a todo el equipo, especificando **etapas, subetapas, roles y pautas** de trabajo.
2. Dentro de la definición, (la cual plantea las etapas para construir la parte técnica), se debe incluir **disciplinas transversales**, como **Planificación y Seguimiento de Proyectos, SCM y QA**, en la definición del proceso, sin importar la metodología utilizada.
3. **Asegurar** que el proceso definido **agregue valor** al producto y **pueda adaptarse** a proyectos específicos, utilizando modelos de mejora de procesos como Kanban.
4. En la **adaptación del proceso**, se deben **considerar aspectos no negociables**, como la realización de pruebas (Testing), que no pueden ser eliminados.

PROCESOS BASADOS EN CALIDAD

Las personas encargadas de la calidad en el desarrollo de software establecen **procesos para garantizar la calidad** del producto final, incluyendo actividades como **planificación, diseño, implementación, pruebas y entrega**. **Evalúan** continuamente la efectividad de estos procesos **mediante la evaluación del producto** resultante, tomando medidas para mejorar el proceso si se encuentran deficiencias. **Si el producto cumple** con los estándares de calidad, **se estandariza** el proceso para su aplicación consistente en proyectos futuros.



ACTIVIDADES RELACIONADAS CON EL ASEGURAMIENTO DE LA CALIDAD DEL SOFTWARE.

ADMINISTRACIÓN DE LA CALIDAD DEL SOFTWARE

La Administración de la Calidad de Software busca garantizar niveles requeridos de calidad mediante la definición y cumplimiento de estándares y procesos. Su objetivo principal es inculcar una cultura de calidad en todo el equipo, donde la responsabilidad recae en cada miembro. Se promueve la detección temprana de oportunidades de mejora en el producto y el proceso.

Reporte del grupo de aseguramiento de calidad (GAC)

- El GAC **no debe reportar al líder del proyecto** para mantener independencia y objetividad.
- Debe tener un **reporte independiente** y estar lo más **cerca** posible **del nivel más alto** de la organización.
- El reporte debe ser **directo a la gerencia** de dirección.
- El GAC debe **reportar** a alguien **interesado en la calidad** del software.

Actividades de Administración de Calidad

1. **Aseguramiento de calidad:** Define estándares y procesos para comparaciones.
2. **Planificación de calidad:** Define actividades, tiempos y selecciona estándares aplicables al proyecto.
3. **Control de calidad:** Ejecución planificada, asegura el respeto de procedimientos y estándares por el equipo.

Control de calidad mediante:

- Revisiones de calidad: Examinan procesos/productos y documentación para identificar problemas.
- Evaluaciones automáticas y mediciones.

Funciones del Aseguramiento de Calidad de Software:

- ✓ Prácticas de aseguramiento de calidad.
- ✓ Evaluación de la planificación del proyecto, requerimientos, diseño y prácticas de programación.
- ✓ Adaptación de procedimientos de calidad para cada proyecto.

Calidad de procesos en la práctica:

- Establecimiento de estándares para revisiones y administración de configuración.
- Monitoreo del proceso de desarrollo para garantizar el respeto a estándares.
- Reporte a la Administración de Proyectos y al responsable del SW.
- Evitar prácticas inapropiadas, incluso si están estandarizadas.

PRINCIPALES MODELOS DE CALIDAD EXISTENTES (CMMI – SPICE – ISO)

MODELOS DE MEJORA DE PROCESOS

La mejora continua de procesos implica **comprender y cambiar los procesos existentes** para mejorar la calidad del producto o reducir costos y tiempo de desarrollo. Los **modelos de mejora** son **recomendaciones para encarar proyectos de mejora de procesos**, con el objetivo de obtener un proceso mejorado que se aplique en futuros proyectos. Estos modelos **no dicen cómo hacer las cosas**, sino que **proporcionan estándares y directrices** para la mejora de procesos.

MODELO IDEAL

El Modelo IDEAL (*Initiating, Diagnosis, Establishing, Acting, Learning*) es una **guía de mejora de procesos**, este modelo se utiliza para **definir proyectos** que ayuden a mejorar los procesos en una organización y es un **enfoque cíclico** para la **mejora continua**.

El Modelo IDEAL se divide en **cinco fases**:

1. **Inicialización:** Se reconoce la necesidad de cambio en la organización y se busca apoyo (*sponsor*) para la mejora. Esto es importante porque **una mejora de proceso nunca es crítica**
2. **Diagnóstico:** Se **evalúa la madurez** actual de la organización y se **identifican los riesgos** asociados al proceso de mejora.
3. **Establecimiento:** Se elabora un **plan detallado** con acciones específicas, entregables y responsabilidades **basado en los resultados del diagnóstico** y los objetivos de mejora.
4. **Ejecución/Acción:** Se **implementan** las acciones planificadas en un **proyecto piloto**, y si es exitoso, se implanta en la organización.
5. **Aprendizaje:** Se **revisa la información recopilada** en pasos anteriores para evaluar los logros y aprender a implementar cambios de manera más efectiva y eficiente en el futuro.

MODELO SPICE

El Modelo SPICE (*Software Process Improvement Capability Evaluation*) es un **modelo dual teórico** utilizado para la **mejora de procesos** de desarrollo de software (ISO 15504). Se divide en **dos partes**:

1. **Modelo de Calidad:** Define **conjuntos predefinidos de procesos** con el objetivo de mejorar la calidad de un producto de software. Entran en los análisis de brecha (dónde estamos y dónde queremos ir)
2. **SPICE + IDEAL:** Son **modelos de mejora de procesos** que se utilizan para crear proyectos de mejora en una organización. El resultado de estos proyectos es un proceso definido que se utiliza en futuros proyectos.

El modelo SPICE **establece 6 niveles de madurez** para clasificar a las organizaciones y proporciona un camino de mejora predefinido. Se utiliza para evaluar y mejorar los procesos de desarrollo de software en una organización.

Nivel	Estado
Nivel 0 - Organización inmadura	La organización no tiene una implementación efectiva de los procesos
Nivel 1 - Organización básica	La organización implementa y alcanza los objetivos de los procesos
Nivel 2 - Organización gestionada	La organización gestiona los procesos y los productos de trabajo se establecen, controlan y mantienen
Nivel 3 - Organización establecida	La organización utiliza procesos adaptados basados en estándares
Nivel 4 - Organización predecible	La organización gestiona cuantitativamente los procesos
Nivel 5 - Organización optimizando	La organización mejora continuamente los procesos para cumplir los objetivos de negocio

MODELOS DE CALIDAD (PARA EVALUAR PROCESOS)

Plantean una **definición teórica (lineamientos)** del proceso que se utilizará en una organización (con diferentes niveles de detalle según lo que se necesite), para luego **compararlo con la ejecución** del proyecto donde se implementa (**instancia**) ese proceso de desarrollo. Esto **permite medir qué tanto se está cumpliendo** en el proyecto con lo que plantea la definición teórica del proceso de desarrollo (definición de roles, asignación de responsabilidades, actividades a realizar, etc.), a través de las auditorías de proyecto. *¡No es lo mismo que un estándar de proceso!* El objetivo de estos modelos de calidad es **acreditar que una organización tiene cierto nivel de madurez** en su proceso de desarrollo adoptado, **o bien que cierta área de esa organización tiene determinado nivel de madurez**. Existen un montón de modelos de calidad. Los más utilizados en Argentina son el CMMI, CMMI 2.0, etc.

CAPABILITY MATURITY MODEL INTEGRATION – CMMI

El Modelo CMMI (Capability Maturity Model Integration) es una **guía de referencia** utilizada por organizaciones de desarrollo de software para **mejorar sus procesos** y alcanzar niveles superiores de calidad y eficiencia. CMMI se enfoca en **áreas de proceso específicas** y **se estructura en niveles de madurez**. Sus principales **características** incluyen:

- Enfoque en Mejora de Procesos:** CMMI tiene como objetivo principal **mejorar la capacidad y madurez** de los procesos organizacionales en el desarrollo de software. Proporciona **pautas y buenas prácticas** para lograr una mayor calidad y satisfacción del cliente.
- Niveles de Madurez:** CMMI utiliza **niveles de madurez** para representar **etapas de mejora evolutiva** en la organización, desde un **nivel inicial ad hoc** hasta un **nivel de optimización continuo**. Esto ayuda a las organizaciones a evaluar y mejorar sus procesos de manera progresiva.
- Áreas de Proceso:** CMMI se enfoca en **áreas de proceso específicas**, como la **planificación**, el **monitoreo y control**, la **gestión de requisitos**, el **diseño**, la **implementación** y la **evaluación**. Estas áreas abordan aspectos clave del desarrollo y gestión de software.
- Evaluación Formal:** CMMI utiliza **evaluaciones formales** realizadas por profesionales capacitados para **medir la madurez y capacidad** de los procesos organizacionales. Esto proporciona una **medida objetiva** de la mejora alcanzada.
- Beneficios:** Al seguir las recomendaciones de CMMI, las organizaciones pueden lograr beneficios como la **reducción de defectos**, **mayor productividad**, **gestión de riesgos efectiva** y **alineación con los objetivos** del negocio.
- Flexibilidad y Adaptabilidad:** CMMI es un modelo flexible y adaptable que **se puede personalizar** según las necesidades y características de cada organización.

CMMI se divide en tres dominios o ámbitos de mejora, conocidos como Constelaciones:

- ★ **CMMI DEV:** Enfocado en el **desarrollo de software**, proporciona guías para medir, monitorear y administrar los procesos de desarrollo.
- ★ **CMMI SVC:** Orientado a la **entrega de servicios**, tanto internos como externos.
- ★ **CMMI ACQ:** Dirigido a la **administración, selección y adquisición de productos o servicios**.

REPRESENTACIONES CMMI

CMMI puede ser representado de **dos formas diferentes**: por etapas y de manera continua. Ambas representaciones comparten áreas de proceso y prácticas idénticas, pero **difieren en su enfoque de implementación**.

La representación por etapas organiza las prácticas de CMMI en niveles de madurez. Cada nivel implica un **conjunto específico de áreas de proceso** que deben cumplirse para alcanzar ese nivel. Los niveles, en orden ascendente de madurez, son:

- **Nivel 1 (Inicial)**: La organización tiene procesos ad hoc y no estandarizados, carece de visibilidad sobre el proceso y es reactiva, centrada en resolver problemas después de que ocurren.
- **Nivel 2 (Administrado)**: Se establecen controles básicos para gestionar proyectos y procesos, con enfoque en estandarización y métricas.
- **Nivel 3 (Definido)**: Los procesos se documentan y estandarizan, roles y responsabilidades se definen, y se promueve la retroalimentación.
- **Nivel 4 (Cuantitativamente Administrado)**: La organización cuantifica y mide su rendimiento, utiliza métricas y datos para analizar el desempeño.
- **Nivel 5 (Optimizado)**: La organización busca la mejora continua y la optimización de procesos.

Las **organizaciones inmaduras** son aquellas que se sitúan en el nivel 1, mientras que las **organizaciones maduras** están en los niveles 2 a 5. Cuanto **mayor es la madurez** de la organización, **mayor es su capacidad para alcanzar sus objetivos**, resulta en una **mejora en la calidad** de sus productos y una **reducción de los riesgos** asociados.

Enfocamos nuestra atención en el **nivel 2**, donde las áreas de proceso incluyen la **administración de acuerdos con proveedores**, que no es obligatoria pero es **necesaria** al contratar a una empresa externa. Hay dos tipos de proveedores: **Tipo A**, que cumplen con el **mismo nivel de calidad**, y **Tipo B**, que **no cumplen con el mismo nivel de calidad**. Al alcanzar el nivel 2, la organización demuestra la madurez para gestionar proyectos y asegurar que el producto final sea previsible.

La representación por etapas **proporciona una estructura clara y progresiva** para mejorar la madurez de la organización, donde cada nivel se construye sobre los logros del nivel anterior. Esta representación tiene la ventaja de proporcionar una **clasificación única** que **facilita las comparaciones** entre organizaciones.

La representación de manera continua del modelo CMMI permite a las organizaciones **elegir áreas de proceso específicas** de entre las **22 disponibles** en el modelo y **mejorarlas por separado**. En lugar de medir la madurez de toda la organización, **se evalúa la capacidad** de procesos **individuales** para cumplir con sus objetivos. Cada área de proceso tiene metas y prácticas específicas, y la evaluación de la capacidad se realiza en una escala de 0 a 5, indicando el grado de implementación y desempeño de esas prácticas.

Esta representación **brinda la flexibilidad** de trabajar en áreas de proceso específicas de acuerdo con las necesidades y prioridades de la organización, sin requerir una secuencia fija. Aunque puede ser más complicado comparar el progreso entre diferentes organizaciones.

En el nivel 1, no hay áreas de proceso, ya que se considera que la organización está en un estado de inmadurez. En el nivel 2, hay siete áreas de proceso, siendo la última opcional y dependiendo de las actividades específicas de la organización

1. REQM → Requirement Management (Gestión de Reqs)
2. PP → Project Planning (Gestión de Proyecto)
3. PMC → Project Monitor and Control (Gestión de Proyecto)
4. MA → Measure and Analytics (Métricas de Proyecto)
5. PPQA → Product and Process Quality Assurance (Disciplina Transversal)
6. SCM → Software Configuration Management (Disciplina Transversal)
7. SAM → Supplier Agreement Management (Gestión de Acuerdos con Proveedores)

Roles y Grupos

CMMI se refiere a roles y grupos en el contexto de las **organizaciones**. **Los grupos están formados por roles** que **desempeñan tareas específicas**, y estos roles **se ajustan al tamaño de la organización** y a sus metas de madurez. Lo fundamental es asegurarse de que alguien sea responsable de cubrir las actividades de cada rol o grupo. Ver niveles de madurez

El **enfoque en el Nivel 2** se centra en la **administración y controles**. Al alcanzar el Nivel 2 de CMMI, la organización **demuestra madurez en la gestión de proyectos y la capacidad de producir productos de software** de acuerdo con expectativas definidas.

Para mejorar los procesos, IDEAL es el modelo de mejora y marco de referencia, y **CMMI se utiliza como el modelo de calidad**. La evaluación se realiza a través de instancias de auditoría o certificaciones, donde un grupo de personas evalúa el proceso. El método formal para evaluar y determinar el nivel o capacidad de CMMI se llama SCAMPI.

INTERESES FUNDAMENTALES EN LA GESTIÓN DE CALIDAD

La gestión de calidad en el **ámbito organizacional** se enfoca en establecer un **marco de proceso y estándares** que **conduzcan a** la producción de **software de alta calidad**. Esto implica que el equipo de gestión de calidad debe definir los procesos de desarrollo del software, los estándares aplicables y la documentación relacionada, como requerimientos, diseño y código del sistema.

A nivel de proceso, la gestión de calidad implica la implementación de **procesos específicos de calidad y la supervisión** para garantizar su cumplimiento, asegurando que los resultados del proyecto cumplan con los estándares correspondientes.

En el nivel del proyecto, la gestión de calidad se ocupa de crear un **plan de calidad que establece metas de calidad y define los procesos y estándares a utilizar en ese proyecto**.

Puntos clave:

- * La calidad del software se puede analizar desde **diversas perspectivas: como proceso y como producto**.
- * **Medir** la calidad del software puede ser **desafiante**.
- * **Mejorar** la calidad del **software** se basa en **mejorar el proceso**.
- * La **inversión** en calidad suele ser más **rentable**.
- * La mejora exitosa de procesos requiere **compromiso y cambios organizacionales**.
- * Hay varios **modelos disponibles** para respaldar los esfuerzos de mejora de calidad en el software.
- * La mejora de proc. en el desarrollo de software ha demostrado generar **retornos significativos de inversión**.

CMMI CARA A CARA CON ÁGIL

- “Nivel 1”
 - Identificar el alcance del trabajo
 - Realizar el trabajo
- “Nivel 2”
 - Política Organizacional para planear y ejecutar
 - Requerimientos, objetivos o planes
 - Recursos adecuados
 - Asignar responsabilidad y autoridad
 - Capacitar a las personas
 - Administración de Configuración para productos de trabajo elegidos
 - Identificar y participar involucrados
 - Monitorear y controlar el plan y tomar acciones correctivas si es necesario
 - **Objetivamente monitorear adherencia a los procesos y QA de productos y/o servicios**
 - Revisar y resolver aspectos con el nivel de administración más alto
- “Nivel 3”
 - Mantener un proceso definido
 - **Medir la performance del proceso**
- “Nivel 4”
 - **Establecer y mantener objetivos cuantitativos para el proceso**
 - **Estabilizar la performance para uno o más subprocesos para determinar su**
 - **habilidad para alcanzar logros**
- “Nivel 5”
 - **Asegurar mejora continua para dar soporte a los objetivos**
 - Identificar y corregir causa raíz de los defectos

Referencias:
Verde: Da soporte
Negro: Neutral
Rojo: Desigual

La relación entre CMMI y el enfoque ágil **implica ciertas diferencias y desafíos**. En un entorno **ágil**, se pone **énfasis en la gestión ágil** de requerimientos y **no se requiere un control estricto** de los requerimientos en cada momento, mientras que **CMMI sí lo requiere**. **Ambas metodologías deben adaptarse para funcionar juntas**, por ejemplo, CMMI podría ceder en la realización de auditorías.

A medida que se avanza en los niveles de madurez de CMMI, **surgen diferencias**. **CMMI exige** definir un proceso y asegurarse de que se cumple, lo que se verifica mediante **auditorías**, mientras que el enfoque **ágil no evalúa** si se ha seguido un proceso definido. Además, **CMMI, a partir de los niveles 3 o 4, se centra en métricas y estadísticas** de proyectos y productos, **y utiliza la comparación** de estas medidas para evaluar el proceso. En cambio, **el enfoque ágil argumenta que la experiencia no es extrapolable a otros proyectos**.

DIFERENCIAS ENTRE CMMI Y AGILE

VALORES ESENCIALES

- **CMMI se enfoca en medir y mejorar el proceso para lograr mejores procesos**, lo que, a su vez, lleva a un **mejor producto**. Por otro lado, los **Métodos Ágiles ponen énfasis en proporcionar respuestas ágiles a los clientes y minimizar la sobrecarga en el proceso**. Para el **refinamiento de requerimientos**, utiliza métodos como **metáforas y casos de negocio**
- **CMMI tiende a asociarse con personas disciplinadas** que **siguen reglas** y tienen **aversión al riesgo**. En contraste, los **Métodos Ágiles valoran a personas cómodas, creativas y dispuestas a asumir riesgos**.
- En cuanto a la comunicación, **CMMI la considera en un sentido organizacional y macro**, mientras que los **Métodos Ágiles se centran en la comunicación de persona a persona y en un nivel micro**.
- La gestión del conocimiento en **CMMI implica la gestión de activos de proceso**, mientras que, en los **Métodos Ágiles, se enfoca más en la gestión del conocimiento a través de las personas**.

CARACTERÍSTICAS

CMMI:

- Enfoque en la mejora organizacional.
- Busca uniformidad y nivelación de procesos.
- Se centra en la capacidad y la madurez buscando el éxito a través de la predictibilidad.
- Tiene un cuerpo de conocimiento que cruza dimensiones y está estandarizado.
- Desalienta las reglas de atajo.
- Implica comités en su funcionamiento.
- La confianza del cliente se basa en la infraestructura del proceso.
- Se caracteriza por estar cargado al frente y enfocado en mover hacia la derecha.
- Su alcance de la vista es amplio, inclusivo y organizacional.
- Nivel de discusión se basa en palabras, definiciones, es duradero y exhaustivo.

Ágiles:

- Enfocados en la mejora en el proyecto.
- Valoran la tradición oral y la innovación.
- Buscan la capacidad y la madurez, centrándose en notar de oportunidades.
- Tienen un cuerpo de conocimiento personal, en constante evolución y temporal.
- Alientan las reglas de atajo.
- Se basa en la toma de decisiones individuales.
- La confianza del cliente se basa en el software funcionando y la participación.
- Se caracteriza por ser conducido por pruebas y enfocado en mover hacia la izquierda.
- Su alcance de la vista es pequeño y focalizado.
- Nivel de discusión se basa en el trabajo en mano.

ENFOQUE

CMMI:

- Tienen un enfoque descriptivo, proporcionando descripciones detalladas
- Se basan en enfoques cuantitativos con números científicos y datos duros.
- Buscan una universalidad en su aplicación.
- Se centran en actividades y siguen un enfoque estratégico.
- Tendencia a preguntar "¿Cómo lo llamaremos?".
- La gestión de riesgos es proactiva.

Ágiles:

- Adoptan un enfoque más prescriptivo, que se centra en la acción y la ejecución.
- Utilizan un enfoque cualitativo basado en conocimiento tácito.
- Se adaptan de manera situacional a las necesidades específicas del proyecto.
- Enfatizan el producto y siguen un enfoque táctico.
- Adoptan una actitud de "¡Sólo hazlo!".
- La gestión de riesgos es reactiva

FOCO

CMMI:

- Se enfoca en el negocio de manera interna, siguiendo reglas y procesos establecidos.
- Busca la predictibilidad y la estabilidad en la ejecución de proyectos.

Métodos Ágiles:

- Centran su atención en el negocio de manera externa, priorizando la innovación y la satisfacción del cliente.
- Ponen énfasis en el rendimiento y la velocidad de ejecución

SIMILITUDES ENTRE CMM Y MÉTODOS ÁGILES

1. Ambos tienen como meta la creación de organizaciones de **alto desempeño** en el desarrollo de software.
2. Tanto CMM como los Métodos Ágiles **implican la planificación**.
3. Comparten la característica de ser **CMMs** (Consultant Money Makers)
4. Ambos **establecen reglas**, que son equivalentes a los **requerimientos del proceso**, y violar estas reglas conlleva consecuencias serias.
5. Ambos enfoques **no son completos** por sí solos y **requieren adaptaciones** a situaciones específicas.
6. Se **basan en la experiencia** y no se centran en la introducción de ideas completamente nuevas.
7. No son aplicables a cualquier proyecto y **requieren consideración de las circunstancias particulares**.

En resumen, tanto enfoques tradicionales como ágiles reconocen la importancia de la calidad en el desarrollo de productos de software y comparten similitudes en su enfoque hacia la mejora de procesos y la planificación.

AUDITORÍAS DE SOFTWARE

Las auditorías de software son **evaluaciones independientes** de **productos o procesos** de software para **garantizar el cumplimiento de estándares, pautas, especificaciones y procedimientos**, utilizando **criterios objetivos y documentación** que abarca:

- La **estructura y contenido** de los productos a desarrollar.
- El **proceso por seguir** para desarrollar esos productos.
- La **manera de medir el cumplimiento** de estándares y pautas.

Estas auditorías, realizadas por **equipos externos** al proyecto, son una **actividad de apoyo** que asegura el cumplimiento de estándares y procedimientos, implican esfuerzo y costos, pero sus **beneficios son significativos**.

Las **ventajas** de las auditorías incluyen:

- Obtener **opiniones objetivas** e imparciales.
- **Identificar áreas de insatisfacción** potencial para el cliente.
- **Asegurar el cumplimiento** de las **expectativas**.
- Brindar **visibilidad sobre los procesos de trabajo** y **oportunidades de mejora**.
- **Ofrecer información** a la gerencia **sobre los procesos de trabajo**.
- **Asegurar una implementación efectiva** de procesos de desarrollo y actividades de soporte.

El resultado de las auditorías es la **mejora de productos, clientes satisfechos y el crecimiento del negocio**.

TIPOS DE AUDITORÍAS

1. Auditoría de Proyecto:

- Valida el cumplimiento del **proceso** de desarrollo con el proceso acordado.
- Se realiza según lo establecido en el **Plan de Aseguramiento de Calidad de Software** (PACS).
- Incluye **inspecciones de software y revisiones de documentación** de diseño y prueba.
- **Objetivo:** garantizar la **consistencia** del producto durante su evolución, asegurando la coherencia con los requerimientos y el diseño, establecidos en la ERS y DDS.

2. Auditorías de Configuración Funcional:

- Valida que el **producto cumpla con los requerimientos especificados** en la Especificación de Requisitos de Software (ERS).
- Compara el software construido con los requerimientos funcionales de la ERS.
- Se asegura de que el código implemente completamente los requerimientos y las capacidades funcionales.
- **Verifica la actualización de la matriz de rastreabilidad** por parte del responsable de QA.

3. Auditoría de Configuración Física:

- Valida que el **ítem de configuración se ajuste a la documentación** técnica que lo describe (esto permite la trazabilidad y la satisfacción de los requerimientos).
- **Compara el código con la documentación** de soporte.
- Asegura que la **documentación** entregada sea **consistente** y **describa** correctamente el **código** desarrollado.
- Se realiza **antes de la entrega del software**, y cualquier desviación encontrada debe ser corregida antes de la entrega.

Se podría agregar un 4to tipo (no estoy segura de que sea un tipo pero a esto se refieren con “auditorías al grupo de calidad” en el temario):

4. Auditorías Externas de Aseguramiento de Calidad:

- Realizadas por **personal externo** a la organización.
- **Evalúan a los miembros de QA** para asegurar la correcta realización de las evaluaciones internas de calidad.
- **Contribuyen a una evaluación objetiva** de la calidad y prácticas de la organización.

ROLES EN UNA AUDITORÍA

1. Auditado:

- **Participa** en la auditoría.
- **Propone la fecha** de la auditoría.
- **Entrega evidencia** requerida.
- **Responde a las preguntas** del auditor.
- **Propone planes de acción** para abordar desviaciones encontradas.
- **Responde al reporte** de auditoría.
- Propone un plan de acción para corregir deficiencias citadas en el reporte.
- **Generalmente es el Líder de Proyecto**, pero no necesariamente.

2. Auditor:

- Debe ser una **persona externa** al proyecto auditado, como el grupo de aseguramiento de calidad.
- **Acuerda la fecha y alcance** de la auditoría.
- **Recolecta y analiza evidencia relevante y suficiente** para evaluar el proyecto.
- **Realiza la auditoría.**
- **Prepara un reporte** de auditoría.
- **Hace seguimiento a los planes de acción** acordados con el auditado.

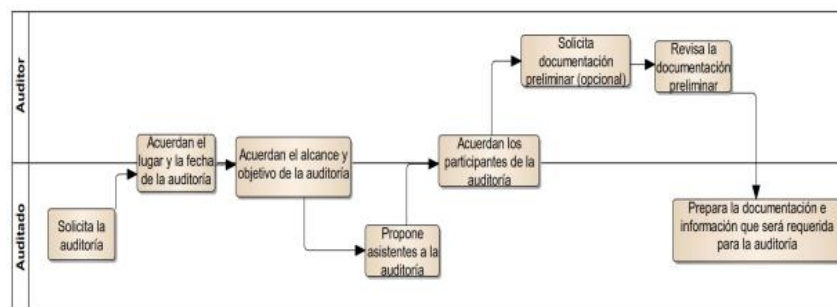
3. Gerente de SQA (Software Quality Assurance):

- **Prepara el plan de auditoría.**
- **Calcula los costos y asigna recursos** para la auditoría.
- **Resuelve las no-conformidades** entre el auditor y el auditado.
- **Enfocado en la gestión de la auditoría.**

PROCESO DE AUDITORÍAS

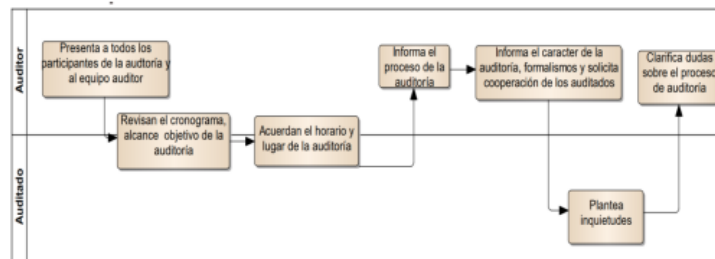
1. Preparación y Planificación:

- La auditoría se planifica conjuntamente entre el auditado y el auditor.
- El líder de proyecto generalmente convoca la auditoría.
- La planificación se realiza de manera anticipada y no es sorpresiva.

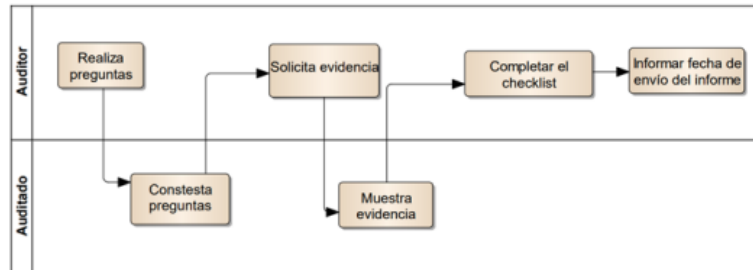


2. Ejecución:

- Durante la ejecución, el auditor solicita documentación y plantea preguntas.
- Busca evidencia objetiva (documentación) y subjetiva (información proporcionada por el equipo).
- Reunión de apertura: se informa cómo será el proceso, indicando el lugar y la hora.

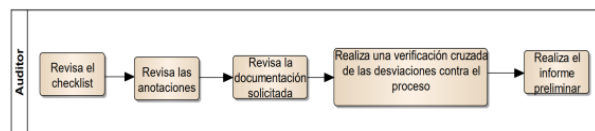


- Ejecución: se responden las preguntas, se muestra la evidencia y se completa el checklist. El auditor escucha lo que la gente dice que hace y luego ve la documentación (lo que debería estar haciéndose).

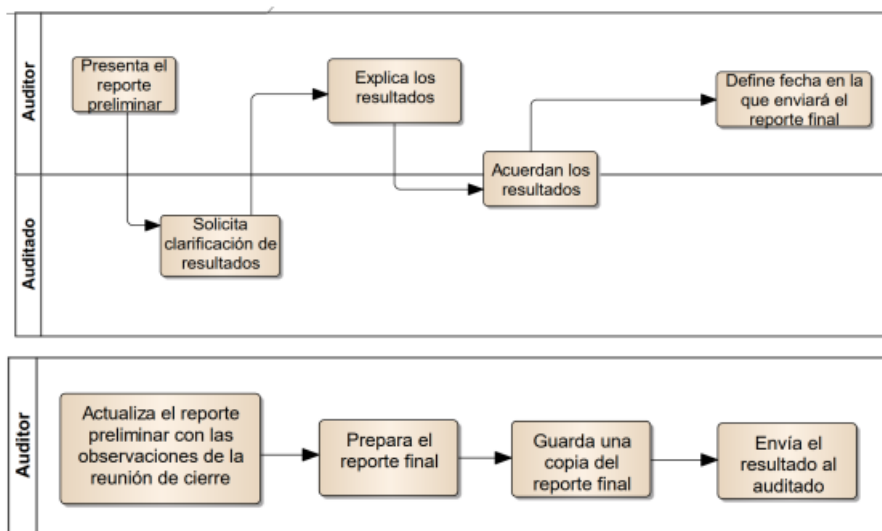


3. Análisis y Reporte de Resultados:

- Se analiza la documentación recopilada.

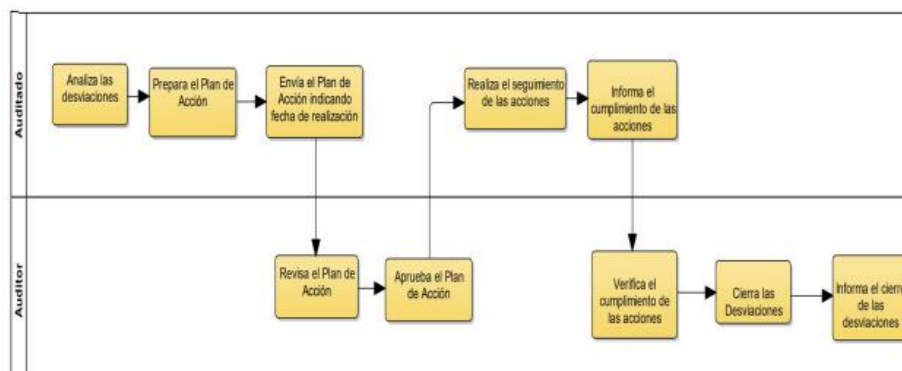


- Se prepara un informe de auditoría que se entrega al auditado.



4. Seguimiento:

- El auditor puede dar seguimiento a las desviaciones encontradas para asegurar que se resuelvan adecuadamente.



Además, se utiliza un checklist de auditoría con preguntas tipo para mantener un enfoque consistente en la auditoría:

- ✓ Fecha de la auditoría.
- ✓ Lista de auditados con identificación de sus roles.
- ✓ Nombre del auditor.
- ✓ Nombre del proyecto.
- ✓ Fase actual del proyecto (si es relevante para la auditoría).
- ✓ Objetivo y alcance de la auditoría.
- ✓ Lista de preguntas específicas a abordar durante la auditoría.

HERRAMIENTAS Y TÉCNICAS UTILIZADAS EN AUDITORÍAS

- **Checklists:** Contienen preguntas estándar para mantener el enfoque consistente en las auditorías. Establecen una base mínima, pero el auditor puede hacer preguntas adicionales según sea necesario.
- **Muestreo:** Implica seleccionar una muestra representativa de productos o procesos para auditar.
- **Revisión de registros:** Se verifica la documentación y registros relacionados
- **Herramientas automatizadas:** Se utilizan herramientas de software para facilitar la auditoría y el data análisis

LISTA DE RESULTADOS DE UNA AUDITORÍA – TIPOS DE RESULTADOS

- **Buenas Prácticas:** Descubrimientos que **superan las expectativas**, como prácticas, procedimientos o instrucciones excepcionales.
- **Desviaciones:** **Incumplimientos** con el proceso o los estándares que **requieren un plan de acción** por parte del auditado.
- **Observaciones:** Condiciones que **podrían mejorar**, pero **no necesariamente** requieren un plan de acción

REPORTE DE AUDITORÍA

En este documento se deben informar las siguientes desviaciones:

- ✓ Cualquier **desviación** que resulta en la **disconformidad** de un **producto** respecto de sus **requerimientos**
- ✓ Cualquier **desviación** al **proceso definido** o a los requerimientos documentados.

MÉTRICAS DE AUDITORÍA

1. **Esfuerzo por auditoría:** Mide el **tiempo y los recursos** dedicados, expresados en hs. o días de trabajo.
2. **Cantidad de desviaciones:** Registra el **número total de incumplimientos** identificados durante la auditoría.
3. **Duración de la auditoría:** Mide el **tiempo total** necesario para llevar a cabo la auditoría, desde la planificación hasta la presentación de resultados.
4. **Cantidad de desviaciones clasificadas por área de proceso (PA) de CMMI:** Desglosa las **desviaciones identificadas según las diferentes áreas** de proceso definidas por el modelo CMMI, permitiendo analizar fortalezas y debilidades específicas del software con relación a los estándares.

Estas métricas son útiles para **evaluar la eficiencia de las auditorías** y proporcionar **información valiosa para la mejora** continua. La elección de métricas dependerá de los objetivos específicos de la auditoría y los estándares aplicables en la organización.

CALIDAD DE PRODUCTO

Toda la introducción a Calidad de Producto está acá: [1](#)

La **planificación de pruebas** es **esencial** para **asegurar la calidad** del software. Involucra la identificación de los niveles y tipos de pruebas necesarios, así como la selección de técnicas y herramientas adecuadas. En otras palabras, es ver todo lo que necesitamos para probar el producto, que lo vamos a ver a continuación:

VERIFICACIÓN Y VALIDACIÓN

Las revisiones técnicas son un **proceso de ciclo de vida completo** que abarca desde la revisión de los requerimientos hasta las pruebas. Tienen **dos aspectos clave**:

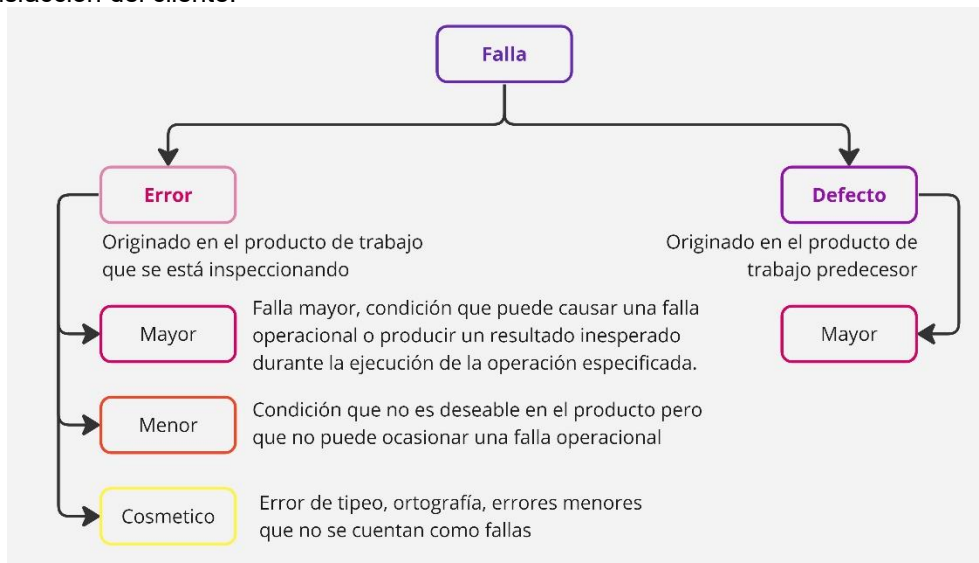
- **Validación:** Se centra en determinar **si estamos construyendo el producto correcto**, es decir, si cumple con los requerimientos y expectativas del cliente.
- **Verificación:** Se enfoca en comprobar **si estamos construyendo el producto correctamente**, siguiendo los estándares y procedimientos establecidos.

Es importante distinguir entre:

- **Falla:** Representa un **error en un producto de trabajo**, es decir, una salida defectuosa que no cumple con los estándares.

- **Producto de trabajo:** Se refiere a las **salidas de cualquier actividad dentro del ciclo de vida de desarrollo**, como documentos, código, diseños, etc.

Las fallas pueden surgir debido a diversos factores, incluyendo **problemas de comunicación** entre el equipo y **limitaciones de memoria**, entre otros. **Identificar y abordar las fallas** es **esencial** para garantizar la calidad del producto y la satisfacción del cliente.



1. Fallas Mayores:

- En el código: Errores lógicos, estructurales u otros que pueden ocasionar fallas operacionales.
- En el diseño: Expresiones en el diseño que, si se implementaran tal como están especificadas, podrían causar fallas operacionales.
- En requerimientos: Expresiones en los requerimientos que podrían llevar a que no se cumplan las necesidades del cliente o que sean ambiguas, lo que requeriría una investigación adicional.
- En el plan de prueba o casos de prueba: Condiciones que podrían impedir la detección de fallas en el programa o la ejecución adecuada de las pruebas.

2. Fallas Menores:

- En código o diseño: Violaciones a los estándares de codificación o diseño (por ejemplo, comentarios en el código) que no causarían fallas operacionales, pero pueden dificultar el mantenimiento.
- En requerimientos: Requerimientos que no se pueden probar.
- En el plan de prueba o casos de prueba: Información poco clara que podría requerir esfuerzo de prueba innecesario debido a la redundancia.

3. Notas Cosméticas:

- En documentación: Errores tipográficos, ortográficos y gramaticales, necesidad de actualizar documentos con plantillas más recientes o la historia de revisiones del documento.
- En código: Necesidad de actualizar los datos de copyright de un código fuente utilizado o sugerencias alternativas, como un algoritmo de búsqueda diferente.

PRINCIPIOS

- * La **prevención** es mejor que la cura
- * **Evitar** es más efectivo que eliminar
- * La **retroalimentación enseña** efectivamente
- * Priorizar lo **rentable**
- * **Olvidarse de la perfección**, no se puede conseguir
- * **Enseñar a pescar**, en lugar de dar el pescado

Existen **dos aproximaciones complementarias**:

1. **Revisiones Técnicas**
2. Pruebas de Software (Testing, es lo próximo, paciencia. Ahí es donde van los Niveles, entre otras cosas).

REVISIONES TÉCNICAS – PEER REVIEW

La revisión técnica es una **actividad realizada por un colega** con el **propósito de mejorar la calidad del software** mediante la **detección temprana de errores** en diversos artefactos, como código, requerimientos, diseño, entre otros. Es un **proceso estático de validación y verificación** que no corrige los errores, pero **ayuda a evitar retrabajos**.

El **objetivo principal** de las revisiones técnicas es **introducir el concepto de verificación y validación, fomentar la realización de un trabajo de mayor calidad y evitar que los errores pasen desapercibidos**.

Es importante destacar que, durante estas revisiones, **no se juzga al autor del artefacto**, solo **se evalúa el artefacto** en sí. La cultura de trabajo debe enfocarse en brindar apoyo en lugar de buscar culpables al encontrar errores. El trabajo técnico necesita ser revisado por la misma razón que los lápices necesitan gomas: **errar es humano**.

Las revisiones técnicas **pueden aplicarse en diversas etapas** del desarrollo y a cualquier representación legible del software.

Aunque **pueden ser difíciles de introducir** y parecen aumentar los costos al principio, a largo plazo, **pueden descubrir muchos errores, evaluar versiones incompletas y considerar otros atributos de calidad** del software. Sin embargo, **requieren tiempo y experiencia** para organizarse y **pueden ralentizar** el proceso de desarrollo inicialmente.

Las revisiones **SON:**

- La forma más **barata y efectiva** de encontrar fallas.
- Una forma de **proveer métricas** al proyecto.
- Una buena forma de **proveer conocimiento cruzado**.
- Una buena forma de **promover el trabajo en grupo**.
- Un método probado para **mejorar la calidad** del producto.

Las revisiones **NO SON:**

- Utilizadas para encontrar **soluciones** a las fallas.
- Usadas para obtener la **aprobación** de un producto de trabajo.
- Usadas para evaluar el **desempeño** de las personas.

TIPOS DE REVISIONES

- * **Formales:** tienen un proceso definido con roles.
 - Inspecciones (Inspección de código de Fagan e inspección de Gilb).
- * **Informales:** cuando no existe un proceso de cómo realizarlo.
 - Walkthrough o recorrido

MÉTODOS/TÉCNICAS DE REVISIONES

Walkthrough:

- Técnica **informal** de análisis **estático**.
- **Dirigida por un diseñador** o programador.
- Los participantes formulan preguntas y hacen comentarios.
- **Objetivos:** Mínima sobrecarga, capacitación de desarrolladores, rápido retorno.
- Proceso informal, **sin mediciones**, no es una revisión técnica formal (FTR).

Inspecciones:

- Actividad **formal** de garantía de calidad de software.
- Proceso **estructurado** con **roles definidos**.
- Utilización de **checklists**.
- **Objetivos:** **Detectar y eliminar** eficazmente todos los **defectos**, **verificar requerimientos y estándares**, **conseguir software uniforme y proyectos manejables**.
- Proceso **formal**, **mediciones**, **fase de verificación** estructurada.
- Son procesos **time boxing** y con un **alto esfuerzo intelectual**

TIPOS DE DOCUMENTOS Y REVISORES

- **Arquitectura o Diseño de alto nivel:**
 - Arquitecto, analista de requerimientos, diseñador, líder de proyecto, testers.
- **Diseño detallado:**
 - Diseñador, arquitecto, programadores, testers.
- **Planes de proyecto:**
 - Líder de proyecto, stakeholders, representante de ventas o marketing, líder técnico, representante del área de calidad.
- **Especificación de requerimientos:**
 - Analista de requerimientos, líder de proyecto, arquitecto, diseñador, testers, representante de ventas y/o marketing.
- **Código fuente:**
 - Programador, diseñador, testers, analista de requerimientos.
- **Plan de testing:**
 - Tester, programador, arquitecto, diseñador, representante del área de calidad, analista de requerimientos.

ROLES

1. **Autor:** Creador o responsable del producto a inspeccionar.
2. **Moderador:** Líder de la revisión que planifica y dirige la reunión de inspección.
3. **Anotador:** Encargado de registrar los hallazgos de la inspección y elaborar el informe.
4. **Lector:** Lee el producto a inspeccionar durante la reunión para mantener la concentración.
5. **Inspector:** Miembro del equipo que examina el producto previamente en busca de defectos.

ETAPAS

1. **Planificación:** El moderador planifica la inspección, definiendo roles, lugar, y duración de la reunión.
2. **Visión general (opcional):** El autor proporciona una descripción general del producto a inspeccionar.
3. **Preparación:** Cada participante adquiere una copia del producto y lo estudia en busca de defectos potenciales.
4. **Reunión de inspección:** El equipo se reúne para analizar el producto, compartir defectos encontrados y tomar decisiones sobre su aceptación.
5. **Corrección:** El autor realiza correcciones en el producto basadas en los defectos identificados durante la inspección.
6. **Seguimiento:** Dependiendo de la gravedad de los defectos, puede ser necesario un proceso de re-inspección o reuniones con el moderador para tratar las correcciones.

TESTING EN AMBIENTES ÁGILES

DEFINICIÓN

ASEGURAR LA CALIDAD VS CONTROLAR LA CALIDAD

Asegurar la calidad no es igual que controlar la calidad. Controlar la calidad es una parte de asegurar la calidad, y es esencial para lograr un producto de calidad. La calidad no puede añadirse al final del proceso; debe ser una parte integral de **todo el proceso de desarrollo**. Detectar errores temprano ahorra tiempo y recursos.

El testing es una parte importante del aseguramiento de calidad, pero no garantiza la calidad del software. La calidad se obtiene a lo largo de todo el proceso de desarrollo, y el control de calidad del proceso es esencial para garantizar un producto de calidad. La administración de configuración (**SCM**) es **fundamental** para permitir las actividades de aseguramiento de calidad. El control de calidad del proceso es crucial, ya que la calidad del producto depende en gran medida de la calidad del proceso de desarrollo que se utiliza. Existen diversos **estándares** (ISO, CMMI, etc.) que permiten **acreditar la calidad** del proceso, debido a que la calidad del producto no es algo estandarizable por la variación de requerimientos de un caso a otro.

Cabe destacar que la actividad de testing no asegura la calidad por sí solo, sino que debe estar acompañada de las demás actividades. QA != Testing



CONCEPTO DE TESTING

Testing es el proceso de **someter a software o componentes de software a condiciones específicas** para determinar su **validez** con respecto a los **requerimientos especificados**. Es una **actividad destructiva** orientada a **encontrar defectos asumidos en el código**. Busca asegurar que el código **se comporte según lo diseñado, sin sorpresas** para los usuarios. Forma **parte del QA (Aseguramiento de Calidad) y del control de calidad**, realizándose después de la construcción del producto a diferencia del QA que se aplica a lo largo del ciclo de vida del software.

El éxito en las actividades de testing **se mide por la detección de defectos** en la ejecución de casos de prueba. Un software con alta calidad dificulta encontrar defectos, resultando en un testing aparentemente no exitoso. Sin embargo, un software de baja calidad implica un alto costo de retrabajo al corregir errores entre actividades de testing y desarrollo. Dada la **naturaleza costosa del testing**, es **crucial lograr una cobertura óptima** considerando el costo-beneficio, aunque nunca se pueda alcanzar el 100% de cobertura. Los criterios de aceptación establecen la base para definir esta cobertura desde el inicio del proceso. En el desarrollo de software confiable, el testing puede representar del **30% al 50% del costo** total.

PRINCIPIOS DEL TESTING

Acá encontré algo distinto en cada resumen/libro, intenté juntar todo:

- 1. Evitar que los Programadores Prueben su Propio Código:**
 - ✓ Los desarrolladores tienden a no querer encontrar sus propios defectos, piensan que son sus hijos, lo que podría afectar la objetividad y exhaustividad del testing.
- 2. Evitar que una Unidad de Programación Pruebe sus Propios Desarrollos:**
 - ✓ La misma lógica se aplica a unidades de programación, evitando que prueben sus propios desarrollos para garantizar una evaluación más imparcial.
- 3. Examina lo que Debe y No Debe Hacer el Software:**
 - ✓ El testing implica verificar tanto si el software cumple con lo planeado como si evita realizar acciones no deseadas.
- 4. No Asumir que no se Encontrarán Defectos al Planificar el Esfuerzo de Testing:**
 - ✓ Se debe partir de la premisa de que siempre existirán defectos que necesitan ser encontrados y corregidos.
- 5. Testing como Tarea Creativa e Intelectualmente Desafiante:**
 - ✓ El testing requiere habilidades creativas e intelectuales para abordar eficazmente la detección de defectos.
- 6. Testing Temprano durante la Elaboración de Requerimientos:**
 - ✓ Iniciar el testing temprano, al elaborar los requerimientos, asegura que estos sean testeables, legibles y libres de errores antes de llegar a desarrollo.
- 7. Actualización y Rediseño de Casos de Prueba:**
 - ✓ Los casos de prueba deben actualizarse y rediseñarse para evitar la paradoja del pesticida, asegurando su efectividad continua.
- 8. Testing Dependiente del Contexto:**
 - ✓ La estrategia de testing debe adaptarse al contexto, evaluando métodos, costos, riesgos y recursos, utilizando los riesgos para priorizar casos de prueba.
- 9. Reconocimiento de la Inevitabilidad de Errores:**
 - ✓ El testing es esencial porque la presencia de errores en el software es inevitable.
- 10. Principios del Testing (Principio Explicativo):**
 - ✓ Estos principios subrayan que el testing es destructivo, busca defectos, y su éxito radica en la detección de estos. Aunque no garantiza calidad, es necesario y puede iniciarse antes de la codificación.
- 11. Principios del Testing (Costo y Calidad):**
 - ✓ El costo del testing es significativo, pero su objetivo es poner de manifiesto defectos, no agregar calidad. Un desarrollo exitoso no garantiza un testing exitoso.
- 12. Principios del Testing (Agrupamiento de Defectos):**
 - ✓ Los defectos suelen agruparse en un conjunto limitado de funcionalidades. La regla de Pareto sugiere enfocar el esfuerzo de testing en estas áreas prioritarias.

MITOS DEL TESTING

- ✗ "El testing es el proceso para demostrar que los errores no están presentes."
- ✗ "El propósito del testing es demostrar que un programa realiza sus funciones previstas de forma correcta."
- ✗ "El testing es el proceso que demuestra que un programa hace lo que se supone que debe hacer."

El objetivo del testing es **agregar valor al producto al revelar su calidad y brindar confianza** en el software al encontrar y eliminar defectos en el código. El testing se inicia con la suposición de que el software contiene defectos y se busca encontrar la mayor cantidad de ellos posible. Además, un programa puede contener defectos incluso si realiza lo que se supone que debe hacer, ya que los errores también pueden estar presentes cuando el software hace algo que no debería hacer.

HASTA DÓNDE TESTEAR – ¿CUÁNTO TESTING ES SUFICIENTE?

El nivel de testing necesario **depende** de factores como el **riesgo** y el **costo** del proyecto. **No es posible realizar un testing exhaustivo** debido al tiempo requerido. Los criterios de aceptación, que pueden definirse en términos de **costos**, **porcentaje de pruebas exitosas**, **severidad de defectos**, **cobertura de pruebas**, entre otros, ayudan a determinar cuándo se ha completado una fase de testing. Estos criterios son utilizados para negociar con los interesados, como el Product Owner en enfoques ágiles o el Líder del Proyecto en enfoques tradicionales, cuántos defectos son aceptables antes de finalizar el proceso de testing.

CONCEPTOS IMPORTANTES

La construcción del software y el testing requieren mentalidades diferentes. Los desarrolladores crean el software, mientras que los testers buscan defectos. Esta distinción puede evitar conflictos dentro de los equipos, ya que la búsqueda de fallas no debe ser vista como una crítica al producto o su autor.

DEFECTOS VS ERRORES

Los errores y defectos en el software se diferencian en el **momento** en que se identifican. Un **error se descubre en la misma etapa** en la que se originó, mientras que un **defecto se encuentra en una etapa posterior**, lo que lo convierte en un problema mayor.

Cuando se trata de **defectos** encontrados durante el **testing**, es esencial considerar la **severidad y la prioridad**. La **severidad** indica la **gravedad del defecto** y se relaciona con su **impacto técnico**, y generalmente se clasifica en cinco niveles (**Bloqueante, Crítico, Mayor, Menor, Cosmética**). La **prioridad**, por otro lado, refleja la **urgencia del cliente o el negocio** para resolver el defecto y se divide en cuatro categorías: **urgencia, alta, media y baja**. La relación entre la severidad y la prioridad puede variar según el contexto del proyecto y el tipo de sistema en desarrollo.

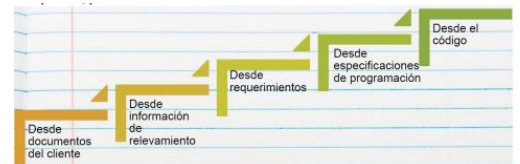
CASOS DE PRUEBA

Los casos de prueba son **conjuntos de condiciones o variables** que permiten a los **testers determinar si el software funciona correctamente**. Estos casos de prueba incluyen **acciones específicas** que se realizan en una funcionalidad particular, **junto con los datos** necesarios para ejecutarlas. Una buena definición de casos de prueba facilita la **reproducción de defectos**, lo que es crucial para su identificación y solución.



El desafío radica en **definir una cantidad mínima de casos de prueba** que ofrezca una cobertura efectiva de las funcionalidades, ya que probar exhaustivamente todas las opciones es costoso e inviable. Por lo tanto, se utilizan **estrategias** de prueba que **se centran en aspectos críticos y valores límite** que son más propensos a errores. Los casos de prueba se **derivan de la documentación** disponible, y su estructura general consta de un **objetivo** (característica a probar), **datos de entrada** y **entorno** (condiciones iniciales) y el **comportamiento esperado** (salida o acción de acuerdo con los requerimientos).

La **derivación de casos de prueba** se basa en la **información disponible, principalmente en los requerimientos**. Si los requerimientos son vagos o poco detallados, es necesario especificarlos para poder crear casos de prueba efectivos. En algunos casos, los casos de prueba pueden servir como una forma de especificar requerimientos más detallados. Es importante recordar que ninguna técnica de generación de casos de prueba es completa por sí sola, y es fundamental **combinar diversas técnicas** para abordar distintos problemas y mejorar la especificación.



Las **condiciones de prueba** son las **respuestas anticipadas** de un sistema **ante estímulos** específicos, que consisten en diversas entradas. **Cada condición de prueba debe ser cubierta por al menos un caso de prueba**. Estas condiciones se establecen antes de definir los casos de prueba y proporcionan el contexto necesario para crearlos de manera efectiva.

CICLOS DE PRUEBA

Un ciclo de pruebas implica **ejecutar todos los casos de prueba** en una **misma versión** del sistema a probar, **identificar los defectos, corregirlos y repetir el proceso** hasta que se cumpla con los criterios de aceptación acordados con el cliente. Por lo general, se llevan a cabo dos ciclos de pruebas, siendo el primero el "ciclo 0," que siempre es manual y se enfoca en la configuración, mientras que el "ciclo 1" y siguientes pueden incluir pruebas automatizadas.

La **regresión** es un proceso crucial **al final de cada ciclo** de pruebas, donde se verifica toda la nueva versión del sistema para prevenir la introducción de nuevos defectos al intentar solucionar los defectos detectados. La regresión **asegura** que la corrección de defectos **no genere nuevos problemas**. En este contexto, hay dos enfoques posibles para abordar el ciclo de pruebas:



NIVELES DE PRUEBA

Uno de los aspectos importantes a la hora de **definir las pruebas** que vamos a hacer es **identificar un esquema o manera** de hacer testing que tiene que ver con los niveles de prueba. Como lo indica su nombre, se trata de **subir escalones o niveles**, e ir abarcando más y más cosas para probar a medida que vamos subiendo. Los niveles de prueba **determinan el foco o la granularidad** de la prueba que se está por ejecutar. En general, primero **se comienzan** probando **componentes pequeños**, y luego se integran en pruebas de mayor granularidad. Esto es al revés de cómo se desarrollan los componentes.

Pruebas Unitarias

Estas pruebas se realizan en **componentes individuales** de manera independiente. Se centran en **aspectos puntuales** y aislados del proyecto y normalmente son ejecutadas por el mismo **desarrollador**. Encuentran **errores** más que defectos y son **fáciles de automatizar**. **Forman parte de la Definición de Terminado (DoD)**.

Pruebas de Integración

Verifican el funcionamiento de **dos o más componentes** que se **integran**. Se ejecutan sobre **builds** y requieren la participación de un **tester**. La integración **puede ser incremental**, desde **lo más general a lo más específico** (top-down) o **viceversa** (bottom-up).

Pruebas de Sistema

Comprueban el funcionamiento del **sistema en su totalidad**, incluyendo aspectos como la **recuperación de fallas**, **seguridad**, **rendimiento** y otros **aspectos no funcionales**. El entorno de prueba debe **reflejar el entorno de producción**.

Pruebas de Aceptación

Realizadas por el **usuario** para determinar si la aplicación se adapta a sus **necesidades**. El objetivo es establecer la **confianza en el sistema y sus características**, **no necesariamente encontrar defectos**. Pueden incluir pruebas alfa en laboratorio y pruebas beta en ambientes de trabajo reales. Se realizan en el despliegue del sistema e involucran al usuario, el gerente de testing, el líder del proyecto y otros empleados con roles funcionales.

AMBIENTES

Los ambientes son los **lugares en donde se trabaja** para el desarrollo de software. Los ambientes para la construcción del software se refieren a los diferentes **entornos** utilizados en el ciclo de vida del desarrollo de software.

Ambiente de Desarrollo

Aquí los **desarrolladores crean, prueban y depuran** el software, utilizando **herramientas** de desarrollo, depuración y pruebas. Es un **entorno local** o en red que les permite **experimentar y probar** sin afectar la producción. Las pruebas **unitarias** se realizan en este ambiente.

Ambiente de Prueba

Es donde se llevan a cabo **pruebas exhaustivas** del software, incluyendo pruebas de integración, funcionales, de rendimiento y otras. **Se asemeja al entorno de producción**, pero sin todas las características, y utiliza conjuntos de datos similares. Las **pruebas de integración** se realizan aquí.

Ambiente de Pre-Producción

También conocido como **entorno de calidad**, es donde se realiza una **prueba final antes del lanzamiento** en producción. Aquí se **simulan las condiciones de producción** y se realizan pruebas de estrés, carga y seguridad. Las **pruebas de sistema** se llevan a cabo en este ambiente.

Ambiente de Producción

Es el **entorno real** en el que el software está en funcionamiento y accesible para los **usuarios finales**. Es altamente **controlado, escalable, seguro y confiable**, con **medidas de respaldo** y recuperación ante desastres. Las **pruebas de aceptación** se realizan en el ambiente de preproducción o en uno similar al de producción (como el ambiente de prueba).

PROCESO DE PRUEBAS

¿Cuántas líneas de código necesito para empezar a hacer testing? **Cero**.

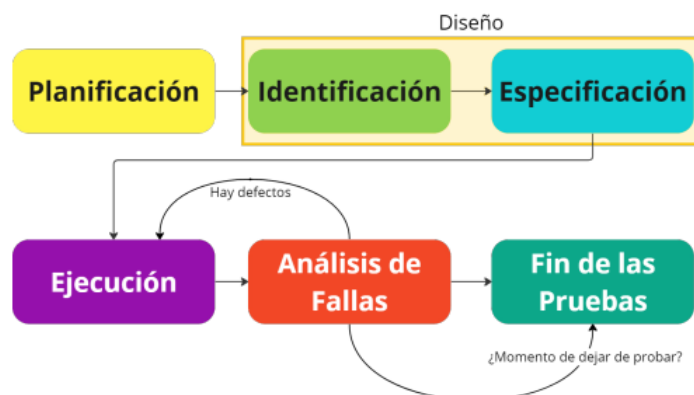
El proceso (definido) de Testing sigue una estructura general que comprende **varias etapas**:

1. **Planificación:** En esta etapa se **definen los objetivos del cliente**, se identifican los **riesgos** y se crea un **plan de pruebas** que incluye **estrategias, recursos y criterios de aceptación**. También se **revisan los resultados** del testing y se toman decisiones.
2. **Diseño de Casos de Prueba:** Se **identifican y especifican los casos de prueba**, **priorizándolos** y definiendo **cómo** se llevarán a cabo, **quién** los ejecutará y **cuándo**. Se analiza la **testabilidad** de los requisitos. Se define si se usa regresión o no.
3. **Ejecución de Pruebas:** En esta etapa, **se ejecutan los casos de prueba**, se **registran** los resultados y se **comparan** con los resultados esperados. Se busca **automatizar el proceso** de ejecución para ahorrar tiempo y costos.



4. **Análisis de Fallas y Reporte:** Se realiza un **seguimiento de la corrección** de los **defectos** encontrados. Se evalúa el criterio de aceptación, se **reportan los resultados** de las pruebas a los interesados y se analizan las lecciones aprendidas. Aquí se confecciona el informe de reportes.
5. **Fin de las Pruebas:** Se concluye el proceso de pruebas cuando se han **corregido** los defectos **bloqueantes, críticos y mayores**, y los **defectos menores** y cosméticos **son limitados**. En empresas maduras, se puede confeccionar un informe final.

ARTEFACTOS DE TESTING



Plan de Pruebas

En este documento **se definen** los **datos** necesarios para las pruebas, los **recursos**, las **herramientas** a utilizar y **si se aplicará la regresión**. Se elabora a **partir de los requerimientos** del proyecto.

Casos de Prueba

Estos son conjuntos de **pasos detallados** que **describen las acciones a realizar** para lograr un resultado específico y esperado. Incluyen **condiciones iniciales** y **datos necesarios para ejecutar el escenario** de prueba. Son **esenciales para descubrir defectos** y deben ser **reproducibles**. Mientras no cambien los requerimientos, estos casos pueden ser utilizados las veces que sea necesario.

Reporte de Incidentes o Defectos

Después de ejecutar los casos de prueba, se genera un informe que identifica los casos que pasaron y aquellos en los que se encontraron defectos. **Proporciona detalles sobre los defectos** encontrados, **cómo reproducirlos** y **se utiliza para la corrección** por parte de los desarrolladores.

Informe Final

Este informe **contiene métricas** y **datos finales** sobre el incremento del producto, como la **cantidad de ciclos de prueba**, la **cantidad de errores por ciclo**, los **métodos** y **tipos de pruebas** utilizados, entre otros. Puede ser un artefacto negociable en algunas organizaciones y se acuerda durante la contratación del trabajo.



EL TESTING EN EL CICLO DE VIDA DEL SOFTWARE

En el desarrollo de software, el enfoque de pruebas varía según el ciclo de vida del proyecto:

- **Modelo en Cascada:** En este enfoque, las pruebas se realizan al final del ciclo de desarrollo, ya que el producto se entrega en su totalidad al final del proceso. Esto puede llevar a la identificación tardía de problemas y aumentar los costos de corrección.
- **Modelo Ágil:** En un enfoque ágil, las pruebas se realizan de manera continua a lo largo de las iteraciones del desarrollo. Se busca identificar problemas temprano y asegurarse de que el software cumpla con los requisitos a medida que se construye. Sin embargo, la integración de incrementos puede introducir errores, por lo que se requiere una gestión cuidadosa.

El "**Modelo en V**" se utiliza para determinar **cuándo se deben realizar ciertas actividades** de prueba en función de la etapa de desarrollo del software. **Las pruebas se realizan en un orden inverso al proceso de desarrollo**, es decir, se desarrolla desde componentes de granularidad gruesa, como son los requerimientos abstraídos de detalles de implementación, hacia componentes de menor granularidad, como son clases o módulos, dependiendo del paradigma. En cambio, para las pruebas, la granularidad se da en sentido inverso.

En este contexto, dos conceptos clave son la **verificación**, que se enfoca en **construir el sistema correctamente** y sin defectos, y la **validación**, que se enfoca en **construir el sistema correcto**, es decir, uno que cumple con los requerimientos del cliente. El objetivo es asegurarse de que el software sea de alta calidad y cumpla con su propósito.

MÉTODOS/ESTRATEGIAS DE PRUEBA

¿Para qué usarlos? El tiempo y el presupuesto es limitado.

Hay que **pasar por la mayor cantidad de funcionalidades con la menor cantidad de pruebas**

El objetivo es abarcar la **mayor cantidad de pruebas con el menor esfuerzo**. Las siguientes dos estrategias, que son complementarias, buscan justamente eso, con presupuesto y tiempo limitado intentan abarcar en las pruebas la mayor cantidad de software posible.

Las estrategias de prueba son esenciales para cubrir funcionalidades de software de manera eficiente, dadas limitaciones de tiempo y presupuesto.

Antes de pasar a los enfoques, es importante mencionar la métrica que utiliza el testing:

COMPLEJIDAD CICLOMÁTICA

La complejidad ciclomática es una **métrica cuantitativa** que evalúa la **complejidad de un programa**, ofreciendo una **medida numérica de la cantidad de caminos independientes en su conjunto básico**. En el contexto del testing, proporciona **un límite inferior para la cantidad de casos necesarios** para ejecutar todas las instrucciones al menos una vez. Representa programas como grafos, donde cada instrucción se convierte en un nodo.

- **Fórmula de Cálculo:** $M = G - N + 2 * P$ (Complejidad ciclomática), donde G es el número de regiones cerradas y P es el número de nodos de salida.
- **Puntos de McCabe:** Asigna niveles de riesgo basados en la complejidad ciclomática:
 - 01-10: Programa simple, bajo riesgo.
 - 11-20: Complejidad moderada, riesgo moderado.
 - 21-50: Complejidad alta, riesgo alto.
 - +50: Muy alta complejidad, riesgo muy alto.

Tenemos dos enfoques principales, Caja Negra y Caja Blanca, ofrecen fortalezas específicas. Se sugiere combinar métodos para lograr un testing efectivo. (Acá no voy a dar tanto detalle porque en los parciales va al práctico y si vas al final, hace memoria del práctico (? y 🍷))

CAJA NEGRA

Se evalúa el software sin conocer su estructura interna, centrándose en entradas y salidas.

- **Métodos:**
 - **Basados en Especificaciones:** Utilizan la documentación del producto.
 - Partición de Equivalencias: Identifica clases de datos con resultados equivalentes.
 - Análisis de Valores Límite: Examina límites de clases de datos para diseñar casos de prueba.
 - **Basados en Experiencia:** Se apoyan en la intuición y experiencia del tester.
 - Adivinanza de Defectos: Identifica posibles defectos basándose en intuición y experiencia.
 - Testing Exploratorio: Genera nuevas pruebas durante la exploración del software.

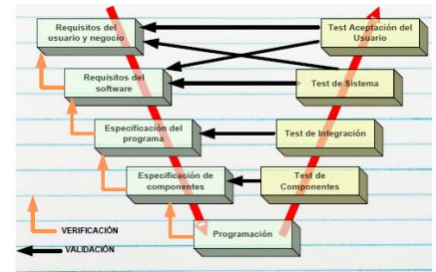
Partición de Equivalencias

- **Objetivo:** Analizar condiciones externas, dividiéndolas en clases de equivalencia para evaluar funcionalidades.
- **Procedimiento:**
 1. Identificar condiciones de entrada/salida.
 2. Dividir condiciones en subconjuntos de valores equivalentes.
 3. Crear casos de prueba con un valor de cada subconjunto.
- **Consideraciones:** Prioridad, nombre del caso, precondiciones, pasos y resultados esperados.

CAJA BLANCA

Se tiene acceso al código, permitiendo diseñar casos de prueba para maximizar la detección de defectos.

- **Coberturas:**
 - **Enunciados o Caminos Básicos:** Busca caminos independientes de la funcionalidad.
 - **Sentencias:** Evalúa todas las instrucciones del código.
 - **Decisión:** Examina todas las ramas de decisiones.
 - **Condición:** Evalúa cada condición en valores verdaderos y falsos.
 - **Decisión-Condición:** Evalúa ramas y condiciones simultáneamente.
 - **Múltiple:** Evalúa todas las combinaciones posibles de condiciones.



CONSIDERACIONES GENERALES

- 👉 La elección de la estrategia depende de la naturaleza del software y sus requerimientos.
- 👉 Combinar estrategias mejora la cobertura y eficacia del testing.
- 👉 La heurística, como la complejidad ciclomática, ayuda a determinar la cantidad mínima de casos de prueba.
- 👉 La flexibilidad y adaptabilidad son clave para ajustarse a diferentes contextos de testing.

TIPOS DE PRUEBA

SMOKE TEST

Es la **primera ejecución de pruebas** de sistema que se realiza para **verificar que el software o la aplicación funcionan** correctamente en un nivel muy básico. El objetivo principal del "Smoke Test" es asegurarse de que las **funcionalidades esenciales** de una aplicación **no provocan una falla catastrófica**. Se centra en probar las funciones más críticas antes de llevar a cabo pruebas más exhaustivas. Si el "Smoke Test" falla, es una señal de que hay problemas fundamentales en el software que deben resolverse antes de continuar con pruebas más detalladas.

SANITY TEST

Se realiza **después de una ronda de pruebas más exhaustivas** o después de aplicar correcciones a problemas críticos identificados previamente. El objetivo principal del "Sanity Test" es **verificar si las correcciones realizadas han solucionado los errores críticos sin introducir nuevos problemas**. En resumen, es una prueba para verificar la "cordura" del software después de las correcciones.

TESTING FUNCIONAL

Estas pruebas se centran en verificar que el software se **comporte de acuerdo con lo que se ha especificado** en la documentación o en los acuerdos con los stakeholders. Se basan en **requerimientos funcionales** y se **centran en responder** la pregunta **"¿Qué hace el sistema?"**. Pueden ser **basadas en requerimientos**, donde se prueban requisitos específicos, o **basadas en procesos de negocio**, donde se prueba un proceso de negocio completo.

TESTING NO FUNCIONAL

Estas pruebas se centran en **aspectos** que **no están directamente relacionados** con las **funciones específicas** del software, sino en **cómo lo hace**. Se centran en características como el **rendimiento**, la **seguridad**, la **usabilidad**, la **escalabilidad** y otros aspectos que no son funciones específicas, sino más bien cualidades del software. Las pruebas no funcionales responden a la pregunta **"¿Cómo lo hace el sistema?"**. La naturaleza de las pruebas no funcionales varía según las necesidades del proyecto y las expectativas de los usuarios. Incluye varios **tipos de prueba** como:

- 👉 **Performance:** Se ve el tiempo de respuesta (escenario esperado respecto a los tiempos de respuesta) y la concurrencia. Deben pasar esta prueba sí o sí.
- 👉 **Carga:** mira el comportamiento de los dispositivos de hardware y las comunicaciones.
- 👉 **Stress:** queremos forzar al sistema para que falle, se lo somete a condiciones más allá de las normales. Se ve el tiempo que hay que esperar para probar nuevamente el sistema y la robustez del sistema.
- 👉 **Mantenimiento:** para ver si el producto está en condiciones de evolucionar, se controla que haya documentación, manual de configuración, etc. Se observa la facilidad que existe para corregir un defecto.
- 👉 **Usabilidad:** que sea cómodo para el usuario.
- 👉 **Portabilidad:** se prueba en los distintos entornos acordados con el cliente.
- 👉 **Fiabilidad:** probamos que podemos depender del sistema. Seguridad física del software.
- 👉 **De interfaz de usuario:** suelen ser más complejas las GUI's que las interfaces de comandos
- 👉 **De configuración.**

TDD – TEST DRIVEN DEVELOPMENT

El TDD, que significa "Test Driven Development" o "Desarrollo Conducido por Pruebas", es una técnica que implica **escribir pruebas antes de desarrollar el código** de una aplicación. Esta técnica se enfoca en el **desarrollo funcional** y consta de dos prácticas principales:

1. **Test First Development (Desarrollo Primero de las Pruebas):** En esta técnica, se comienza escribiendo las pruebas, generalmente pruebas unitarias, antes de escribir el código real. El objetivo es definir primero lo que se espera que haga el código y luego escribir el código para que pase esas pruebas con éxito.
2. **Refactoring (Refactorización):** Después de que se ha escrito el código y se han superado las pruebas, se lleva a cabo la refactorización, que implica reestructurar el código existente sin cambiar su comportamiento externo. El propósito del refactoring es mejorar los aspectos no funcionales del software, como la claridad y la reducción de la complejidad del código, para que sea más fácil de mantener.

El proceso del TDD **se repite en ciclos**, donde se escriben pruebas, se desarrolla el código correspondiente para pasar esas pruebas, se realiza la refactorización y luego se escriben nuevas pruebas. Este enfoque cíclico se repite hasta que se complete el desarrollo de la aplicación.

ATDD - ACCEPTANCE TEST-DRIVEN DEVELOPMENT

El **Desarrollo Dirigido por Pruebas de Aceptación** (ATDD, por sus siglas en inglés: Acceptance Test-Driven Development) es una práctica de desarrollo de software que se centra en la **colaboración estrecha entre desarrolladores, testers y stakeholders** del proyecto, con el objetivo de **crear software que cumpla con los requisitos del usuario desde el inicio** del proceso de desarrollo. Los requisitos se expresan en forma de pruebas automatizadas que verifican que el software cumple con esas especificaciones.

Proceso:

1. **Definición de Requisitos:**
 - Los requisitos se definen en términos de comportamiento esperado y funcionalidades deseadas.
2. **Creación de Pruebas de Aceptación:**
 - Se crean pruebas de aceptación que representan casos concretos que deben aprobarse para que el software sea considerado aceptable.
3. **Automatización de Pruebas:**
 - Las pruebas de aceptación se automatizan para que puedan ejecutarse repetidamente y verificar que el software sigue cumpliendo con los requisitos.
4. **Desarrollo Incremental:**
 - Los desarrolladores trabajan en incrementos pequeños de código, implementando funcionalidades que hacen pasar las pruebas de aceptación.
5. **Iteración Continua:**
 - Se repite el ciclo, ajustando las pruebas de aceptación según sea necesario y desarrollando nuevas funcionalidades.

Beneficios:

1. Entendimiento Compartido:
2. Detección Temprana de Problemas:
3. Documentación Viva:
4. Feedback Rápido:
5. Entregas Confiables:

A partir de acá voy a citar los temas incluidos en los posters científicos, salvo los que ya se desarrollaron anteriormente.

MANIFIESTO DE TESTING

Siguiendo la filosofía del desarrollo ágil, Samantha Laing y Karen Greaves propusieron el Manifiesto para el Testing Ágil, que destaca los valores esenciales para llevar a cabo prácticas de testing efectivas dentro de un entorno ágil. En su libro "A Coach's Guide to Agile Testing", presentan los siguientes valores:

1. **Testing durante por sobre testing al final:**
 - Prioriza la integración continua del testing a lo largo de todo el ciclo de desarrollo en lugar de dejarlo para las etapas finales, fomentando la detección temprana de posibles problemas.
2. **Prevenir bugs por sobre encontrar bugs:**
 - Enfatiza la importancia de implementar prácticas proactivas para prevenir la introducción de errores en lugar de simplemente detectarlos después de que ocurran.
3. **Entender lo que se está testeando por sobre verificar la funcionalidad:**
 - Resalta la necesidad de comprender en profundidad el contexto y los requerimientos del software que se está probando, más allá de verificar simplemente la funcionalidad superficial.
4. **Construir el mejor sistema por sobre romper el sistema:**
 - Aboga por adoptar una mentalidad constructiva, enfocándose en construir un sistema de alta calidad en lugar de centrarse únicamente en encontrar defectos o romper el sistema.
5. **El equipo es responsable de la calidad por sobre el tester es responsable de la calidad:**
 - Establece la responsabilidad colectiva de todo el equipo en la garantía de calidad, desplazando la idea de que solo los testers son responsables de la calidad del software.

TESTING TRADICIONAL VS. TESTING ÁGIL

Ambos enfoques, tradicional y ágil, son **métodos efectivos** con ventajas para usuarios y testers. La elección depende de los requisitos específicos. Las pruebas tradicionales son intensivas en tiempo y esfuerzo, mientras que las ágiles permiten entregas rápidas y se benefician de herramientas automatizadas.

	Testing tradicional	Testing Ágil
Definición	El testing tradicional, a menudo conocido como modelo en cascada, sigue un enfoque secuencial y lineal en el desarrollo de software. Cada fase, como el diseño, la implementación, el testing y la implementación, se realiza de manera independiente y en secuencia.	Las pruebas ágiles, alineadas con la metodología ágil de desarrollo de software, implican un enfoque colaborativo en el que el equipo trabaja conjuntamente para identificar defectos, validar la efectividad, calidad, rendimiento y confiabilidad del software.
Características	1. Secuencial: Las fases del desarrollo y el testing se llevan a cabo de manera secuencial. 2. Documentación extensiva: Se produce una cantidad significativa de documentación antes de pasar a la siguiente fase. 3. Enfocado en la corrección final: El testing se realiza principalmente al final del desarrollo, después de que se ha completado la implementación. 4. Dirigido por el gerente del proyecto.	1. Modelo iterativo, adaptable a cambios en los requisitos. 2. Requiere documentación mínima. 3. Colaboración directa con los usuarios finales. 4. Uso de herramientas automatizadas. 5. Obtención de feedback del equipo y usuarios finales al completar cada ciclo de prueba, garantizando un proceso de validación continuo.
Ventajas	1. Estructura clara: El modelo en cascada ofrece una estructura clara y fácil de entender. 2. Documentación detallada: La extensa documentación facilita la comprensión del sistema. 3. Ofrece una cobertura exhaustiva del software.	1. Permite ajustarse a requisitos cambiantes. 2. Reduce la carga de documentación y automatiza las pruebas 3. Asegura la entrega rápida del producto con calidad óptima 4. Retroalimentación continua del equipo y del usuario
Desventajas	1. Rigidez: La falta de flexibilidad para realizar cambios durante el desarrollo. 2. Detección tardía de errores: Los errores pueden no detectarse hasta las etapas finales del desarrollo, lo que puede aumentar los costos de corrección. 3. La interacción entre los testers es mínima o nula.	1. Presenta un desafío en la gestión de proyectos extensos. 2. Falta documentación detallada que puede dificultar el mantenimiento. 3. No es apto para todo tipo de proyectos. 4. Es más difícil predecir cuándo se va a completar el proyecto.

PRÁCTICAS DE TESTING ÁGIL

1. Desarrollo Dirigido por Pruebas (TDD):

- **Definición:** Enfoque al diseño de programas donde se entrelazan el desarrollo de pruebas y código.
- **Proceso:** El programador escribe pruebas para funcionalidades, desarrolla código que pase las pruebas y avanza incrementalmente.
- **Beneficios:** Desarrollo incremental, detección temprana de errores.

2. Desarrollo Dirigido por Pruebas de Aceptación (ATDD):

- **Objetivo:** Colaboración para crear entendimiento compartido de los requisitos.
- **Proceso:** Ciclo similar a TDD, creando pruebas de aceptación antes de desarrollar el código.
- **Enfoque:** Transformar especificaciones en casos de uso para aceptación automatizados.

3. Pruebas Exploratorias:

- **Características:** Estrategia de pruebas como aprendizaje, diseño y ejecución simultáneos.
- **Enfoque:** Detección de defectos difíciles de cubrir con otras pruebas.
- **Importancia:** Adopción de integración continua y satisfacción de demanda del mercado.

4. Pruebas de Unidad e Integración Automatizadas:

- **Automatización:** Ejecución mediante máquinas o frameworks con scripts de prueba predefinidos.
- **Pruebas de Unidad:** Verifican el funcionamiento de una sola unidad de código.
- **Pruebas de Integración:** Comprueban el funcionamiento conjunto de dos o más componentes.

5. Control de Versión de Pruebas con el Código:

- **Herramienta:** Permite rastrear y gestionar cambios en el código fuente del proyecto.

- **Beneficios:** Colaboración, reversión de cambios, ramificaciones, seguimiento de cambios e integración continua.

6. Pruebas de Regresión:

- **Definición:** Proceso de probar un producto después de introducir nuevos cambios.
- **Importancia:** Confirmar que cambios recientes no afecten funciones existentes adversamente.
- **Método:** Volver a ejecutar casos de prueba para confirmar el correcto funcionamiento de la aplicación.

DESAFÍOS CULTURALES EN LA TRANSICIÓN DE EQUIPOS DE PRUEBA TRADICIONALES A ÁGILES

En la transición de metodologías tradicionales a enfoques ágiles en equipos de prueba, se enfrentan a desafíos culturales como falta de experiencia, desacuerdo con valores ágiles, falta de apoyo gerencial y presión externa. La falta de abordaje de estos desafíos es un factor determinante del fracaso en la adopción ágil. La pérdida de identidad de los testers y la resistencia al desarrollo sostenible son desafíos comunes. Se destaca la importancia de gestionar cambios no solo técnicos, sino también culturales, enfocándose en aspectos organizativos para lograr una adopción exitosa de frameworks ágiles. Se sugieren prácticas como usar retrospectivas, celebrar éxitos y contratar especialistas en pruebas ágiles para facilitar la integración cultural de los equipos.

CUADRANTES DEL TESTING

Los Cuadrantes del Testing son una **herramienta esencial** para la **planificación y organización** de las actividades de prueba en entornos ágiles, garantizando una **cobertura integral** de la calidad del software. Se dividen en **cuatro categorías**:

- **Cuadrante 1 (Tecnología y Equipo):** Incluye pruebas unitarias y de componentes, automatizadas y escritas en el mismo lenguaje que la aplicación.
- **Cuadrante 2 (Negocio y Equipo):** Engloba pruebas funcionales, de aceptación, de integración, de regresión y de interfaz de usuario, verificando que el software cumple con los requisitos del cliente.
- **Cuadrante 3 (Negocio y Producto):** Comprende pruebas exploratorias, de usabilidad, de experiencia de usuario, entre otras, evaluando la calidad externa del producto.
- **Cuadrante 4 (Tecnología y Producto):** Involucra pruebas no funcionales, como rendimiento, seguridad y fiabilidad, para evaluar la calidad interna del producto.

En conclusión, los Cuadrantes del Testing son una **guía valiosa** que fomenta la **colaboración**, **facilita la selección adecuada de herramientas** y **se adapta al contexto** de proyectos específicos. Aunque **no son reglas estrictas**, es esencial considerar las limitaciones, como la falta de información actualizada y la dificultad para comparar resultados.



PIRÁMIDE DEL TESTING

La **Pirámide de Automatización** de Pruebas Funcionales de Mike Cohn propone un **enfoque inverso** al "anti-patrón de cono de helado" común en el **desarrollo de software no ágil**. Se destaca por su **orientación a la prevención de errores** en lugar de encontrarlos. La pirámide consta de tres niveles:

1. Nivel Inicial: Pruebas Unitarias

- Enfocado en ejecutar **pruebas unitarias después de cada compilación**.
- **Identifica y elimina errores** de manera **rápida**, con una vida útil corta.
- **Proporciona feedback constante y sostenido**.

2. Nivel Intermedio: API/Integración/Pruebas de Componentes

- Se ejecutan **pruebas de integración** para garantizar el **correcto funcionamiento de los componentes juntos**.
- **Recomienda la automatización** en este nivel para un **mantenimiento más sencillo** y una **ejecución más rápida**.
- Permite **probar la lógica del sistema** de manera eficiente.

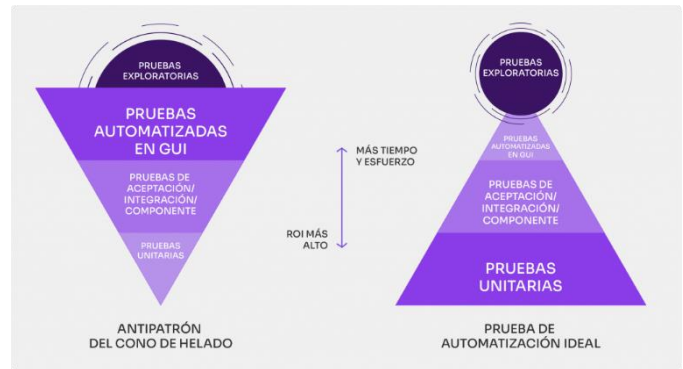
3. Nivel Superior: Pruebas de GUI

- **Menos ejecutadas y costosas**, se centran en asegurar el **correcto funcionamiento de la interfaz de usuario**.
- **Automatiza solo pruebas críticas**, desde el inicio de sesión hasta la acción final.
- **Se complementan con pruebas manuales** y exploratorias para mitigar falsos negativos y positivos.

La pirámide proporciona una **base sólida** en **pruebas unitarias**, permitiendo la construcción de pruebas más avanzadas en fases posteriores. Se destaca la importancia de seguir este enfoque piramidal para lograr un software de alta calidad en entornos ágiles. La adopción completa de los valores ágiles, como el cambio, la innovación y la creación de valor, es crucial para el éxito de metodologías ágiles como Scrum.

¿Por qué automatizar testing?

La automatización de pruebas de software **mejora la velocidad, precisión y eficiencia** al **eliminar tareas** repetitivas y que consumen tiempo en las pruebas de funciones del software. Esto **posibilita que los equipos se mantengan al día con el rápido ritmo de desarrollo**, **liberando tiempo** para actividades más estratégicas y garantizando una entrega de software más rápida y de alta calidad.



ROLES Y COMPETENCIAS DEL TESTER

1. **Tester:** El rol principal del tester implica la **identificación de errores y defectos** en el software a través de **pruebas exhaustivas**. Debe poseer **habilidades técnicas sólidas** para comprender los entresijos del software y realizar evaluaciones minuciosas. La **atención al detalle** es crucial, ya que **se espera que documente y comunique con precisión** cualquier problema identificado durante las pruebas.
2. **Líder de Testing:** El líder de testing asume un papel más **estratégico**, **supervisando** y **coordinando** el **equipo de testing**. Este rol implica la **planificación de estrategias de prueba**, **asignación de tareas**, y **garantizar la ejecución eficiente de los procesos de testing**. Además de habilidades técnicas, se requieren **habilidades de gestión** para **guiar al equipo** hacia el cumplimiento de los objetivos.
3. **Consultor de Testing:** El consultor de testing aporta un enfoque más orientado a la **consultoría**. Debe comprender los **objetivos del negocio** y las **necesidades del cliente** para **diseñar estrategias de prueba efectivas**. Este rol a menudo implica **colaborar con otros departamentos** para garantizar que las pruebas estén alineadas con los requisitos comerciales y de usuario final.

SER ÁGIL Y HACER ÁGIL

Porque no terminar con la típica pregunta de parcial y final: **¿Cuál es la diferencia de ser ágil y hacer ágil?**

"Ser ágil" implica **adoptar una mentalidad y comportamiento alineados** con los **valores y principios ágiles**, lo que lleva a **beneficios** como la **satisfacción del cliente**, **compromiso**, **innovación** y **aprendizaje continuo**. Esta perspectiva se enfoca en un **cambio cultural a nivel organizacional**, involucrando a todos los niveles de liderazgo.

En contraste, **"Hacer ágil"** se refiere a la **implementación de prácticas y métodos ágiles**, como **rituales (ke)** y **ceremonias específicas**. Aunque estas prácticas son indicativos de "Hacer ágil", **no garantizan un entorno de trabajo ágil**, equipos de alto rendimiento o una mejora significativa en la calidad del producto.

La transición exitosa hacia la agilidad se logra mejor cuando **"Hacer ágil"** se **complementa con "Ser ágil"**. Esto implica **confiar en los equipos**, **practicar la transparencia**, **gestionar indicadores de progreso** de manera diferente y **adoptar una mentalidad ágil** en la cultura organizacional. Se destaca la necesidad de un **enfoque integral**, que vaya **más allá de la implementación** de prácticas ágiles, para lograr una transformación cultural exitosa. Se recomienda buscar apoyo de expertos para guiar este proceso de cambio.